

CS2 AI COACHING SYSTEM

# Ultimate CS2 Coach

---

## Part 3 — Application, UI, Tools and Build

Application Logic, UI Qt/PySide6, Ingestion Pipeline, 17 Diagnostic Tools, 81  
Test Files, Remediation

**Autore: Renan Augusto Macena**

Marzo 2026

# Ultimate CS2 Coach — Part 3: Program, UI, Tools, and Build

---

**Topics:** Complete database schema (three-tier storage, SQLite WAL, SQLAlchemy ORM), Training regime and maturity gates (CALIBRATING/LEARNING/MATURE), Loss function catalog, Complete Program Logic (from launch to advice): Session Engine, Digester, Teacher, Hunter, Pulse; Qt/PySide6 desktop interface (13 screens, ViewModels, Qt Signals); Ingestion pipeline (demo and pro); Unified control console; New user onboarding; Storage architecture; Playback engine and tactical viewer; Spatial data and maps; Observability and logging; Reporting; Quotas and limits; Fault tolerance; Complete user journey (4 flows); Tool suite (validation and diagnostics); Test suite; Pre-commit hooks; Build, packaging, deployment; Alembic migrations; HLTV sync service; RASP Guard; MatchVisualizer; Runtime configuration files; Root-level entry points; Glossary.

**Author:** Renan Augusto Macena

This document is the continuation of **Part 2** — *Services, Analysis, and Database*, which covers coaching services, analysis engines, knowledge, and the Database. Part 3 drops down to the program level: how the entire system boots, how the daemons orchestrate, how the desktop interface makes coaching visible, how the code is tested, validated, and deployed.

---

## Table of Contents

---

### Part 3 — Program, UI, Tools, and Build (this document)

- 9. [Database schema and data lifecycle](#)
- 10. [Training regime and maturity gates](#)
- 11. [Loss function catalog](#)
- 12. [Complete Program Logic — From Launch to Advice](#)
  - 12.1 Application startup and bootstrap
  - 12.2 Session Engine and Quad-Daemon
  - 12.3 Digester Daemon and ingestion pipeline
  - 12.4 Teacher Daemon and training orchestrator
  - 12.5 Desktop Interface (Qt/PySide6, 13 screens, ViewModels)
  - 12.6 Ingestion Pipeline ( [ingestion/](#) )

- 12.7 Unified Control Console ( `backend/control/` )
- 12.8 Onboarding and New User Flow
- 12.9 Storage Architecture ( `backend/storage/` )
- 12.10 Playback Engine and Tactical Viewer
- 12.11 Spatial Data and Map Management
- 12.12 Observability and Logging
- 12.13 Reporting and Visualization
- 12.14 Quota and Limit Management
- 12.15 Fault Tolerance and Recovery
- 12.16 Complete User Journey — 4 Main Flows
- 12.17 Tool Suite — Validation and Diagnostics ( `tools/` )
- 12.18 Test Suite Architecture ( `tests/` )
- 12.19 The 12 Phases of Systematic Remediation
- 12.20 Pre-commit Hooks and Quality Gates
- 12.21 Build, Packaging, and Deployment
- 12.22 Alembic Migration System
- 12.23 Main Ingestion Orchestrator ( `run_ingestion.py` )
- 12.24 HLTV Sync Service and Background Daemon
- 12.25 RASP Guard — Runtime Code Integrity
- 12.26 MatchVisualizer — Advanced Rendering
- 12.27 Runtime Data and Configuration Files
- 12.28 Root-Level Entry Points
- [Architectural Summary](#)
- [Cross-Part Interconnection Map](#)
- [Technical Glossary](#)

---

## 9. Database schema and data lifecycle

---

The project uses **SQLModel** (Pydantic + SQLAlchemy) with SQLite (WAL mode) and a specialized **three-tier storage architecture**. In total **24 SQLModel tables** ( `backend/storage/db_models.py` : 21 monolith+HLTV tables; `backend/storage/match_data_manager.py` : 3 per-match tables) distributed across 3 storage tiers:

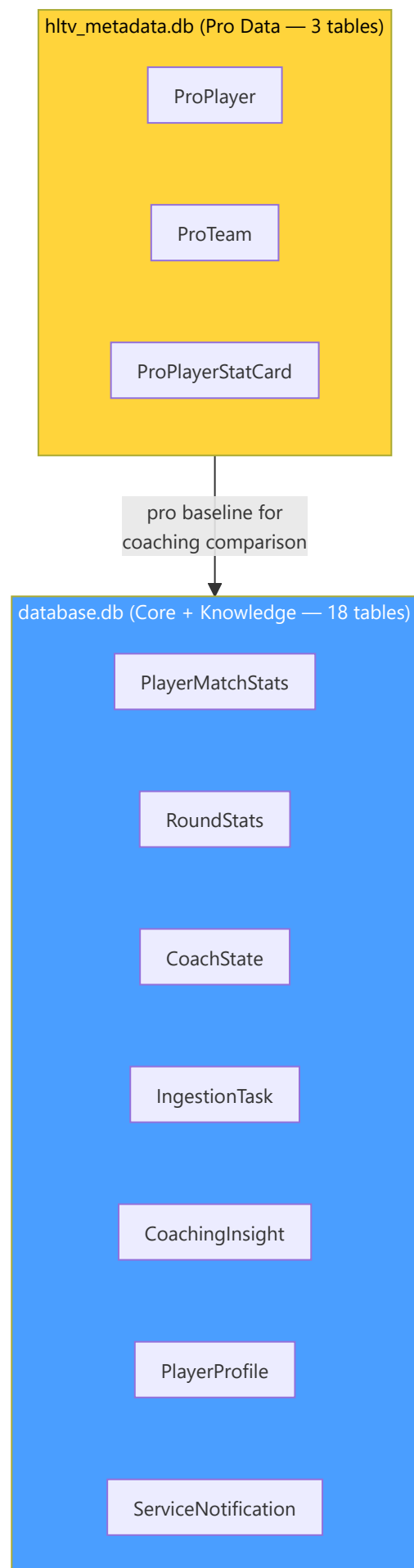
1. `database.db` — Main monolithic application database (**18 tables**, explicitly listed in `database.py:_MONOLITH_TABLES` lines 54-73). Contains all core tables: player statistics ( `PlayerMatchStats` ), coach state ( `CoachState` ), ingestion tasks ( `IngestionTask` ), coaching insights ( `CoachingInsight` ), user profiles ( `PlayerProfile` ), system notifications ( `ServiceNotification` ), RAG base ( `TacticalKnowledge` ), COPER experience bank ( `CoachingExperience` ), match results

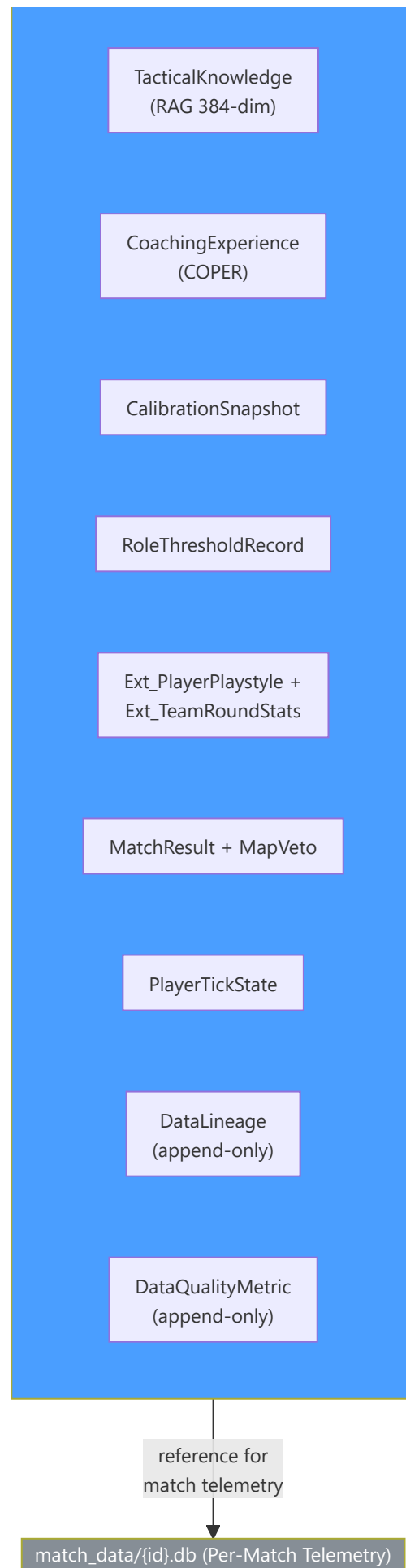
( `MatchResult` , `MapVeto` ), calibrations ( `CalibrationSnapshot` ), role thresholds ( `RoleThresholdRecord` ), provenance tracking ( `DataLineage` , `DataQualityMetric` ), and extended tables for team round ( `Ext_TeamRoundStats` ) and playstyle ( `Ext_PlayerPlaystyle` ), plus archival `RoundStats` and `PlayerTickState` .

2. `hltv_metadata.db` — Professional metadata database (**3 tables**, `database.py:HLTV_TABLES` lines 78-82): pro player profiles ( `ProPlayer` , `ProTeam` ) and stat cards ( `ProPlayerStatCard` ). Kept separate from the monolith because it is written by a separate process (HLTV sync service) to eliminate WAL contention with the session engine daemons.
3. `match_data/{id}.db` — Per-match telemetry database (**3 tables**, defined in `match_data_manager.py` : `MatchTickState` :110, `MatchEventState` :190, `MatchMetadata` :242). Each match has its own dedicated SQLite file containing tick-by-tick data (~100,000 rows per match). Managed by `MatchDataManager` . This separation solves the "Telemetry Cliff" problem — it prevents the monolith database from growing indefinitely with high-frequency data.

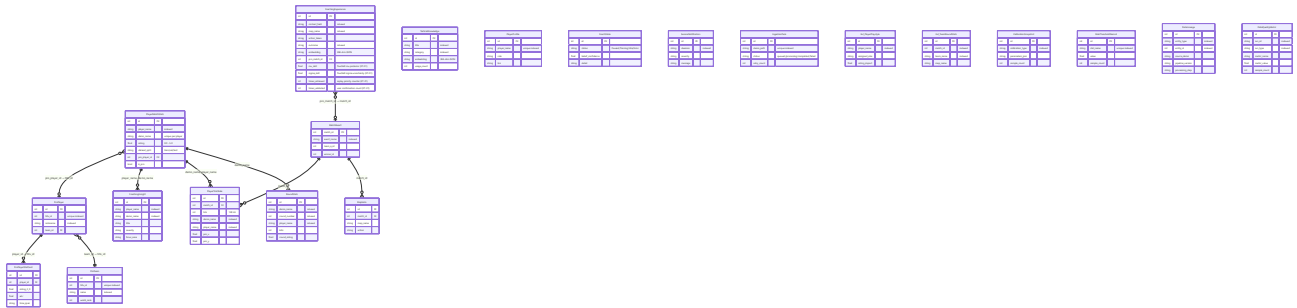
This three-tier separation ensures that the session engine's intensive write operations (demo ingestion, ML training → `database.db` ) do not contend for WAL locks with HLTV scraping in a separate process ( `hltv_metadata.db` ), and that high-frequency per-match telemetry does not bloat the monolith ( `match_data/{id}.db` ).







PlayerTickState  
(~100,000 rows/match)  
Position, health, weapon  
every 1/128 second



**Table descriptions:** **PlayerMatchStats** — aggregated per-player per-match statistics (kills, deaths, ADR, HLTV 2.0 rating, 25 normalized features). **PlayerTickState** — tick-by-tick telemetry at 64/128 Hz: position (x,y,z), health, armor, view angle, current weapon, economy, state for each player in each tick. **RoundStats** — isolated per-round statistics: kills, deaths, damage, noscope kills, flash assists, round rating, useful for granular coaching. **CoachingExperience** — COPER record: coaching moment context, advice given, outcome (positive/negative/neutral), numerical effectiveness, 384-dim embedding for semantic search. **CoachingInsight** — coaching insights generated by the system and shown to the user in the UI. **TacticalKnowledge** — RAG knowledge base: strategies, positioning, utility usage, indexed with Sentence-BERT 384-dim embeddings, version v3, 14 categories. **RoleThresholdRecord** — learned thresholds for the 10 tactical role classification, persisted across restarts. **CalibrationSnapshot** — timestamp and sample count of each Bayesian death estimator auto-calibration. **Ext\_PlayerPlaystyle** — playstyle metrics from external CSVs, used for NeuralRoleHead training. **ServiceNotification** — error/event messages from background daemons, shown in the UI via polling. **DataLineage** — append-only audit trail: traces which demo originated each entity and through which pipeline step it was processed. **DataQualityMetric** — quantitative quality metrics per pipeline run (percentage of discarded samples, zero-tensor fallback rate).

## Data lifecycle:

| Phase                    | Tables written                                                                            | Volume                            |
|--------------------------|-------------------------------------------------------------------------------------------|-----------------------------------|
| Demo ingestion           | <code>PlayerMatchStats</code> , <code>PlayerTickState</code> , <code>MatchMetadata</code> | ~100,000 ticks/match              |
| Round enrichment         | <code>RoundStats</code> (per-round, per-player isolation)                                 | ~30 rows/match (rounds × players) |
| HLTV scan                | <code>ProPlayer</code> , <code>ProTeam</code> , <code>ProPlayerStatCard</code>            | ~500 players                      |
| CSV import               | External tables via <code>csv_migrator.py</code>                                          | ~10,000 rows                      |
| Playstyle data           | <code>Ext_PlayerPlaystyle</code> (from CSV for NeuralRoleHead)                            | ~300+ players                     |
| Feature engineering      | <code>PlayerMatchStats.dataset_split</code> updated                                       | In place                          |
| RAG population           | <code>TacticalKnowledge</code>                                                            | ~200 articles                     |
| Experience extraction    | <code>CoachingExperience</code>                                                           | ~1,000 per match                  |
| Coaching output          | <code>CoachingInsight</code>                                                              | ~5-20 per match                   |
| Role threshold learning  | <code>RoleThresholdRecord</code>                                                          | 9 thresholds                      |
| Belief calibration       | <code>CalibrationSnapshot</code> (after retraining)                                       | 1 per calibration run             |
| System telemetry         | <code>CoachState</code> , <code>ServiceNotification</code> , <code>IngestionTask</code>   | Continuous                        |
| Provenance tracking      | <code>DataLineage</code> (append-only per processed entity)                               | ~N rows per ingested demo         |
| Pipeline quality metrics | <code>DataQualityMetric</code> (append-only per pipeline run)                             | ~5-10 metrics per run             |
| Backup                   | Automated via <code>BackupManager</code> (7 daily rotations + 4 weekly)                   | Full database copy                |

### Indexes and query optimization:

The most queried tables have strategic indexes to guarantee fast queries:

| Table             | Index             | Columns                 | Query type optimized           |
|-------------------|-------------------|-------------------------|--------------------------------|
| PlayerMatchStats  | idx_pms_player    | player_name             | Lookup by player               |
| PlayerMatchStats  | idx_pms_demo      | demo_name               | Lookup by match                |
| PlayerMatchStats  | idx_pms_processed | processed_at            | Chronological sort             |
| RoundStats        | idx_rs_demo_round | demo_name, round_number | Lookup for specific round      |
| IngestionTask     | idx_it_status     | status                  | Work queue (status=queued)     |
| CoachingInsight   | idx_ci_player     | player_name             | Insights per player            |
| TacticalKnowledge | idx_tk_category   | category                | RAG lookup by category         |
| ProPlayer         | idx_pp_name       | nickname                | Pro lookup by name             |
| DataLineage       | idx_dl_entity     | entity_type, entity_id  | Provenance tracking per entity |
| DataQualityMetric | idx_dqm_run       | run_id, run_type        | Quality metrics per run        |

### Integrity constraints:

| Constraint                                                      | Tables                        | Enforcement              |
|-----------------------------------------------------------------|-------------------------------|--------------------------|
| player_name NOT NULL                                            | PlayerMatchStats, RoundStats  | Schema-level             |
| demo_name UNIQUE per player                                     | PlayerMatchStats              | Prevents duplicates      |
| status CHECK IN ('queued', 'processing', 'completed', 'failed') | IngestionTask                 | Enum enforcement         |
| dataset_split CHECK IN ('train', 'val', 'test')                 | PlayerMatchStats              | Split validity           |
| Foreign key demo_name                                           | RoundStats → PlayerMatchStats | Round-match relationship |

### Key table details:

**PlayerMatchStats** (32 fields) — the most queried table in the system:

The PlayerMatchStats table contains all aggregated per-player, per-match statistics. It is the "report card" for every analyzed match:

| Field                                                           | Type       | Description                                 |
|-----------------------------------------------------------------|------------|---------------------------------------------|
| <code>id</code>                                                 | Integer PK | Unique identifier                           |
| <code>player_name</code>                                        | String     | Player name (indexed)                       |
| <code>demo_name</code>                                          | String     | Demo file name (unique per player)          |
| <code>kills</code> , <code>deaths</code> , <code>assists</code> | Integer    | Base KDA                                    |
| <code>adr</code>                                                | Float      | Average Damage per Round                    |
| <code>kast</code>                                               | Float      | Kill/Assist/Survive/Trade %                 |
| <code>headshot_percentage</code>                                | Float      | HS%                                         |
| <code>hltv_rating</code>                                        | Float      | Computed HLTV 2.0 Rating                    |
| <code>dataset_split</code>                                      | String     | "train" / "val" / "test"                    |
| <code>is_pro</code>                                             | Boolean    | Pro player flag                             |
| <code>processed_at</code>                                       | DateTime   | Processing timestamp                        |
| <code>map_name</code>                                           | String     | Map played                                  |
| ...                                                             | ...        | 20+ additional fields for advanced features |

**TacticalKnowledge** — the RAG base:

| Field                    | Type       | Description                                                    |
|--------------------------|------------|----------------------------------------------------------------|
| <code>id</code>          | Integer PK | Identifier                                                     |
| <code>title</code>       | String     | Document title (indexed)                                       |
| <code>description</code> | String     | Description of tactical content                                |
| <code>category</code>    | String     | "positioning" / "economy" / "utility" / "aim" (indexed)        |
| <code>map_name</code>    | String     | Specific map (indexed, optional)                               |
| <code>situation</code>   | String     | Situational context: "T-side pistol round", "CT retake A site" |
| <code>pro_example</code> | String     | Reference to pro demo (optional)                               |
| <code>embedding</code>   | String     | 384-dim JSON vector (sentence-transformers)                    |
| <code>created_at</code>  | DateTime   | Creation timestamp                                             |
| <code>usage_count</code> | Integer    | Usage counter                                                  |

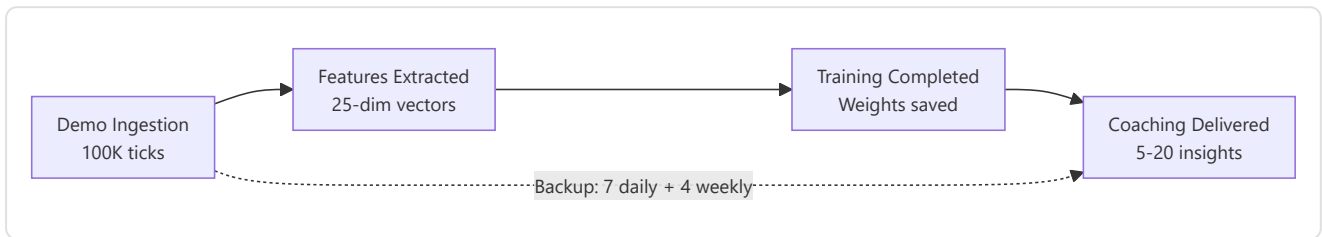
RAG semantic search works by computing the **cosine similarity** between the user query embedding and the precomputed embeddings of each document. The top-3 results with similarity > 0.5 are used to enrich the coaching context. The `situation` field enables contextual filtering (e.g., "T-side pistol round") before semantic search.

**CoachingExperience** — the COPER bank (22+ fields):

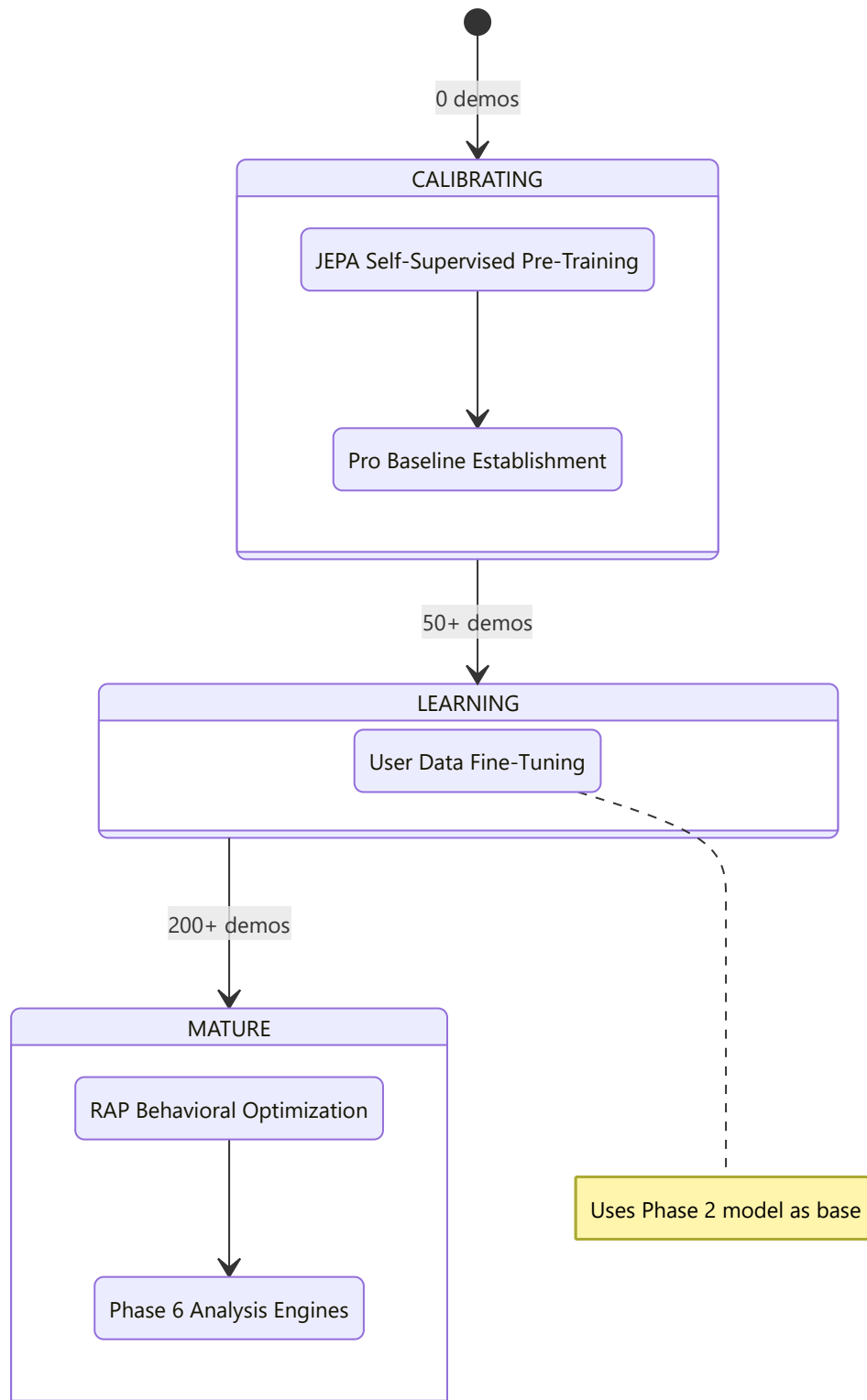
| Field                              | Type       | Description                                                                 |
|------------------------------------|------------|-----------------------------------------------------------------------------|
| <code>id</code>                    | Integer PK | Identifier                                                                  |
| <code>context_hash</code>          | String     | Hash of game state for fast lookup (indexed)                                |
| <code>map_name</code>              | String     | Map (indexed)                                                               |
| <code>round_phase</code>           | String     | "pistol" / "eco" / "full_buy" / "force"                                     |
| <code>side</code>                  | String     | "T" / "CT"                                                                  |
| <code>position_area</code>         | String     | "A-site" / "Mid" / etc. (indexed, optional)                                 |
| <code>game_state_json</code>       | String     | Full tick snapshot (max 16KB, validated with <code>field_validator</code> ) |
| <code>action_taken</code>          | String     | "pushed" / "held_angle" / "rotated" / "used_utility" / etc.                 |
| <code>outcome</code>               | String     | "kill" / "death" / "trade" / "objective" / "survived" (indexed)             |
| <code>delta_win_prob</code>        | Float      | Win probability change from this action                                     |
| <code>confidence</code>            | Float      | Reliability/generalizability 0.0-1.0                                        |
| <code>usage_count</code>           | Integer    | How many times retrieved for coaching                                       |
| <code>pro_match_id</code>          | Integer FK | Reference to MatchResult (ON DELETE SET NULL)                               |
| <code>pro_player_name</code>       | String     | Reference pro player name (indexed, optional)                               |
| <code>embedding</code>             | String     | 384-dim JSON vector for semantic search (optional)                          |
| <code>source_demo</code>           | String     | Source demo (optional)                                                      |
| <code>created_at</code>            | DateTime   | Creation timestamp                                                          |
| <code>outcome_validated</code>     | Boolean    | Whether the outcome was validated                                           |
| <code>effectiveness_score</code>   | Float      | Score -1.0 to 1.0                                                           |
| <code>follow_up_match_id</code>    | Integer    | Follow-up match for tracking                                                |
| <code>times_advice_given</code>    | Integer    | How many times the advice was given                                         |
| <code>times_advice_followed</code> | Integer    | How many times the advice was followed                                      |
| <code>last_feedback_at</code>      | DateTime   | Last feedback received                                                      |
| <code>mu_skill</code>              | Float      | TrueSkill $\mu$ posterior — experience quality estimate (KT-01)             |
| <code>sigma_skill</code>           | Float      | TrueSkill $\sigma$ uncertainty — quality uncertainty (KT-01)                |
| <code>times_retrieved</code>       | Integer    | Replay priority counter — how many times retrieved (KT-01)                  |
| <code>times_validated</code>       | Integer    | User confirmation counter — how many times validated (KT-01)                |

The COPER system uses these experiences to **learn from its own advice**: the `context_hash` field enables fast lookup of similar situations, `outcome` and `delta_win_prob` measure action effectiveness, and the feedback loop ( `outcome_validated` , `times_advice_given/followed` , `effectiveness_score` ) lets the system prioritize advice that has historically produced positive outcomes. The `game_state_json` field is capped at

16KB to prevent uncontrolled database growth. The TrueSkill `mu_skill` / `sigma_skill` fields (KT-01) provide a Bayesian estimate of experience quality, used for replay prioritization:  $\text{confidence\_score} = \text{mu\_skill} - \kappa \times \text{sigma\_skill}$ . The `times_retrieved` counter implements replay priority, while `times_validated` tracks user confirmations for semantic CRUD.

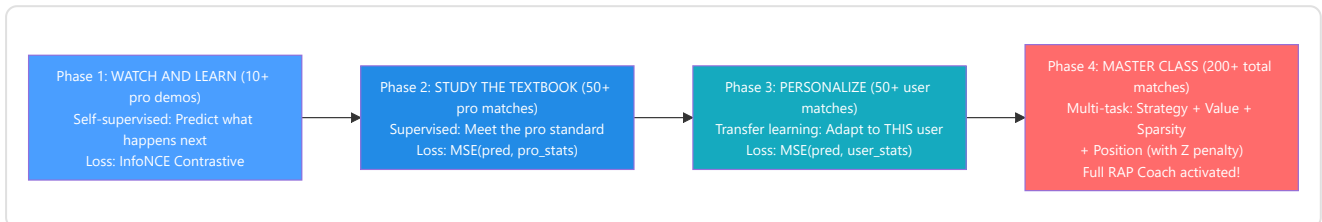


## 10. Training regime and maturity gates



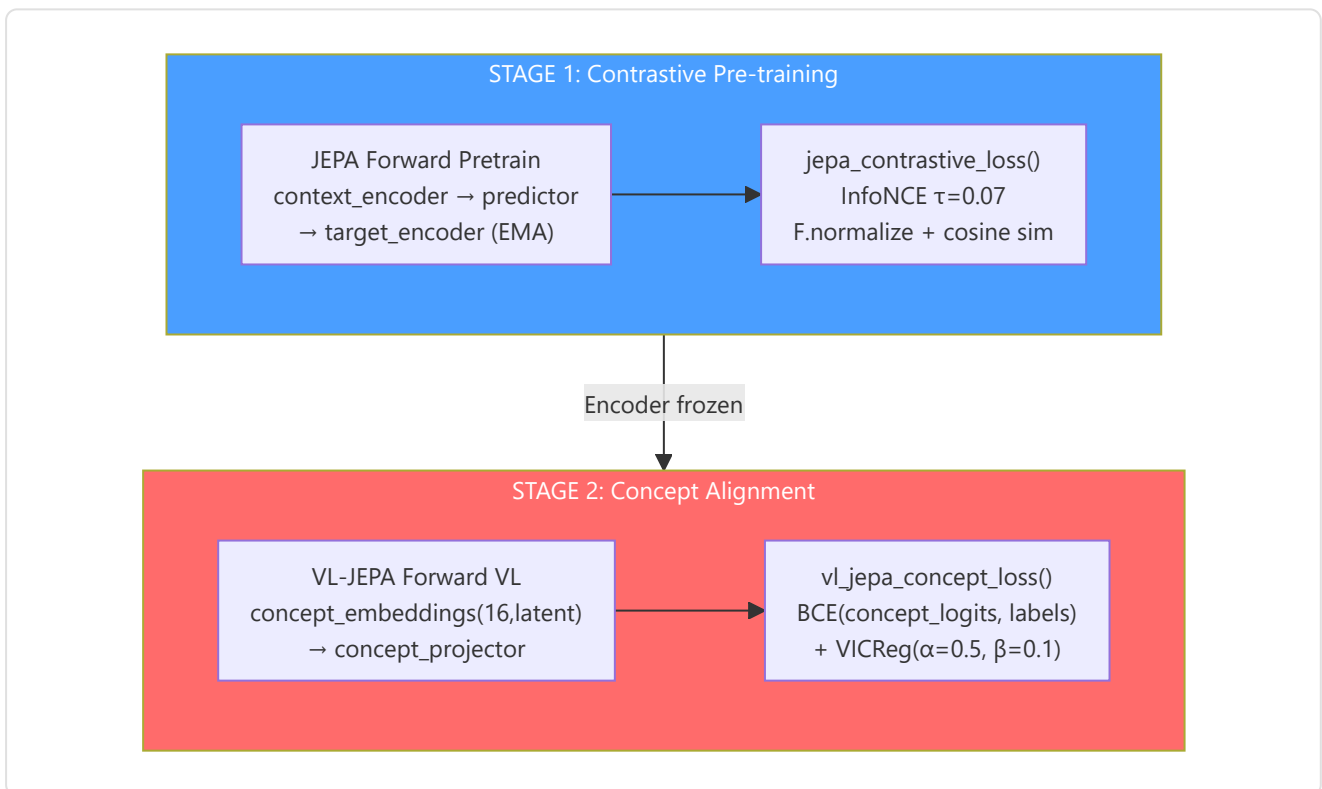
**Data requirements per phase:**

| Phase                | Minimum data      | Training type             | Primary loss                           |
|----------------------|-------------------|---------------------------|----------------------------------------|
| 1. JEPA Pre-training | 10 pro demos      | Self-supervised (InfoNCE) | Contrastive with in-batch negatives    |
| 2. Pro baseline      | 50 pro matches    | Supervised                | MSE(pred, pro_stats)                   |
| 3. User fine-tuning  | 50 user matches   | Supervised (transfer)     | MSE(pred, user_stats)                  |
| 4. RAP optimization  | 200 total matches | Multi-task                | Strategy + Value + Sparsity + Position |



### VL-JEPA Two-Stage Protocol (Concept Alignment):

When VL-JEPA is active, Phases 1-2 are extended with a **two-stage protocol** that aligns latent representations with the 16 coaching concepts:



| Stage | What is trained                                            | What is frozen                 | Loss                    | Purpose                             |
|-------|------------------------------------------------------------|--------------------------------|-------------------------|-------------------------------------|
| 1     | Context encoder, predictor                                 | Target encoder (EMA)           | InfoNCE ( $\tau=0.07$ ) | General latent representations      |
| 2     | Concept embeddings, concept projector, concept temperature | Encoder (optional fine-tuning) | BCE + VICReg diversity  | Alignment with 16 coaching concepts |

**The 16 Coaching Concepts (taxonomy):**

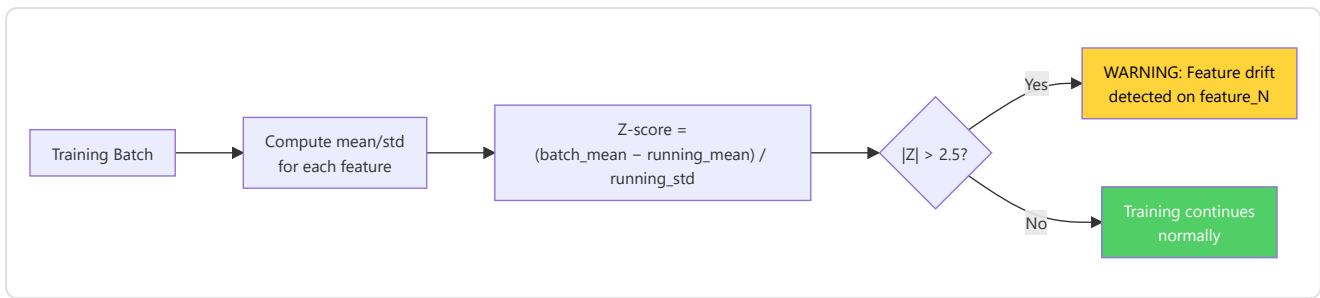
| Index | Concept                  | Category    | Description                                |
|-------|--------------------------|-------------|--------------------------------------------|
| 0     | Positioning Quality      | Positioning | Position quality relative to context       |
| 1     | Trade Readiness          | Positioning | Readiness for trade kill                   |
| 2     | Rotation Speed           | Positioning | Rotation speed between sites               |
| 3     | Utility Usage            | Utility     | Grenade use frequency and quality          |
| 4     | Utility Effectiveness    | Utility     | Effectiveness of utilities used            |
| 5     | Decision Quality         | Decision    | In-game decision quality                   |
| 6     | Risk Assessment          | Decision    | Pre-action risk evaluation                 |
| 7     | Engagement Timing        | Engagement  | Timing of engagements                      |
| 8     | Engagement Distance      | Engagement  | Optimal engagement distance                |
| 9     | Crosshair Placement      | Engagement  | Pre-peek crosshair placement               |
| 10    | Recoil Control           | Engagement  | Recoil control                             |
| 11    | Economy Management       | Decision    | Team economy management                    |
| 12    | Information Gathering    | Decision    | Information gathering (peek, info utility) |
| 13    | Composure Under Pressure | Psychology  | Composure under pressure                   |
| 14    | Aggression Control       | Psychology  | Aggression control                         |
| 15    | Adaptation Speed         | Psychology  | Speed of adaptation to opponent meta       |

**AdamW + CosineAnnealing (JEPA Trainer):**

| Hyperparameter | Value             | Purpose                               |
|----------------|-------------------|---------------------------------------|
| Optimizer      | AdamW             | Weight decay separated from gradients |
| Learning rate  | 1e-4 (default)    | Initial learning rate                 |
| Weight decay   | 0.01              | L2 regularization                     |
| Scheduler      | CosineAnnealingLR | Cosine LR decay down to 0             |
| EMA decay      | 0.996             | Target encoder momentum update        |
| Gradient clip  | 1.0               | Gradient explosion prevention         |

**DriftMonitor (z\_threshold=2.5):**

The `DriftMonitor` integrated in the JEPA Trainer monitors **feature drift** during training. If a feature's Z-score exceeds 2.5, it emits a warning indicating possible distribution shift:



**Retraining trigger:** The Teacher daemon monitors growth in the pro demo count; it triggers retraining when `count ≥ last_count × 1.10`.

**JEPAPretrainDataset** ( `jepa_train.py` ):

The JEPA pre-training dataset uses **temporal windows** to create context-target pairs:

| Parameter                | Value              | Purpose                                      |
|--------------------------|--------------------|----------------------------------------------|
| <code>context_len</code> | 10 ticks           | Context window length (input)                |
| <code>target_len</code>  | 10 ticks           | Target window length (to predict)            |
| Gap                      | 0-5 ticks (random) | Variable distance between context and target |
| Batch size               | 32                 | Number of pairs per batch                    |

## 11. Loss function catalog

| Model                  | Loss name              | Formula                                                                                                                                                                              | Purpose                                             |
|------------------------|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| <b>JEPA</b>            | InfoNCE<br>Contrastive | $-\log(\exp(\text{sim}(\text{pred}, \text{target})/\tau) / \sum \exp(\text{sim}(\text{pred}, \text{neg}_i)/\tau))$ , $\tau=0.07$ , <code>F.normalize</code> before cosine similarity | Aligning context predictions with target embeddings |
| <b>JEPA</b>            | Fine-tuning            | <code>MSE(coaching_head(Z_ctx), y_true)</code>                                                                                                                                       | Supervised coaching score                           |
| <b>AdvancedCoachNN</b> | Supervised             | <code>MSELoss(MoE_output, y_true)</code>                                                                                                                                             | Match-level training                                |
| <b>RAP</b>             | Strategy               | <code>MSELoss(advice_probs, target_strat)</code>                                                                                                                                     | Correct tactical recommendation                     |
| <b>RAP</b>             | Value                  | $0.5 \times \text{MSE}(V(s), \text{true\_advantage})$                                                                                                                                | Accurate advantage estimation                       |
| <b>RAP</b>             | Sparsity               | <code>L1(gate_weights)</code>                                                                                                                                                        | Expert specialization                               |
| <b>RAP</b>             | Position               | <code>MSE(xy) + 2 × MSE(z)</code>                                                                                                                                                    | Optimal positioning with Z-axis penalty             |
| <b>WinProb</b>         | Prediction             | <code>BCEWithLogitsLoss(pred, outcome)</code>                                                                                                                                        | Round outcome prediction                            |
| <b>NeuralRoleHead</b>  | KL-Divergence          | <code>KLDivLoss(log_softmax(pred), target)</code> with label smoothing $\epsilon=0.02$                                                                                               | Role probability distribution match                 |
| <b>VL-JEPA</b>         | Concept Alignment      | <code>BCE(concept_logits, concept_labels) + VICReg(concept_diversity)</code>                                                                                                         | Visual-language concept grounding                   |

### Detail: InfoNCE Contrastive Loss (JEPA)

InfoNCE is the main loss for JEPA pre-training. Its purpose is to align context predictions with target embeddings while simultaneously pushing away negatives (other samples in the batch):

$$L_{\text{InfoNCE}} = -\log\left(\frac{\exp(\text{sim}(\text{pred}, \text{target}^+))}{\tau} / \sum_i \frac{\exp(\text{sim}(\text{pred}, \text{target}_i))}{\tau}\right)$$

| Component                       | Value                                            | Role                                                 |
|---------------------------------|--------------------------------------------------|------------------------------------------------------|
| <code>sim()</code>              | Cosine similarity after <code>F.normalize</code> | Similarity measure [-1, +1]                          |
| $\tau$<br>(temperature)         | 0.07                                             | Distribution sharpness — low values = more selective |
| <code>target<sup>+</sup></code> | The correct target embedding for this context    | The "positive" — the correct answer                  |
| <code>target<sub>i</sub></code> | All embeddings in the batch                      | In-batch negatives — the wrong answers               |
| Batch size                      | 32                                               | Number of negatives = batch_size - 1 = 31            |

### Detail: VL-JEPA Concept Loss (VICReg Components)

The VL-JEPA concept alignment loss combines two components:

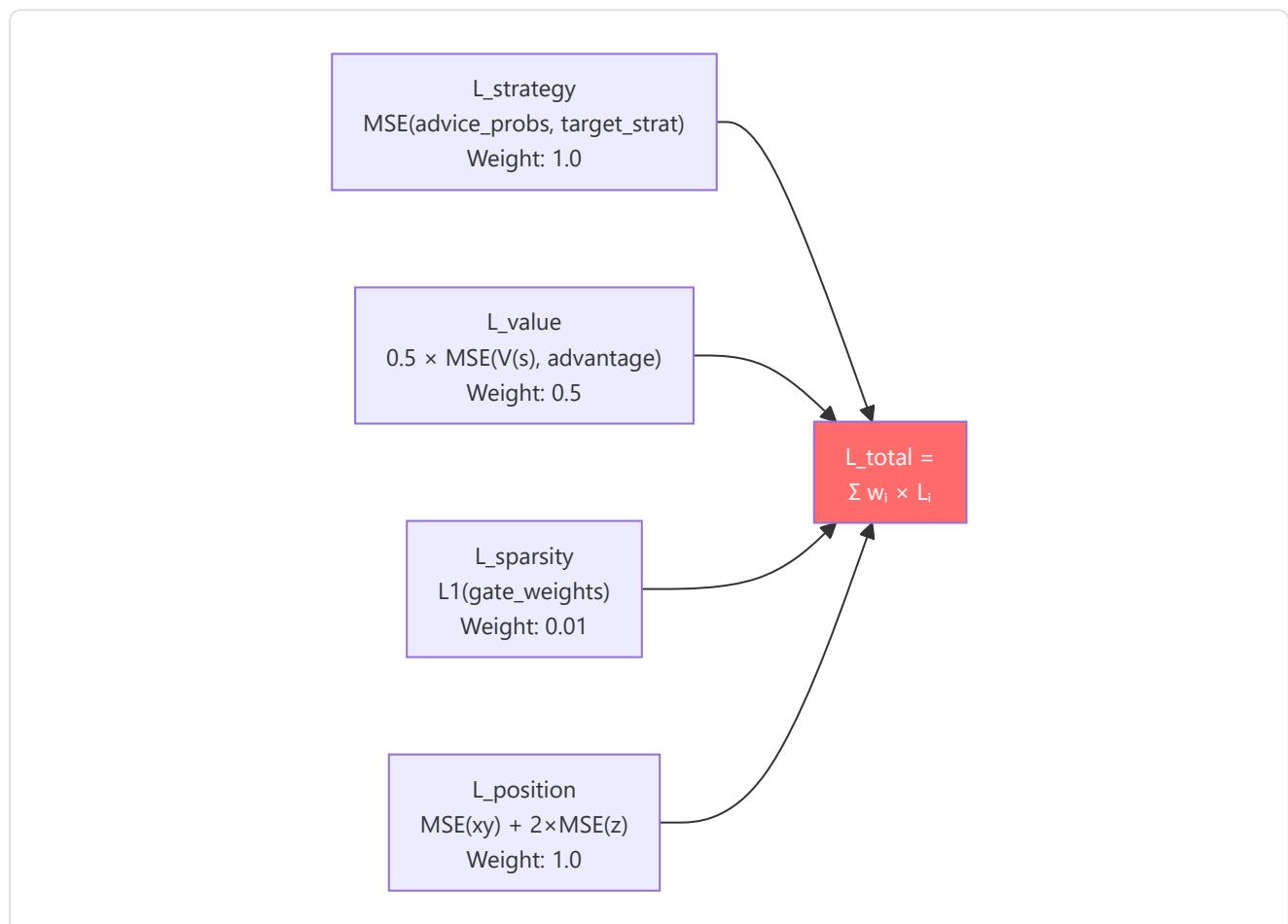
$$L_{\text{concept}} = \text{BCE}(\text{concept\_logits}, \text{concept\_labels}) + \alpha \cdot \text{VICReg\_diversity}$$

Where `VICReg_diversity` is composed of:

| VICReg Term       | Formula                                              | Weight       | Purpose                                                           |
|-------------------|------------------------------------------------------|--------------|-------------------------------------------------------------------|
| <b>Variance</b>   | $\max(0, \gamma - \text{std}(z))$ for each dimension | $\alpha=0.5$ | Prevents representation collapse — each dimension must vary       |
| <b>Covariance</b> | $\sum_{i \neq j} \text{cov}(z_i, z_j)^2$             | $\beta=0.1$  | Decorrelation — each dimension must capture different information |

### Detail: RAP Multi-Task Loss

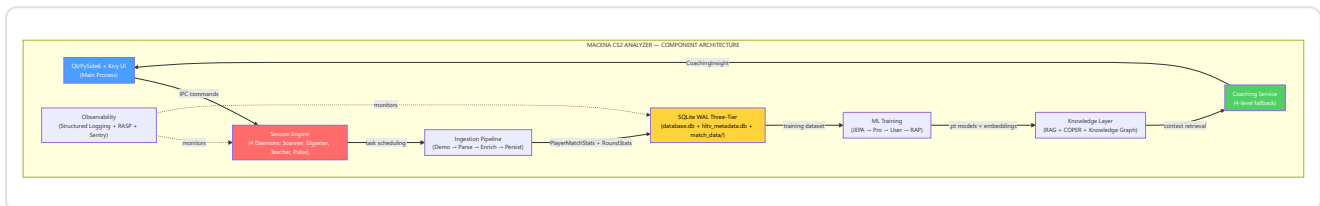
The RAP Coach combines 4 losses into a total weighted loss:



The  $2 \times Z$  penalty in position loss reflects the fact that in CS2 getting the **floor** wrong (upper vs lower in Nuke/Vertigo) is much worse than getting horizontal position wrong. A 100-unit error on the Z axis (wrong floor) is strategically catastrophic, while the same error on X/Y could be irrelevant.

## 12. Complete Program Logic — From Launch to Advice

This chapter documents the **complete logic** of Macena CS2 Analyzer, from the moment the user launches the application to when they receive coaching advice. Unlike the previous chapters that focus on AI subsystems, here it is explained how **every component of the program** works together: the desktop interface, the quad-daemon architecture, the ingestion pipeline, the storage system, tactical playback, observability, and the application lifecycle.



### 12.1 Entry Point and Startup Sequence

The system has **two main entry points** (Qt primary, Kivy legacy) and three utility entry points:

| # | Entry Point         | Command                                                             | Role                   |
|---|---------------------|---------------------------------------------------------------------|------------------------|
| 1 | <b>Qt (primary)</b> | <code>python -m Programma_CS2_RENAN.apps.qt_app.app</code>          | PySide6 desktop UI     |
| 2 | Kivy (legacy)       | <code>python -m Programma_CS2_RENAN.main</code>                     | Kivy/KivyMD desktop UI |
| 3 | Session Engine      | <code>python -m Programma_CS2_RENAN.backend.console</code>          | Headless backend       |
| 4 | Headless Validator  | <code>python -m Programma_CS2_RENAN.tools.headless_validator</code> | CI/CD validation       |
| 5 | HLTV Sync           | <code>python -m Programma_CS2_RENAN.tools.hltv_sync</code>          | Pro data scraping      |

#### 12.1.1 Qt Entry Point (Primary) — `apps/qt_app/app.py`

**File:** `Programma_CS2_RENAN/apps/qt_app/app.py`

The Qt startup sequence follows a more modern approach based on **QApplication** and **signal/slot** instead of the Kivy event loop:

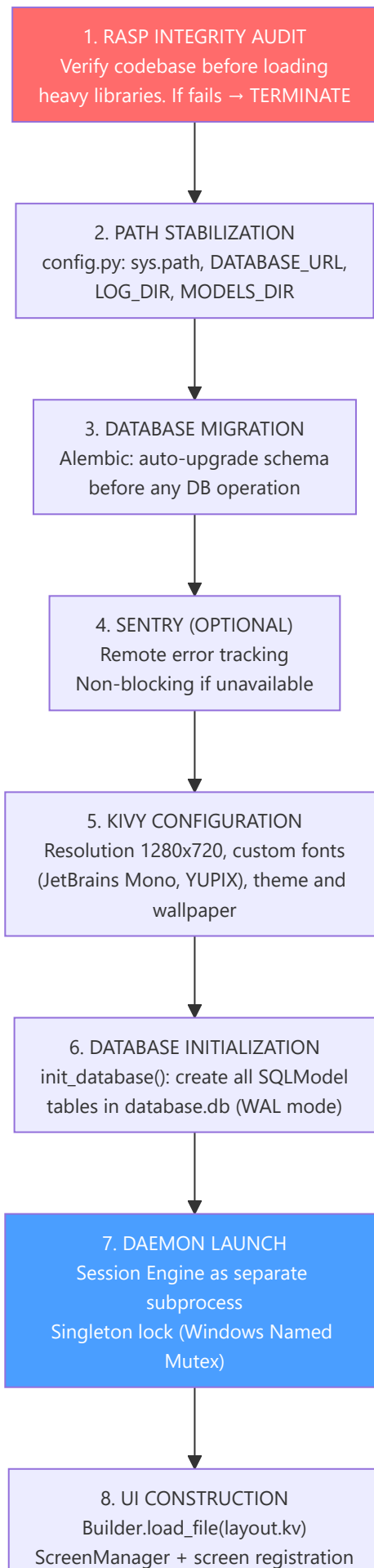
1. **High-DPI setup** — Enables automatic scaling for high-density displays
2. **QApplication** — Creates the Qt application instance with args handling
3. **Theme and fonts** — Registers the 3 themes (CS2, CSGO, CS1.6) via QSS and the custom fonts
4. **MainWindow** — Builds `QMainWindow` with sidebar + `QStackedWidget` (14 screens)
5. **Signal wiring** — Connects Qt signals between sidebar, screens, and backend
6. **First-run gate** — If `SETUP_COMPLETED=False`, shows the wizard; otherwise the home
7. **Backend console boot** — Launches the Session Engine as a subprocess
8. **Window show** — Shows the window and starts the Qt event loop
9. **CoachState polling** — Qt timer for periodic state updates

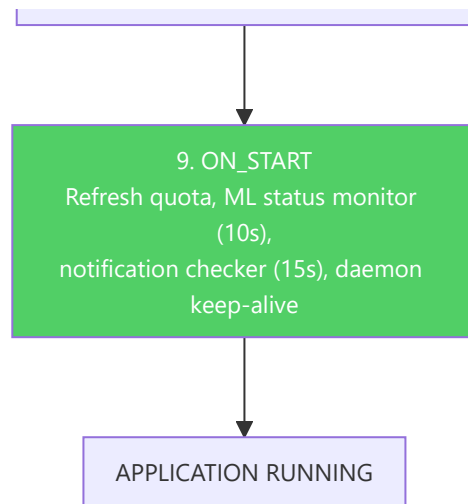
### 12.1.2 Kivy Entry Point (Legacy) — `main.py`

**File:** `Programma_CS2_RENAN/main.py`

When the user launches the application via the legacy entry point, `main.py` orchestrates a strictly ordered **9-phase startup sequence**. Each phase must complete successfully before the next can begin. If a critical phase fails, the application terminates with an explicit message — never silently.







### Critical phase details:

| Phase | Component                        | What it does                                                                         | Failure consequence                        |
|-------|----------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------|
| 1     | <code>integrity.py</code> (RASP) | Verifies source file hashes against manifest                                         | Immediate termination — possible tampering |
| 2     | <code>config.py</code>           | Stabilizes <code>sys.path</code> , defines all path constants                        | Cascading import errors                    |
| 3     | <code>db_migrate.py</code>       | Runs pending Alembic migrations                                                      | Incompatible schema — crash on DB ops      |
| 5     | Kivy <code>Config</code>         | Sets <code>KIVY_NO_CONSOLELOG</code> , registers fonts, loads <code>.kv</code>       | UI not renderable                          |
| 6     | <code>database.py</code>         | <code>create_all()</code> on SQLite engine with <code>check_same_thread=False</code> | No persistence possible                    |
| 7     | <code>lifecycle.py</code>        | Launches subprocess with correct PYTHONPATH, checks mutex                            | No background automation                   |

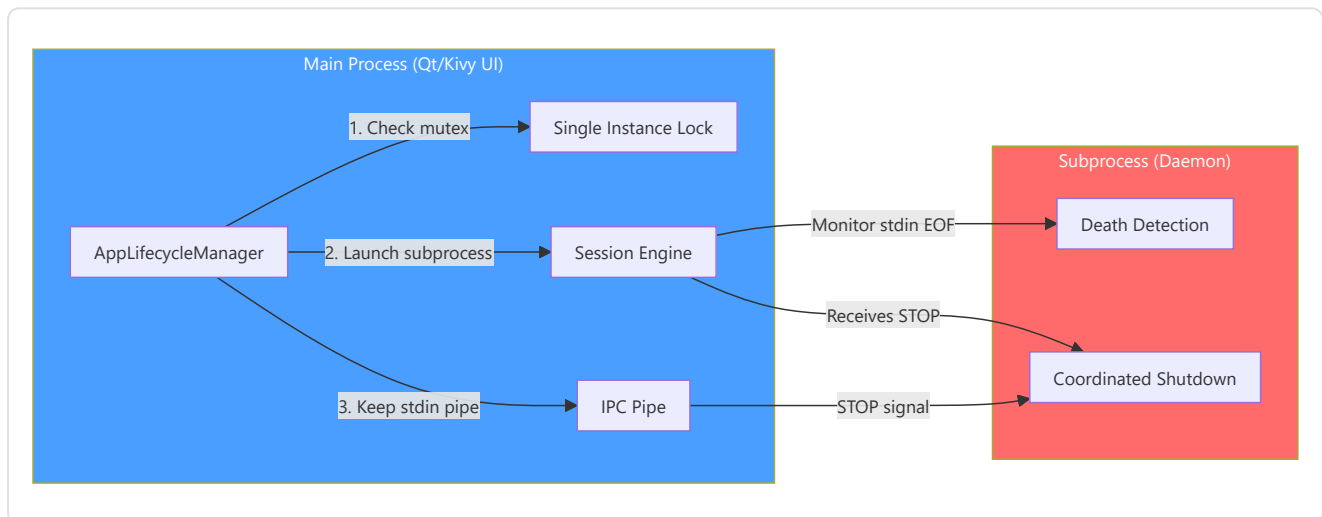
## 12.2 Lifecycle Management ( `lifecycle.py` )

**File:** `Programma_CS2_RENAN/core/lifecycle.py`

The `AppLifecycleManager` is a **Singleton** that manages the lifecycle of the entire application: from guaranteeing that only one instance is active, to launching the daemon subprocess, to coordinated shutdown.

### Main mechanisms:

| Mechanism                     | Implementation                                                                | Purpose                                                    |
|-------------------------------|-------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>Single Instance Lock</b>   | Windows Named Mutex / file lock on Linux                                      | Prevents multiple instances (DB corruption)                |
| <b>Daemon Launch</b>          | <code>subprocess.Popen(session_engine.py)</code> with <code>PYTHONPATH</code> | Separate process for heavy work                            |
| <b>Parent Death Detection</b> | The daemon monitors EOF on <code>stdin</code>                                 | If the main process dies, the daemon shuts down            |
| <b>Graceful Shutdown</b>      | Sends "STOP" via <code>stdin</code> → daemon terminates within 5s             | No data loss or zombie tasks                               |
| <b>Status Polling</b>         | UI queries <code>CoachState</code> every 10s                                  | Daemon state update without direct IPC                     |
| <b>Error Recovery</b>         | If daemon dies, error logged + ServiceNotification                            | User informed, no silent crash                             |
| <b>Keep-Alive</b>             | UI checks heartbeat every 15s                                                 | If heartbeat > 30s stale → warning "Daemon not responding" |

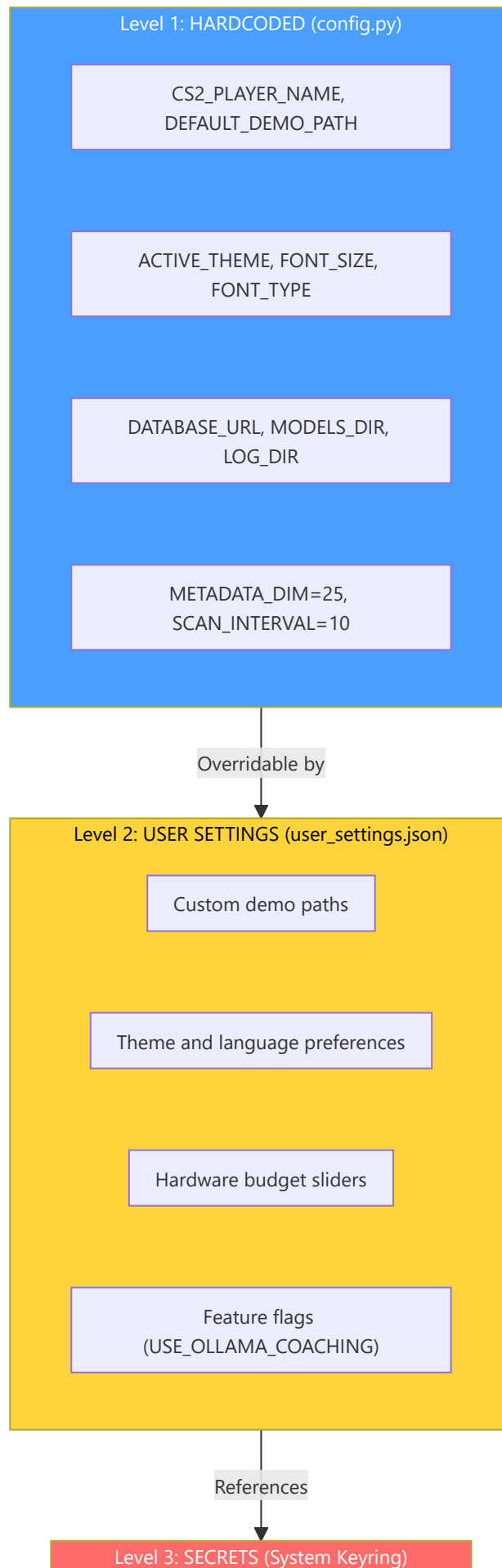


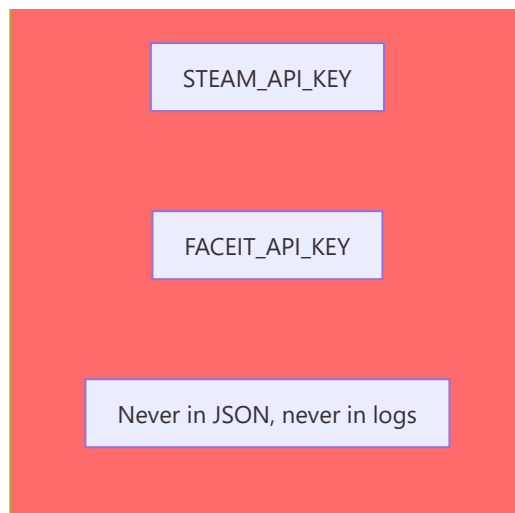
## 12.3 Configuration System ( `config.py` )

**File:** `Programma_CS2_RENAN/core/config.py`

The system uses **three configuration levels**, each with a different level of persistence and security:







### Critical system constants:

| Constant                               | Value | File                                | Purpose                                        |
|----------------------------------------|-------|-------------------------------------|------------------------------------------------|
| <code>METADATA_DIM</code>              | 25    | <code>config.py</code>              | Feature vector dimension — unified contract    |
| <code>SCAN_INTERVAL</code>             | 10    | <code>config.py</code>              | Scanner scan interval (seconds)                |
| <code>MAX_DEMOS_PER_MONTH</code>       | 10    | <code>config.py</code>              | Monthly demo upload quota                      |
| <code>MAX_TOTAL_DEMOS</code>           | 100   | <code>config.py</code>              | Lifetime total demo limit                      |
| <code>MIN_DEMOS_FOR_COACHING</code>    | 10    | <code>config.py</code>              | Threshold for full personalized coaching       |
| <code>TRADE_WINDOW_TICKS</code>        | 192   | <code>trade_kill_detector.py</code> | Trade kill time window (~3 seconds at 64 tick) |
| <code>HLTV_BASELINE_KPR</code>         | 0.679 | <code>demo_parser.py</code>         | HLTV 2.0 baseline for KPR                      |
| <code>HLTV_BASELINE_SURVIVAL</code>    | 0.317 | <code>demo_parser.py</code>         | HLTV 2.0 baseline for survival                 |
| <code>FOV_DEGREES</code>               | 90    | <code>player_knowledge.py</code>    | Simulated player field of view                 |
| <code>MEMORY_DECAY_TAU</code>          | 160   | <code>player_knowledge.py</code>    | Memory decay constant in ticks                 |
| <code>CONFIDENCE_ROUNDS_CEILING</code> | 300   | <code>correction_engine.py</code>   | Round ceiling for max confidence               |
| <code>SILENCE_THRESHOLD</code>         | 0.2   | <code>explainability.py</code>      | Threshold below which silence is valid action  |
| <code>MIN_SAMPLES_FOR_VALIDITY</code>  | 10    | <code>role_thresholds.py</code>     | Minimum samples for a valid role threshold     |
| <code>HALF_LIFE_DAYS</code>            | 90    | <code>pro_baseline.py</code>        | Temporal decay for pro data                    |
| <code>Z_LEVEL_THRESHOLD</code>         | 200   | <code>connect_map_context.py</code> | Z threshold for floor classification           |

### Path architecture:

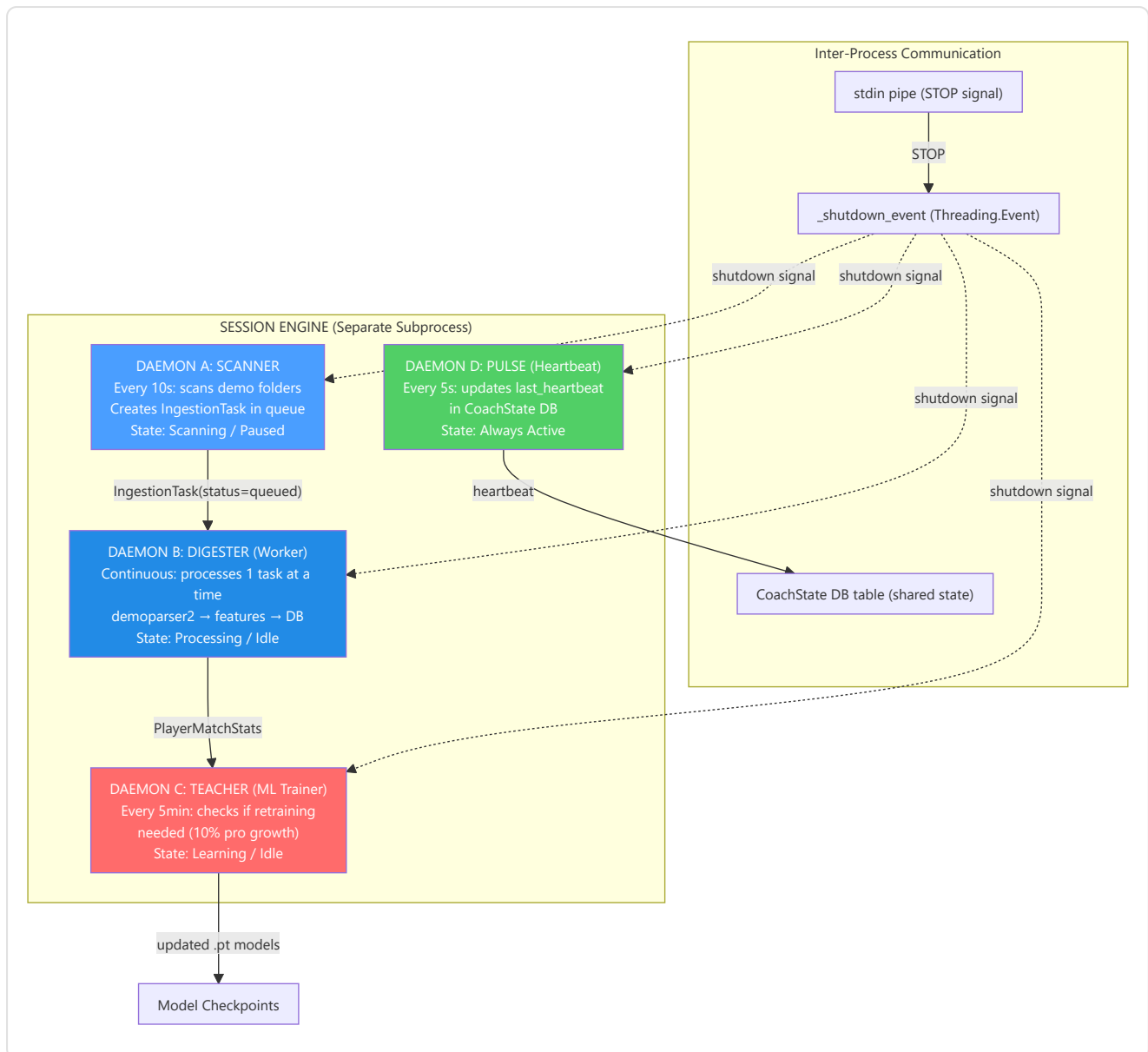
The system handles paths with particular attention to **Windows/Linux portability**. The heart is `BRAIN_DATA_ROOT` : a user-configurable directory that contains models, logs, and derived data. If it doesn't exist, the system falls back to the project folder.

| Path                           | Contents                                              | Configurable                              |
|--------------------------------|-------------------------------------------------------|-------------------------------------------|
| <code>DATABASE_URL</code>      | Main monolithic database ( <code>database.db</code> ) | No — always in the project folder         |
| <code>BRAIN_DATA_ROOT</code>   | Root for derived data (models, logs)                  | Yes — via <code>user_settings.json</code> |
| <code>MODELS_DIR</code>        | Model checkpoints <code>.pt</code>                    | Derived from <code>BRAIN_DATA_ROOT</code> |
| <code>LOG_DIR</code>           | Application log files                                 | Derived from <code>BRAIN_DATA_ROOT</code> |
| <code>MATCH_DATA_PATH</code>   | Per-match database ( <code>match_XXXX.db</code> )     | Derived from <code>BRAIN_DATA_ROOT</code> |
| <code>DEFAULT_DEMO_PATH</code> | User demo folder                                      | Yes — via UI Settings                     |
| <code>PRO_DEMO_PATH</code>     | Pro demo folder                                       | Yes — via UI Settings                     |

## 12.4 Session Engine — Quad-Daemon Architecture ( `session_engine.py` )

**File:** `Programma_CS2_RENAN/core/session_engine.py`

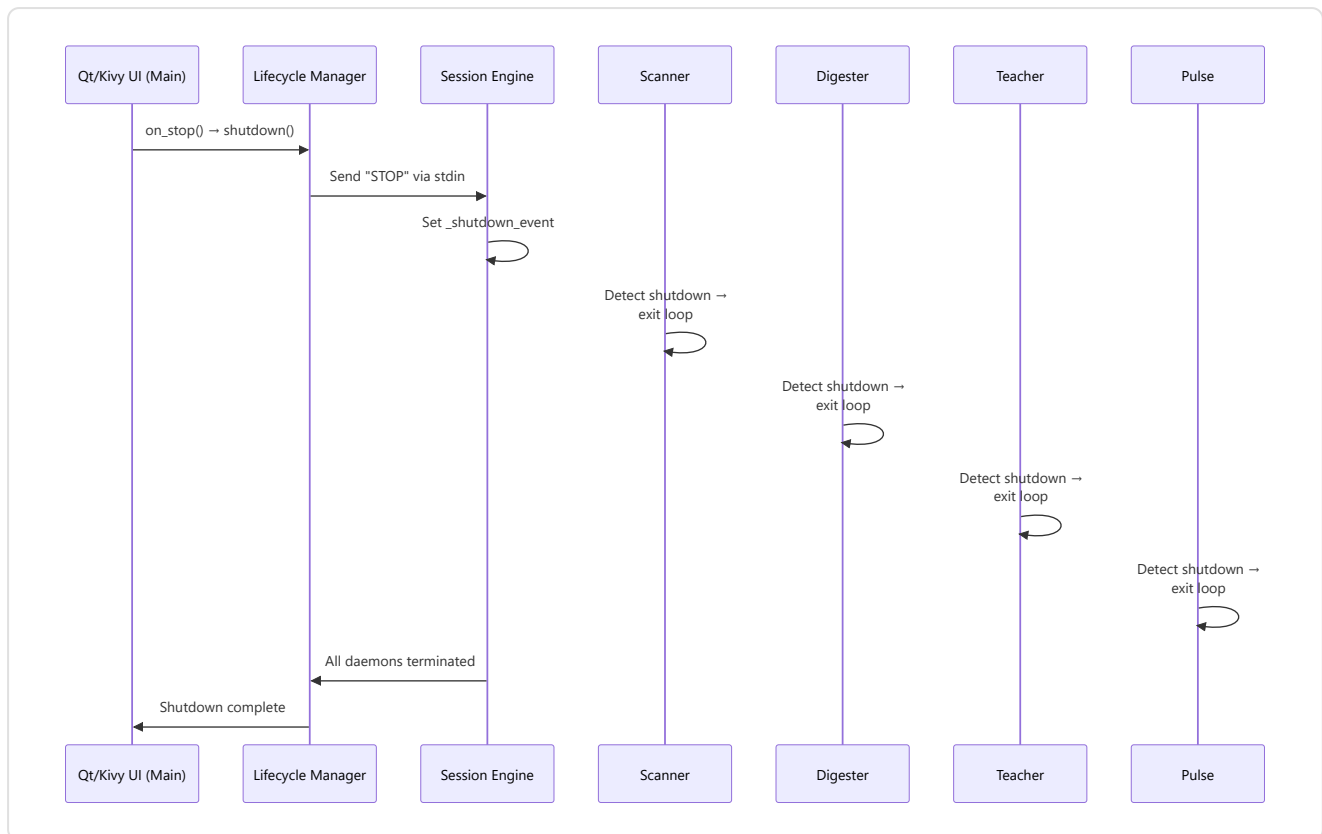
The Session Engine is the **beating heart** of the system's automation. It lives as a separate subprocess and hosts **4 daemon threads** that work in parallel, each with a well-defined responsibility. This design completely separates the heavy work (demo parsing, ML training) from the user interface, guaranteeing that the Kivy GUI always remains responsive.



### Lifecycle of each daemon:

| Daemon          | Interval            | Work per cycle                                                                           | Trigger                                                  |
|-----------------|---------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <b>Scanner</b>  | 10 seconds          | Scans pro and user folders, creates <code>IngestionTask</code> for new <code>.dem</code> | Always active (if state = Scanning)                      |
| <b>Digester</b> | Continuous          | Picks 1 task from queue, performs full parsing                                           | <code>_work_available_event</code> (signaled by Scanner) |
| <b>Teacher</b>  | 300 seconds (5 min) | Checks pro sample growth; if $\geq 10\%$ → <code>run_full_cycle()</code>                 | <code>pro_count &gt;= last_count × 1.10</code>           |
| <b>Pulse</b>    | 5 seconds           | Updates <code>CoachState.last_heartbeat</code> in the database                           | Always active                                            |

### Shutdown sequence:



### Daemon implementation details:

| Daemon   | Main Method                          | Internal Loop                                                                                         | Exit Condition                   |
|----------|--------------------------------------|-------------------------------------------------------------------------------------------------------|----------------------------------|
| Scanner  | <code>_scanner_daemon_loop()</code>  | <code>while not _shutdown_event.is_set() → scan_all_paths() → sleep(10)</code>                        | <code>_shutdown_event</code> set |
| Digester | <code>_digester_daemon_loop()</code> | <code>while not _shutdown_event.is_set() → _work_available_event.wait(2) → process_next_task()</code> | <code>_shutdown_event</code> set |
| Teacher  | <code>_teacher_daemon_loop()</code>  | <code>while not _shutdown_event.is_set() → _check_retrain_needed() → sleep(300)</code>                | <code>_shutdown_event</code> set |
| Pulse    | <code>_pulse_daemon_loop()</code>    | <code>while not _shutdown_event.is_set() → state_mgr.heartbeat() → sleep(5)</code>                    | <code>_shutdown_event</code> set |

### Error handling in daemons:

Each daemon is protected by a global `try/except`. If a daemon crashes:

1. The error is logged with a full traceback
2. The `StateManager` registers the error ( `set_error(daemon, message)` )
3. A `ServiceNotification` is created for the user
4. The daemon **is not automatically restarted** (by design: a daemon crash indicates a bug, not a transient error)
5. The other daemons continue to function independently

**Zombie Task Cleanup:** On startup, the Session Engine looks for tasks with `status="processing"` left over from a previous crash and resets them to `status="queued"`, enabling automatic recovery without data loss.

**Automatic Backup:** On Session Engine startup, `BackupManager.should_run_auto_backup()` checks whether a backup is needed and, if so, creates a checkpoint labeled `"startup_auto"`. Backups follow a rotation of 7 daily + 4 weekly copies.

## 12.5 Desktop Interface

The desktop interface has two implementations: **Qt/PySide6** (primary) and **Kivy/KivyMD** (legacy). Both follow the **MVVM** pattern (Model-View-ViewModel).

### 12.5.1 Qt Interface (Primary) — `apps/qt_app/`

**Directory:** `Programma_CS2_RENAN/apps/qt_app/` **Key files:** `app.py`, `main_window.py`, `core/i18n_bridge.py`, `core/theme_engine.py`, `screens/`

The Qt interface is built with **PySide6 (Qt 6)** and uses an **MVVM pattern with Qt Signals/Slots**. The `MainWindow` (`QMainWindow`) is composed of a **navigation sidebar** and a `QStackedWidget` that hosts the 14 screens.

| Specification     | Detail                                                              |
|-------------------|---------------------------------------------------------------------|
| <b>Framework</b>  | PySide6 (Qt 6 for Python)                                           |
| <b>Pattern</b>    | MVVM with Qt Signals/Slots                                          |
| <b>Platforms</b>  | Windows, macOS, Linux                                               |
| <b>Resolution</b> | Adaptive, native High-DPI                                           |
| <b>Themes</b>     | 3: CS2 (orange), CSGO (blue-gray), CS1.6 (green) — QSS + QPalette   |
| <b>i18n</b>       | 3 languages: EN, IT, PT — JSON + <code>QtLocalizationManager</code> |
| <b>Charts</b>     | QPainter for tactical map, native Qt widgets for charts             |

**i18n system:** The `QtLocalizationManager` (`core/i18n_bridge.py`) loads per-language JSON files (`en.json`, `it.json`, `pt.json`) and handles runtime language switching via Qt signals. Every UI string is dynamically resolved via a localization key.

**Theme system:** The `ThemeEngine` (`core/theme_engine.py`) applies QSS stylesheets and configures Qt's `QPalette` for each theme:

- **CS2** — Orange palette (#FF6600) with dark background, inspired by the CS2 UI
- **CSGO** — Blue-gray palette (#4A90D9) with cool tones, inspired by CS:GO
- **CS1.6** — Green palette (#33CC33) on black background, inspired by the retro look of CS 1.6

**Web Views** (`apps/qt_app/views/`):

The Qt interface includes **2 web views** based on HTML/Jinja2 templates, served via `QWebEngineView` for content that benefits from web-native rendering:

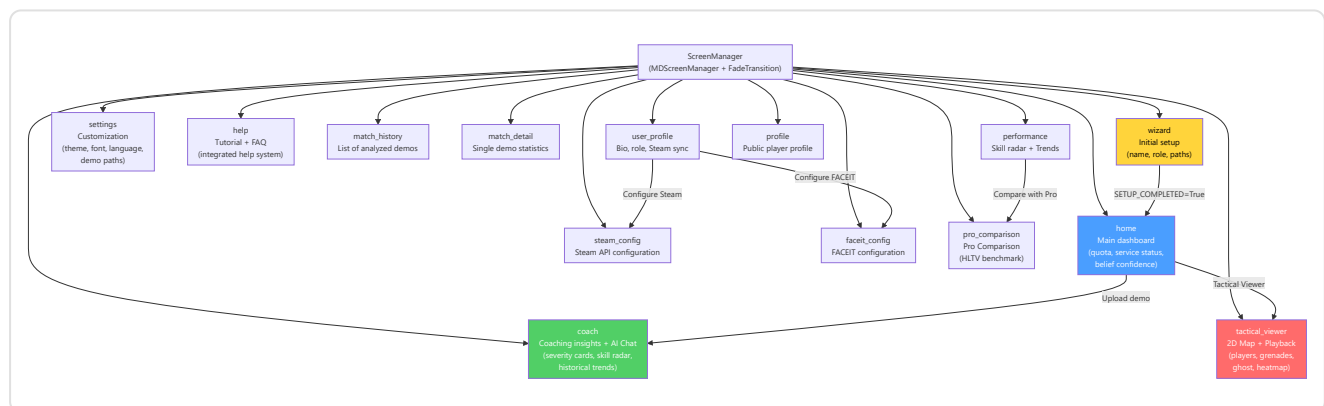
| Web View            | File                              | Description                                                                                                          |
|---------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>Match Detail</b> | <code>match_detail_view.py</code> | Detailed match visualization with timeline, per-round statistics and heatmap — HTML rendering for interactive charts |
| <b>Coach Chat</b>   | <code>coach_chat_view.py</code>   | Chat interface with the AI coach — HTML rendering for rich text formatting, markdown, code highlighting              |

The `WebBridge` (`core/web_bridge.py`) handles bidirectional communication between the Python backend and web views via `QWebChannel`, exposing Python methods as JavaScript APIs callable from the HTML template. This hybrid architecture allows using web visualization libraries (interactive charts, rich formatting) while keeping business logic in the Python backend.

### 12.5.2 Kivy Interface (Legacy) — `apps/desktop_app/`

**Directory:** `Programma_CS2_RENAN/apps/desktop_app/` **Key files:** `layout.kv`, `wizard_screen.py`, `player_sidebar.py`, `tactical_viewer_screen.py`, `tactical_viewmodels.py`, `tactical_map.py`, `timeline.py`, `widgets.py`, `help_screen.py`, `ghost_pixel.py`

The legacy interface is built with **Kivy + KivyMD** and follows the **MVVM** pattern (Model-View-ViewModel). The `ScreenManager` handles navigation between screens with `FadeTransition` transitions.



**14 interface screens:**

| Screen                 | Role                 | Key components                                                                                                         |
|------------------------|----------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Wizard</b>          | First-time setup     | Player name, role, demo folder paths                                                                                   |
| <b>Home</b>            | Dashboard            | Monthly quota (X/10), service status (green/red), belief confidence (0-1), active tasks, processed matches counter     |
| <b>Coach</b>           | Coaching insights    | Color-coded cards by severity, multi-dimensional skill radar, historical trends, AI chat (Ollama/Claude), active tasks |
| <b>Tactical Viewer</b> | Tactical playback    | 2D map with players/grenades/ghost, timeline with event markers, CT/T player sidebar, speed controls (0.25x→8x)        |
| <b>Settings</b>        | Customization        | Theme (CS2/CSGO/CS1.6), font, text size, language, demo paths, wallpaper                                               |
| <b>Help</b>            | User support         | Interactive tutorial, FAQ, troubleshooting                                                                             |
| <b>Match History</b>   | Match archive        | List of analyzed demos with filters and sorting                                                                        |
| <b>Match Detail</b>    | Match detail         | Detailed statistics for a single analyzed demo                                                                         |
| <b>Performance</b>     | Progress             | 5-axis skill radar, trend charts, temporal comparisons                                                                 |
| <b>User Profile</b>    | User profile         | Bio, preferred role, Steam/FACEIT sync                                                                                 |
| <b>Profile</b>         | Public profile       | Public player profile view                                                                                             |
| <b>Steam Config</b>    | Steam configuration  | Entering and validating Steam API key                                                                                  |
| <b>Pro Comparison</b>  | Pro comparison       | User statistics comparison with HLTV pro players, performance benchmark                                                |
| <b>FACEIT Config</b>   | FACEIT configuration | Entering and validating FACEIT API key                                                                                 |

### Custom widgets:

| Widget                     | File                           | Function                                                                                                                         |
|----------------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <code>PlayerSidebar</code> | <code>player_sidebar.py</code> | CT/T list with role icons, health/armor, current weapon, money, and alive/dead status                                            |
| <code>TacticalMap</code>   | <code>tactical_map.py</code>   | 2D canvas with multi-layer rendering: map texture → heatmap → players → grenades → ghost (QPainter in Qt, Canvas Kivy in legacy) |
| <code>Timeline</code>      | <code>timeline.py</code>       | Horizontal scrubber with tick numbers, color-coded event markers, drag-to-see, double-click jump                                 |
| <code>GhostPixel</code>    | <code>ghost_pixel.py</code>    | Rendering of the semi-transparent ghost circle (optimal position predicted by RAP)                                               |

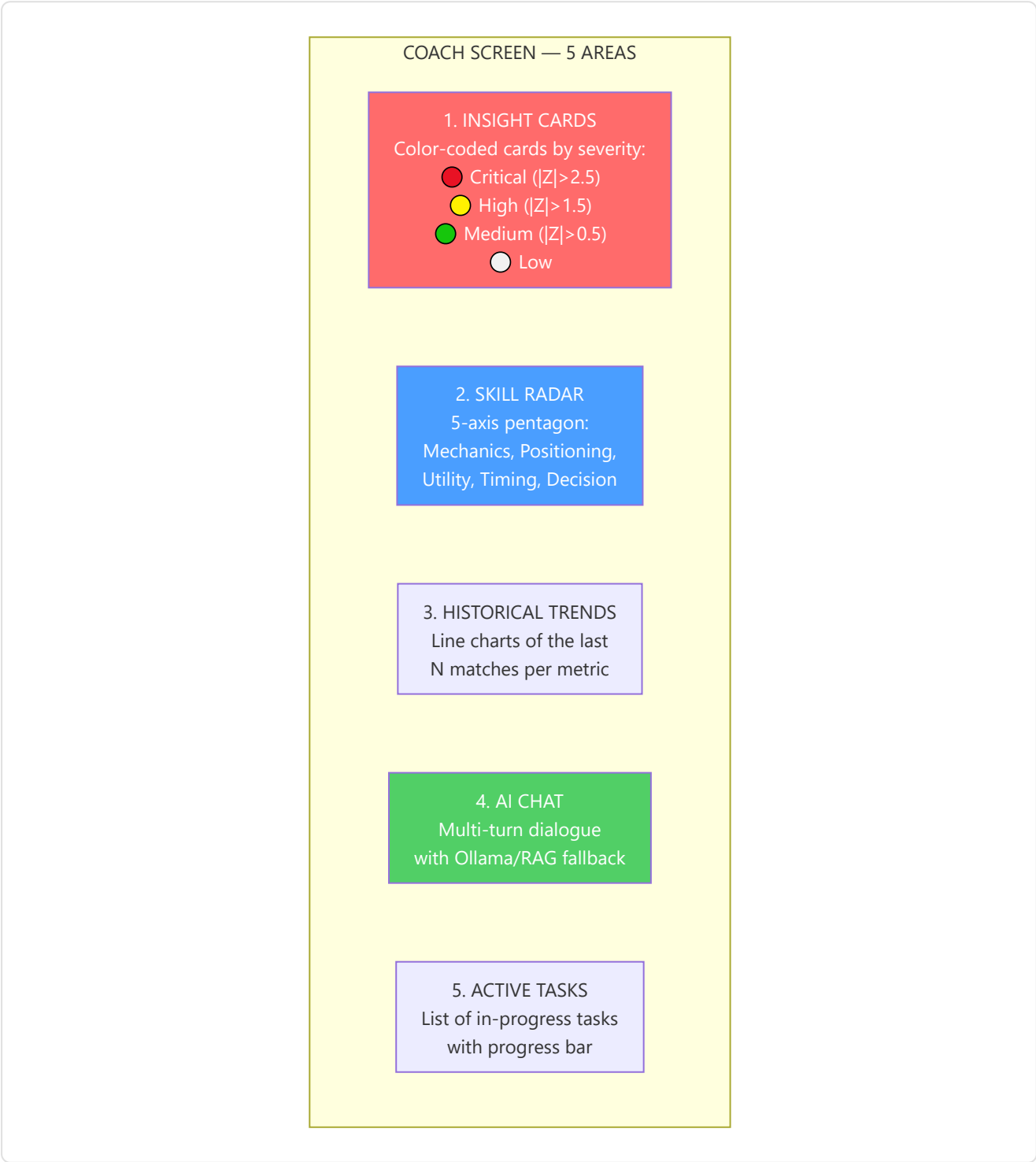
### Available themes:

The application supports **3 themes** selectable from the Settings screen, each with a custom color palette and wallpaper:

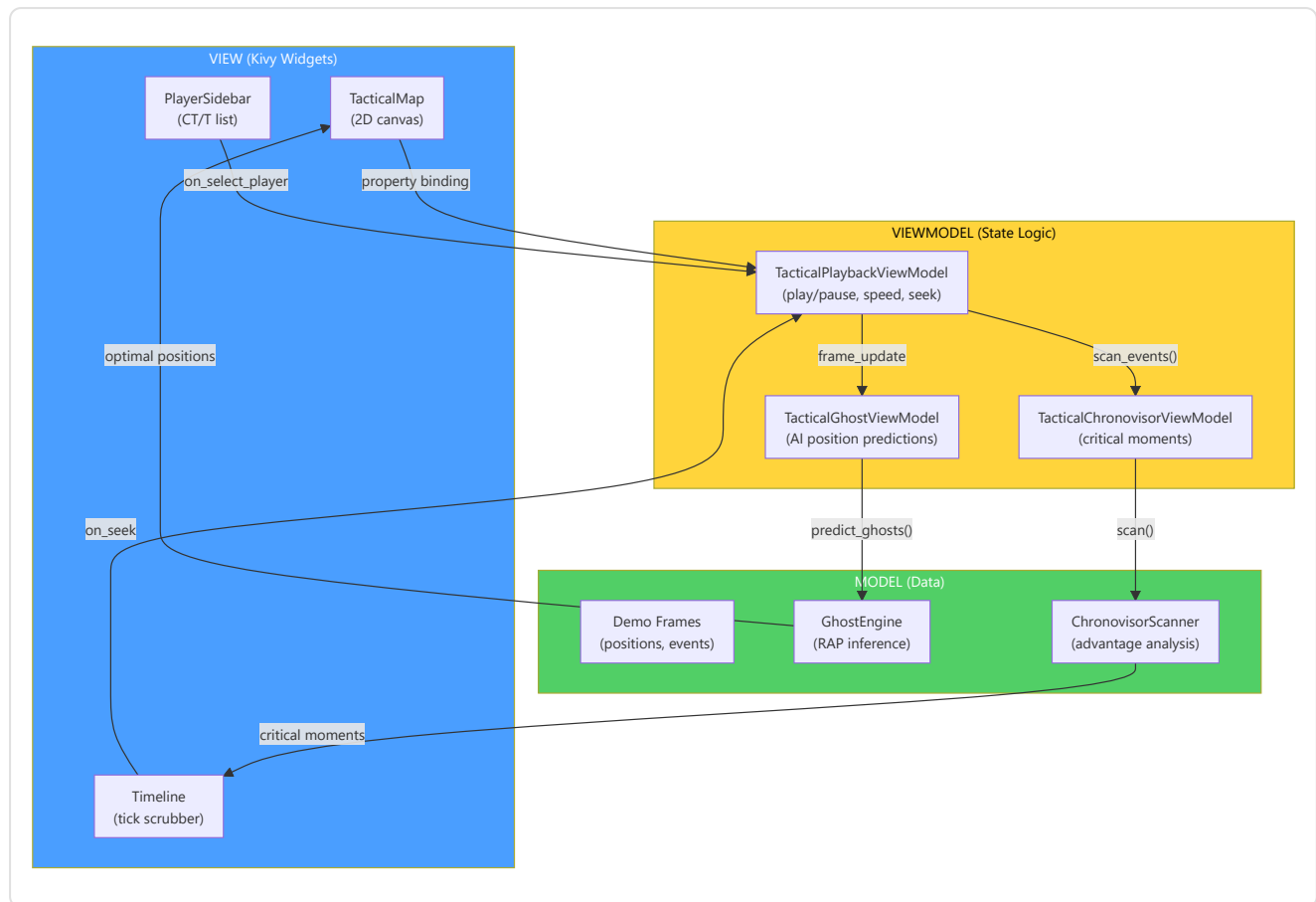
| Theme         | Primary palette         | Inspiration                            |
|---------------|-------------------------|----------------------------------------|
| CS2 (default) | Steel blue + orange     | Modern Counter-Strike 2 UI             |
| CSGO          | Military green + yellow | Counter-Strike: Global Offensive       |
| CS 1.6        | Dark brown + lime green | Classic Counter-Strike 1.6 (nostalgia) |

Coach Screen — Detailed layout:

The Coach screen is the most complex in the application, with 5 functional areas:



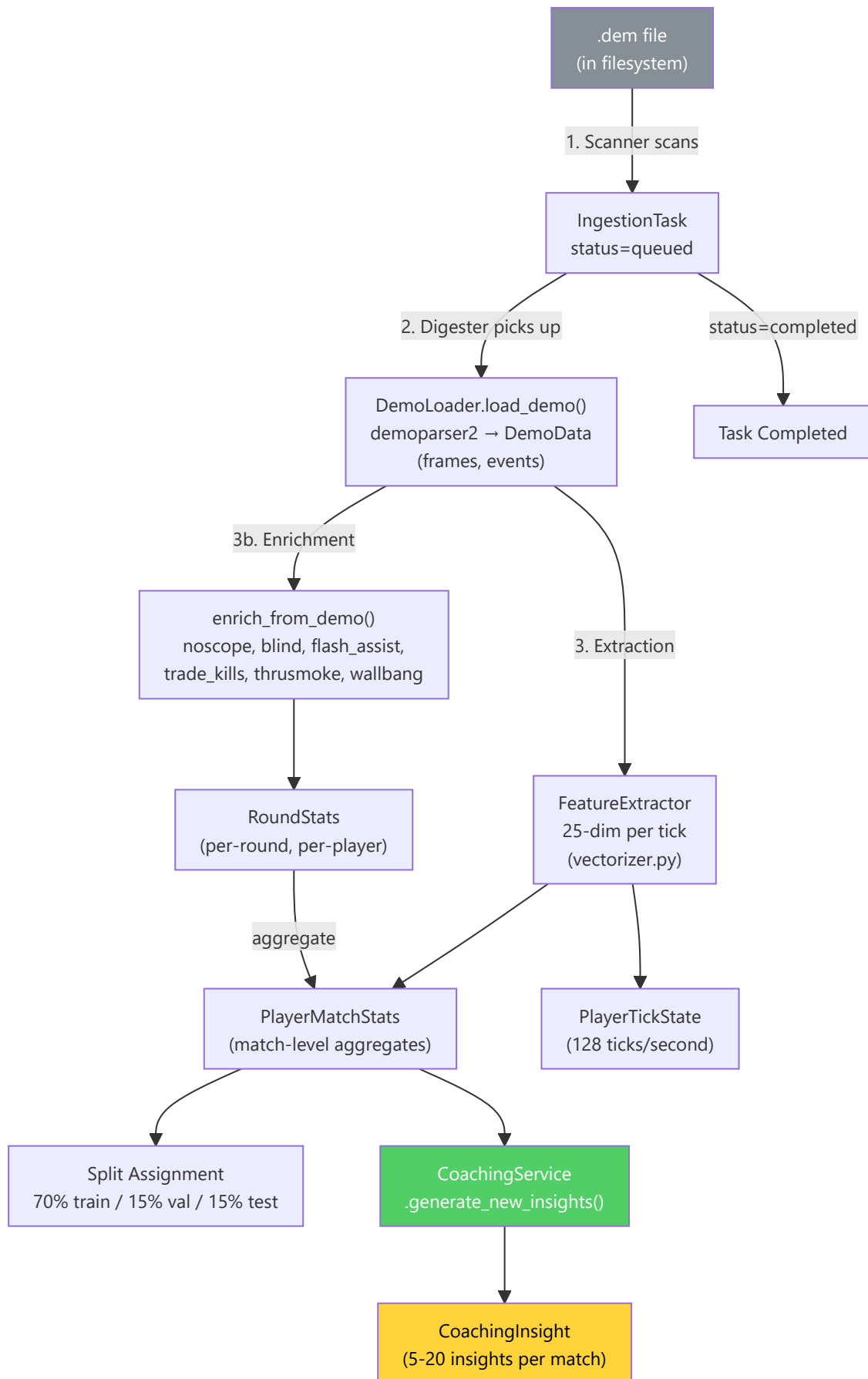
## MVVM pattern in the Tactical Viewer:



## 12.6 Ingestion Pipeline ( `ingestion/` )

**Directory:** `Programma_CS2_RENAN/ingestion/` **Key files:** `demo_loader.py` , `steam_locator.py` , `integrity.py` , `registry/` , `pipelines/user_ingest.py` , `pipelines/json_tournament_ingestor.py`

The ingestion pipeline is the **complete path** a `.dem` file takes from the filesystem to becoming coaching insights in the database. It is orchestrated by the Scanner daemon (discovery) and the Digester daemon (processing).



### DemoLoader — The parser at the heart of the pipeline:

The `DemoLoader` is the wrapper around **demoparser2** (high-performance Rust library) that transforms a binary `.dem` file into Python data structures:

| Parsing Phase       | Output                                                      | Typical size            |
|---------------------|-------------------------------------------------------------|-------------------------|
| 1. Header parsing   | Metadata (map, server, duration)                            | ~100 bytes              |
| 2. Frame extraction | List of frames (position, health, weapon per tick)          | ~100,000 frames/match   |
| 3. Event extraction | List of events (kill, death, bomb_plant, round_start, etc.) | ~500-2,000 events/match |
| 4. Player summary   | Aggregated per-player statistics                            | ~10 records             |

**FeatureExtractor** ( `backend/processing/feature_engineering/vectorizer.py` ):

The FeatureExtractor is the component that transforms raw demo data into **25-dimensional numeric vectors** ( `METADATA_DIM=25` ) usable by the neural networks. **Important:** these are **tick-level** features (128 Hz), not match-level aggregated statistics. Each individual game frame produces a 25-dim vector that captures the instantaneous state of the player:

| Dim | Feature         | Type   | Range              | Description                                                                  |
|-----|-----------------|--------|--------------------|------------------------------------------------------------------------------|
| 0   | health          | Float  | [0, 1]             | Normalized health                                                            |
| 1   | armor           | Float  | [0, 1]             | Normalized armor                                                             |
| 2   | has_helmet      | Binary | 0/1                | Helmet equipped                                                              |
| 3   | has_defuser     | Binary | 0/1                | Defuse kit equipped                                                          |
| 4   | equipment_value | Float  | [0, 1]             | Normalized equipment value                                                   |
| 5   | is_crouching    | Binary | 0/1                | Crouched                                                                     |
| 6   | is_scoped       | Binary | 0/1                | Scope active                                                                 |
| 7   | is_blinded      | Binary | 0/1                | Flashed                                                                      |
| 8   | enemies_visible | Float  | [0, 1]             | Visible enemies (normalized, clamped)                                        |
| 9   | pos_x           | Float  | [-1, 1]            | X position (normalized $\pm$ pos_xy_extent)                                  |
| 10  | pos_y           | Float  | [-1, 1]            | Y position (normalized $\pm$ pos_xy_extent)                                  |
| 11  | pos_z           | Float  | [0, 1]             | Z position (normalized, handles Nuke/Vertigo)                                |
| 12  | view_x_sin      | Float  | [-1, 1]            | sin(yaw) — cyclic continuity of horizontal angle                             |
| 13  | view_x_cos      | Float  | [-1, 1]            | cos(yaw) — cyclic continuity of horizontal angle                             |
| 14  | view_y          | Float  | [-1, 1]            | Normalized pitch (vertical angle)                                            |
| 15  | z_penalty       | Float  | [0, 1]             | Vertical level distinction (floor penalty)                                   |
| 16  | kast_estimate   | Float  | [0, 1]             | KAST estimate (participation ratio)                                          |
| 17  | map_id          | Float  | [0, 1]             | Deterministic map hash                                                       |
| 18  | round_phase     | Float  | {0, 0.33, 0.66, 1} | Economic phase: pistol/eco/force/full_buy                                    |
| 19  | weapon_class    | Float  | {0–1.0}            | Weapon class: 0=knife, 0.2=pistol, 0.4=SMG, 0.6=rifle, 0.8=sniper, 1.0=heavy |
| 20  | time_in_round   | Float  | [0, 1]             | Seconds in round / 115 (clamped)                                             |
| 21  | bomb_planted    | Binary | 0/1                | Bomb planted                                                                 |
| 22  | teammates_alive | Float  | [0, 1]             | Teammates alive (count / 4)                                                  |
| 23  | enemies_alive   | Float  | [0, 1]             | Enemies alive (count / 5)                                                    |
| 24  | team_economy    | Float  | [0, 1]             | Team money average / 16000 (clamped)                                         |

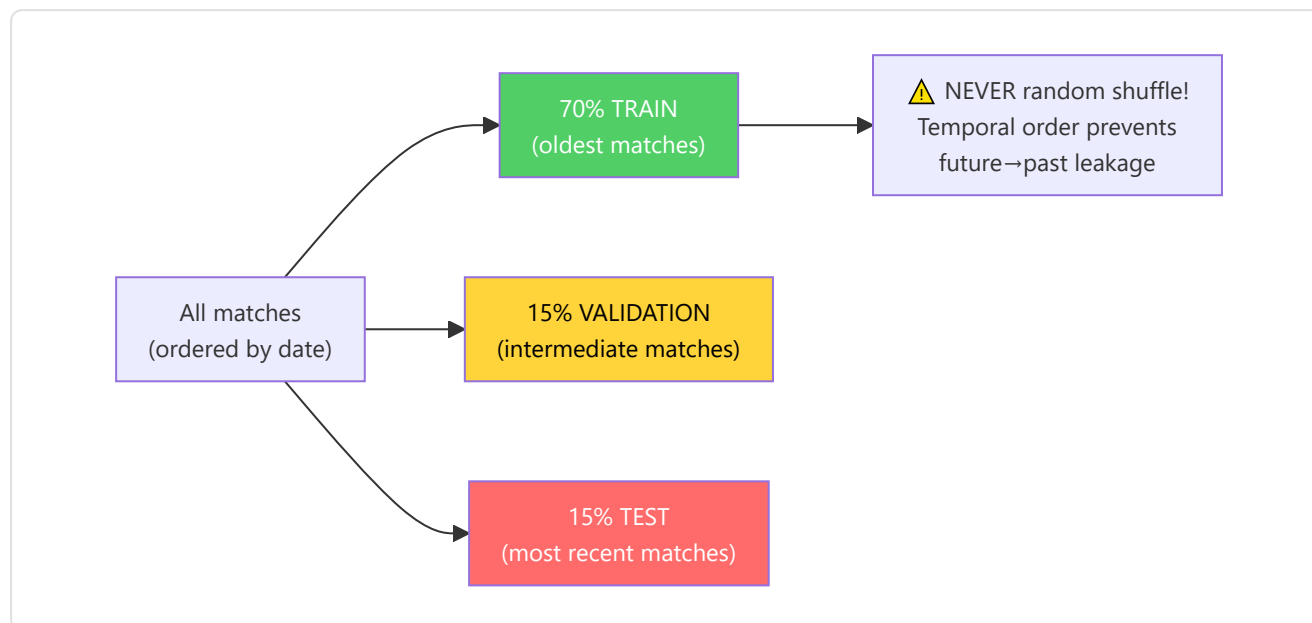
### Normalization and bounds:

Normalization is integrated directly into the FeatureExtractor, with bounds configurable via `HeuristicConfig` (externalized to JSON). The cyclic encoding of view angles (sin/cos for yaw) prevents discontinuities at the 0°/360° boundary. The `z_penalty` automatically distinguishes floors on multi-level

maps (Nuke, Vertigo). Weapon class uses an ordinal categorical mapping (6 classes) defined in the `WEAPON_CLASS_MAP` constant (also includes grenades=0.1 and special equipment=0.05).

### Temporal Dataset Split:

The dataset split follows a strict **chronological ordering** to prevent temporal data leakage:



### Enrich From Demo — Post-parsing enrichment:

After base parsing, `enrich_from_demo()` adds advanced metrics computed from events:

| Enriched Metric     | Computation                                                                      | Source                           |
|---------------------|----------------------------------------------------------------------------------|----------------------------------|
| Trade kills         | <code>TradeKillDetector.detect()</code> with <code>TRADE_WINDOW_TICKS=192</code> | Kill/death events                |
| Flash assists       | Count of blinds within time window before a kill                                 | Blind + kill events              |
| Noscope kills       | Kills with sniper weapon without scope active                                    | Kill event + weapon state        |
| Wallbang kills      | Kills through penetrable surfaces                                                | Kill event with penetration flag |
| Through-smoke kills | Kills with smoke active in line of fire                                          | Kill event + smoke position      |
| Blind kills         | Kills while the player is flashed                                                | Kill event + flash state         |

### Specific components:

| Component                     | File                                               | Role                                                                                           |
|-------------------------------|----------------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>DemoLoader</b>             | <code>demo_loader.py</code>                        | Wraps <code>demoparser2</code> , extracts frames and events from the <code>.dem</code> file    |
| <b>SteamLocator</b>           | <code>steam_locator.py</code>                      | Automatically locates the CS2 demo folder via Steam registry / <code>libraryfolders.vdf</code> |
| <b>IntegrityChecker</b>       | <code>integrity.py</code>                          | Verifies that demo files are valid, complete, and not corrupt before parsing                   |
| <b>UserIngestPipeline</b>     | <code>pipelines/user_ingest.py</code>              | Complete pipeline for user demos: parse → enrich → stats → coaching                            |
| <b>JsonTournamentIngestor</b> | <code>pipelines/json_tournament_ingestor.py</code> | Imports tournament data from structured JSON files                                             |
| <b>Registry</b>               | <code>registry/registry.py</code>                  | Tracks all processed demos, prevents duplicates                                                |
| <b>ResourceManager</b>        | <code>ingestion/resource_manager.py</code>         | Hardware resource management: CPU/RAM throttling, disk space                                   |
| <b>JsonTournamentIngestor</b> | <code>pipelines/json_tournament_ingestor.py</code> | Imports tournament data from structured JSON files                                             |
| <b>RegistryLifecycle</b>      | <code>registry/lifecycle.py</code>                 | Lifecycle management of ingestion records                                                      |

**SteamLocator** ( `ingestion/steam_locator.py` ) — automatic CS2 demo location:

The SteamLocator implements a **cross-platform discovery** algorithm to automatically find the CS2 demo folder:

| Platform        | Strategy                                                       | Typical path                                                                                                 |
|-----------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>Windows</b>  | System registry → <code>libraryfolders.vdf</code>              | <code>C:\Program Files (x86)\Steam\steamapps\common\Counter-Strike Global Offensive\game\csgo\replays</code> |
| <b>Linux</b>    | <code>~/.steam/steam/</code> → <code>libraryfolders.vdf</code> | <code>~/.steam/steam/steamapps/common/...</code>                                                             |
| <b>Fallback</b> | Prompts user via UI Settings                                   | Custom path                                                                                                  |

**IntegrityChecker** ( `ingestion/integrity.py` ):

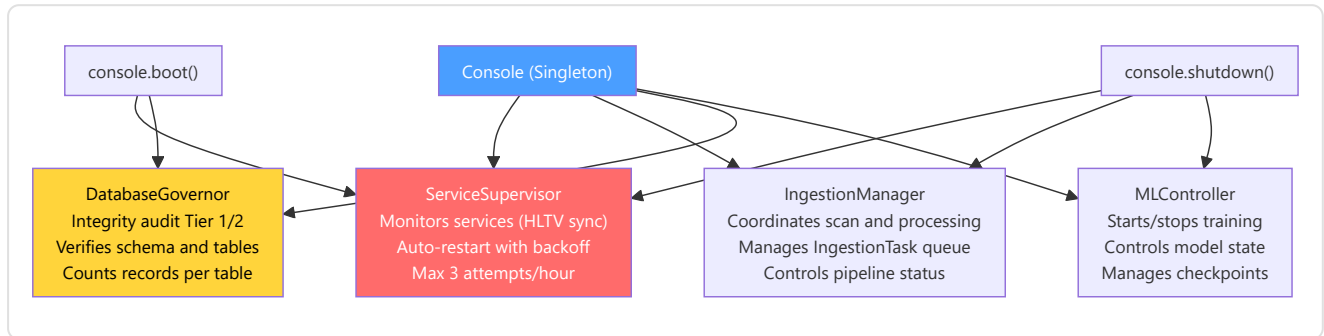
Preliminary check of each demo file before expensive parsing:

- **Magic bytes:** `PBDEMS2\0` (CS2 Source 2) or `HL2DEMO\0` (legacy Source 1)
- **Size bounds:** minimum 1KB (not empty), maximum 5GB (not corrupt/excessive)
- **Read test:** attempts to read the first N bytes to verify the file is accessible

## 12.7 Unified Control Console ( `backend/control/` )

**File:** `Programma_CS2_RENAN/backend/control/console.py` , `ingest_manager.py` , `db_governor.py` , `ml_controller.py`

The Console is a **Singleton** that serves as a central coordination point for all backend subsystems. It is the "control panel" through which every part of the system can be controlled.

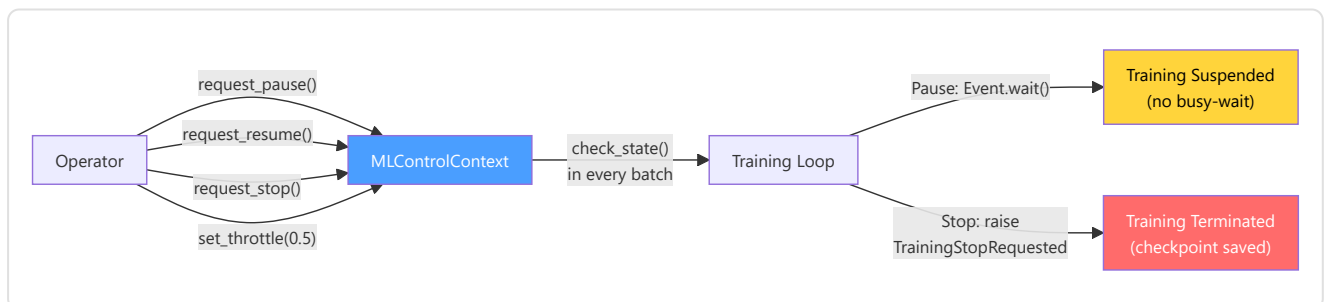


### Console boot sequence:

1. `ServiceSupervisor` starts the "hunter" service (HLTV sync) as a monitored process
2. `DatabaseGovernor` performs an integrity audit: verifies all tables, counts records, checks schema
3. `MLController` remains on standby — training is handled by the Teacher daemon
4. `IngestionManager` stays idle — active work is handled by the Scanner/Digester daemons

### MLControlContext — Live Training Control:

The `MLController` uses a control token called `MLControlContext` ( `backend/control/ml_controller.py` ) that is passed to training cycles to enable **real-time intervention** by the operator. This replaces the previous approach based on `StopIteration` with a more robust thread-safe system.

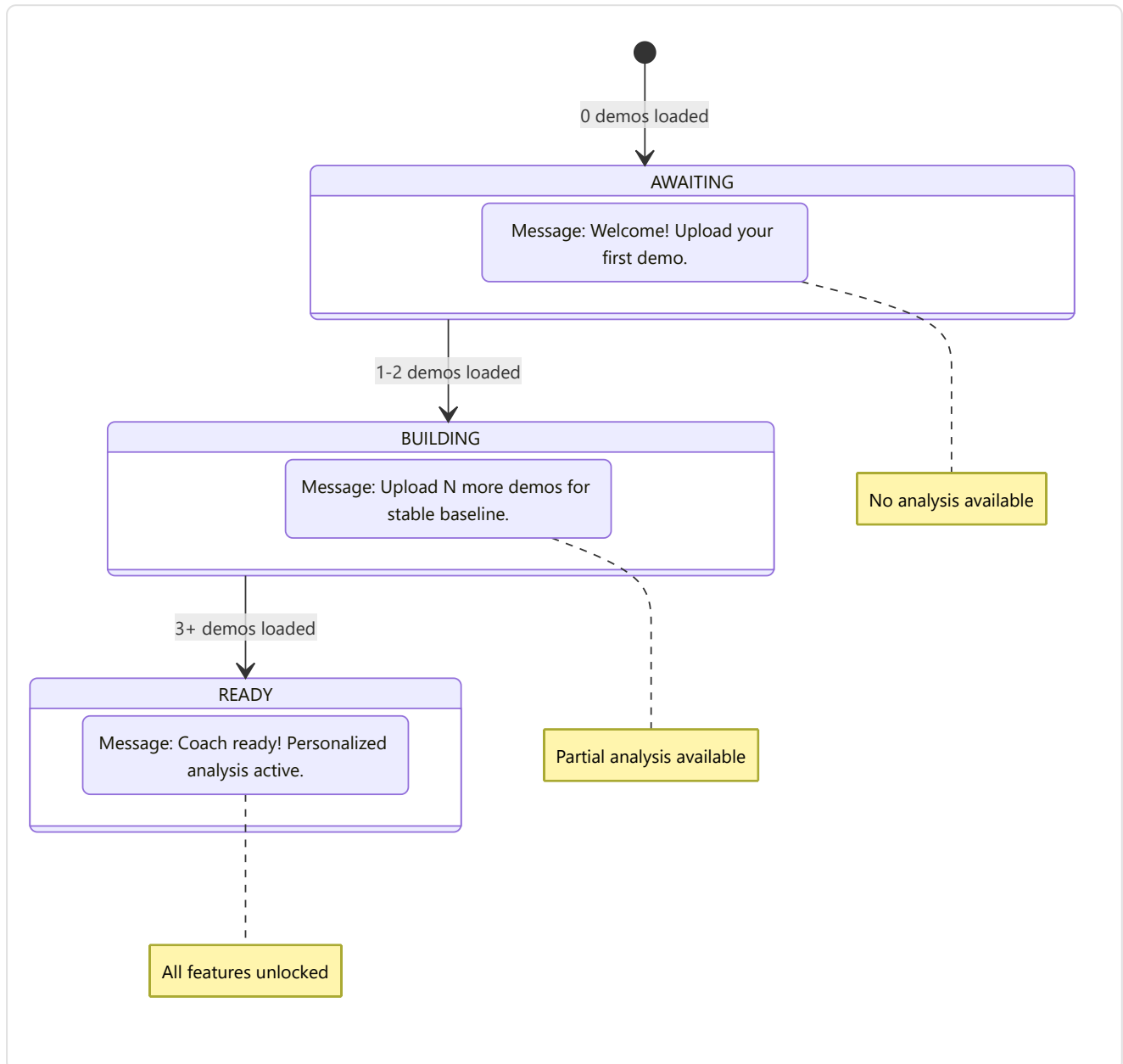


| Command         | Method                            | Effect                                                                 |
|-----------------|-----------------------------------|------------------------------------------------------------------------|
| <b>Pause</b>    | <code>request_pause()</code>      | <code>_resume_event.clear()</code> → blocks <code>check_state()</code> |
| <b>Resume</b>   | <code>request_resume()</code>     | <code>_resume_event.set()</code> → unblocks training                   |
| <b>Stop</b>     | <code>request_stop()</code>       | Raises <code>TrainingStopRequested</code> (custom exception)           |
| <b>Throttle</b> | <code>set_throttle(factor)</code> | Adds <code>time.sleep(factor)</code> after each batch                  |

## 12.8 Onboarding and New User Flow

**File:** `Programma_CS2_RENAN/backend/onboarding/new_user_flow.py`

The `OnboardingManager` guides new users through a **3-phase progression** that automatically adapts to the amount of available data.



### Wizard Screen (First-time Setup):

On first run ( `SETUP_COMPLETED = False` ), the user is guided through the Wizard:

1. **Player name** — The name that will appear in analyses
2. **Preferred role** — Entry Fragger, AWPer, Lurker, Support, or IGL
3. **Demo folder paths** — Where the system automatically looks for demos
4. On completion: `SETUP_COMPLETED = True` , redirect to Home

**Quota cache (Task 2.16.1):** The demo count is cached for 60 seconds to avoid repeated DB queries. `invalidate_cache()` is called after every new upload, ensuring the UI always shows the correct count without overloading the database.

**Help System** ( `backend/knowledge/help_system.py` ):

The integrated help system provides contextual support to the user:

| Feature                     | Implementation                                                                            |
|-----------------------------|-------------------------------------------------------------------------------------------|
| <b>Interactive tutorial</b> | Step-by-step guides for the main features                                                 |
| <b>Contextual FAQ</b>       | Frequently asked questions filtered by current screen                                     |
| <b>Troubleshooting</b>      | Decision tree for common problems (Steam path not found, demo not parsed, empty coaching) |
| <b>Tooltips</b>             | Inline explanations for complex metrics (KAST, HLTV 2.0 Rating, ADR)                      |

## 12.9 Storage Architecture ( `backend/storage/` )

**Directory:** `Programma_CS2_RENAN/backend/storage/` **Key files:** `database.py` , `db_models.py` , `match_data_manager.py` , `storage_manager.py` , `maintenance.py` , `state_manager.py` , `stat_aggregator.py` , `backup.py`

The storage system uses a **three-tier storage** architecture based on SQLite in WAL mode (Write-Ahead Logging), which enables concurrent reads and writes without blocking.







**The 21 SQLModel tables:**

| #  | Table               | Database         | Category   | Description                                                                                               |
|----|---------------------|------------------|------------|-----------------------------------------------------------------------------------------------------------|
| 1  | PlayerMatchStats    | database.db      | Core       | Aggregated per-player/match statistics (32 fields)                                                        |
| 2  | PlayerTickState     | database.db      | Core       | Per-tick state (128 Hz), also archived in separate per-match DBs                                          |
| 3  | PlayerProfile       | database.db      | User       | User profile (name, role, Steam ID, monthly quota)                                                        |
| 4  | RoundStats          | database.db      | Core       | Per-round isolated statistics (kills, rating, enrichment)                                                 |
| 5  | CoachingInsight     | database.db      | Coaching   | Advice generated by the coaching service                                                                  |
| 6  | CoachingExperience  | database.db      | Coaching   | COPER experience bank (context, outcome, effectiveness, TrueSkill $\mu/\sigma$ , replay priority — KT-01) |
| 7  | IngestionTask       | database.db      | System     | Work queue for the Digester daemon                                                                        |
| 8  | CoachState          | database.db      | System     | Global state (training metrics, heartbeat, status)                                                        |
| 9  | ServiceNotification | database.db      | System     | Error/event messages from daemons → UI                                                                    |
| 10 | TacticalKnowledge   | database.db      | Knowledge  | RAG base (384-dim embedding in JSON)                                                                      |
| 11 | ProPlayer           | hltv_metadata.db | Pro        | Pro player profiles                                                                                       |
| 12 | ProTeam             | hltv_metadata.db | Pro        | Pro team metadata                                                                                         |
| 13 | ProPlayerStatCard   | hltv_metadata.db | Pro        | Seasonal statistics per pro player                                                                        |
| 14 | Ext_PlayerPlaystyle | database.db      | External   | Playstyle data from CSV (for NeuralRoleHead)                                                              |
| 15 | Ext_TeamRoundStats  | database.db      | External   | External tournament statistics                                                                            |
| 16 | MatchResult         | database.db      | Matches    | Match outcomes                                                                                            |
| 17 | MapVeto             | database.db      | Matches    | Map selection history                                                                                     |
| 18 | CalibrationSnapshot | database.db      | System     | Belief model calibration log (timestamp, samples, outcome)                                                |
| 19 | RoleThresholdRecord | database.db      | System     | Learned thresholds for role classification (persistent across restarts)                                   |
| 20 | DataLineage         | database.db      | Provenance | Append-only data provenance log: entity_type, entity_id, source_demo, pipeline_version, processing_step   |
| 21 | DataQualityMetric   | database.db      | Provenance | Append-only quality metrics per run: run_id, run_type, metric_name, metric_value, sample_count            |

### Support enums (not tables):

| Enum                      | Type                       | Description                                                                                                      |
|---------------------------|----------------------------|------------------------------------------------------------------------------------------------------------------|
| <code>DatasetSplit</code> | <code>str</code> ,<br>Enum | Split categories (train/val/test/unassigned) — used as constraint on <code>PlayerMatchStats.dataset_split</code> |
| <code>CoachStatus</code>  | <code>str</code> ,<br>Enum | Coach states (Paused/Training/Idle/Error) — used as constraint on <code>CoachState.status</code>                 |

### Detailed Storage Components:

**MatchDataManager** ( `backend/storage/match_data_manager.py` ) — the largest component of the storage layer:

The MatchDataManager is responsible for managing high-density per-match data (PlayerTickState with ~100,000 rows per match). To prevent the main database from growing uncontrollably, each match has its own separate SQLite database ( `match_XXXX.db` ).

| Method                                         | Description                                                               |
|------------------------------------------------|---------------------------------------------------------------------------|
| <code>create_match_db(demo_name)</code>        | Creates a new per-match database with <code>PlayerTickState</code> schema |
| <code>store_tick_data(demo_name, ticks)</code> | Bulk insert of tick data into the dedicated DB                            |
| <code>load_match_frames(demo_name)</code>      | Loads all frames for the Tactical Viewer                                  |
| <code>get_match_db_path(demo_name)</code>      | Resolves the path of the per-match DB                                     |
| <code>list_available_matches()</code>          | Lists all matches with DB available                                       |
| <code>delete_match_data(demo_name)</code>      | Removes the per-match DB and updates the registry                         |
| <code>get_match_statistics(demo_name)</code>   | Computes aggregate statistics from tick data                              |

**StorageManager** ( `backend/storage/storage_manager.py` ):

The StorageManager is the **high-level storage coordinator** that manages quotas, uploads, and data lifecycle:

| Responsibility       | Implementation                                                                                                   |
|----------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Monthly quota</b> | <code>can_user_upload()</code> → checks <code>MAX_DEMOS_PER_MONTH=10</code> and <code>MAX_TOTAL_DEMOS=100</code> |
| <b>Upload flow</b>   | <code>handle_demo_upload(path)</code> → validation → copy to working dir → create IngestionTask                  |
| <b>Cleanup</b>       | <code>cleanup_old_data(days)</code> → removes old match DBs and tasks                                            |
| <b>Disk space</b>    | <code>get_storage_usage()</code> → reports size for each database and directory                                  |

**StateManager** ( `backend/storage/state_manager.py` ):

The StateManager is a **Singleton** that maintains the system's runtime state and persists it to the database via the `CoachState` table:

| Method                                                           | Purpose                                                       |
|------------------------------------------------------------------|---------------------------------------------------------------|
| <code>update_status(daemon, text)</code>                         | Updates the state of a specific daemon                        |
| <code>heartbeat()</code>                                         | Updates the <code>last_heartbeat</code> timestamp             |
| <code>get_state()</code>                                         | Returns the current state as a <code>CoachState</code> object |
| <code>set_error(daemon, message)</code>                          | Registers an error for a daemon with timestamp                |
| <code>update_training_metrics(epoch, loss, val_loss, eta)</code> | Updates training metrics in real time                         |
| <code>get_belief_confidence()</code>                             | Returns the belief model's confidence level (0.0-1.0)         |

**StatAggregator** ( `backend/storage/stat_aggregator.py` ):

Computes aggregate statistics from raw per-round data:

| Aggregation                  | Formula                                             | Use                    |
|------------------------------|-----------------------------------------------------|------------------------|
| <code>avg_kills</code>       | <code>mean(RoundStats.kills)</code>                 | Dashboard, radar chart |
| <code>avg_adr</code>         | <code>mean(RoundStats.damage_dealt / rounds)</code> | Pro comparison         |
| <code>avg_kast</code>        | <code>mean(rounds_with_kast / total_rounds)</code>  | HLTV metric            |
| <code>accuracy</code>        | <code>sum(hits) / sum(shots_fired)</code>           | Mechanical performance |
| <code>trade_kill_rate</code> | <code>trade_kills / team_deaths</code>              | Team play              |

**BackupManager** ( `backend/storage/backup.py` ):

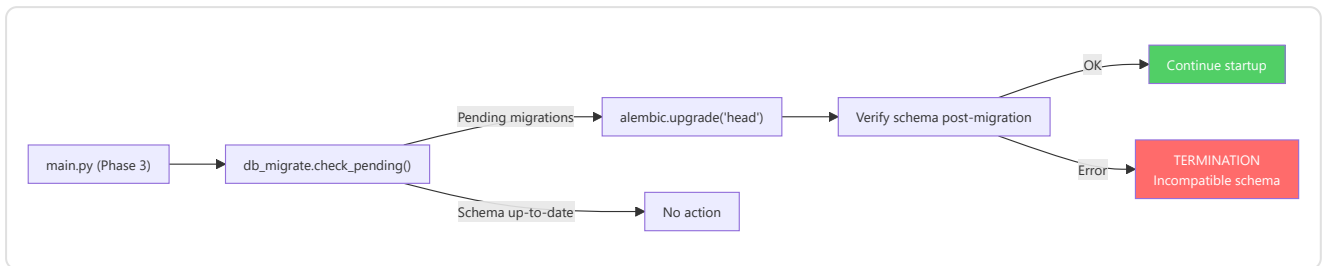
| Feature                  | Detail                                                                       |
|--------------------------|------------------------------------------------------------------------------|
| <b>Daily rotation</b>    | 7 copies — the oldest is overwritten                                         |
| <b>Weekly rotation</b>   | 4 copies — additional weekly backup                                          |
| <b>Automatic trigger</b> | On Session Engine startup via <code>should_run_auto_backup()</code>          |
| <b>Manual trigger</b>    | Via Console or UI Settings                                                   |
| <b>Format</b>            | Full copy of the <code>.db</code> file (not SQL dump)                        |
| <b>Labeling</b>          | <code>startup_auto</code> , <code>manual</code> , <code>pre_migration</code> |

**Maintenance** ( `backend/storage/maintenance.py` ):

| Operation                      | Frequency               | Purpose                                                               |
|--------------------------------|-------------------------|-----------------------------------------------------------------------|
| <code>vacuum()</code>          | Monthly                 | Compacts the database, reclaims space from deleted records            |
| <code>analyze()</code>         | After every bulk insert | Updates SQLite query optimizer statistics                             |
| <code>wal_checkpoint()</code>  | On startup              | Forces merge of WAL into the main database                            |
| <code>integrity_check()</code> | On startup              | <code>PRAGMA integrity_check</code> — verifies structural consistency |

**DbMigrate** ( `backend/storage/db_migrate.py` ):

Wrapper around Alembic that automates the execution of migrations:



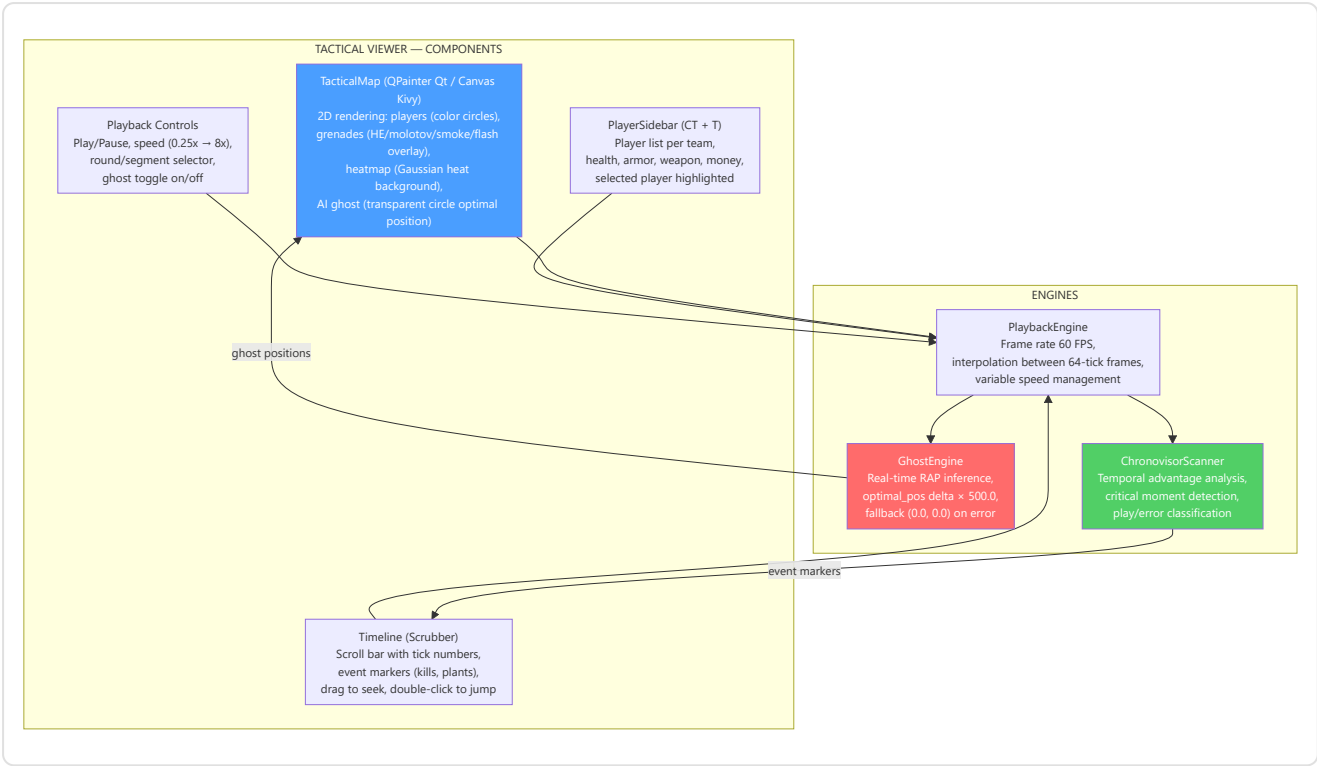
### Connection Pooling and Concurrency:

| Parameter                      | Value              | Purpose                                     |
|--------------------------------|--------------------|---------------------------------------------|
| <code>check_same_thread</code> | <code>False</code> | Enables multi-thread access                 |
| <code>timeout</code>           | 30 seconds         | Busy timeout for WAL contention             |
| <code>pool_size</code>         | 1                  | Single SQLite writer (single-writer safety) |
| <code>max_overflow</code>      | 4                  | Overflow connections for load spikes        |
| WAL mode                       | Enabled            | Unlimited concurrent reads                  |

## 12.10 Playback Engine and Tactical Viewer

**File:** `Programma_CS2_RENAN/core/playback.py` , `playback_engine.py` ,  
`apps/desktop_app/tactical_viewer_screen.py` , `tactical_map.py` , `timeline.py` , `player_sidebar.py`

The tactical playback system allows the user to **relive their matches** on an interactive 2D map, with AI overlay (optimal ghost position), event markers (kills, bomb plants), and full playback controls.



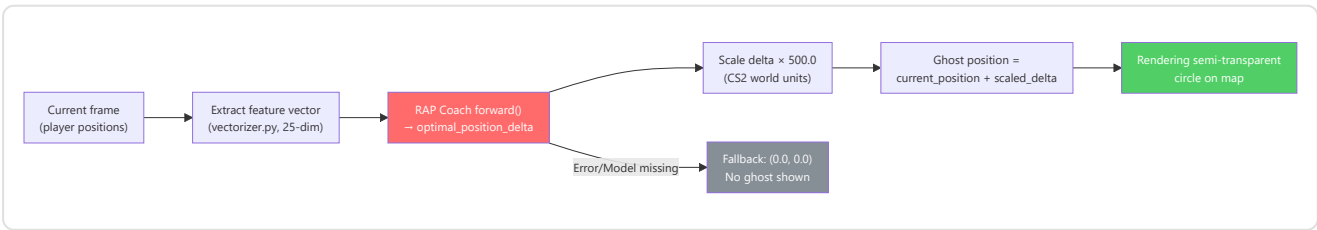
PlaybackEngine — Internal architecture:

The PlaybackEngine handles frame-by-frame playback with temporal interpolation. In the Qt implementation, the update timer uses `QTimer` instead of `Kivy Clock.schedule_interval`, keeping the same interpolation and buffering logic:

| Feature         | Detail                                                        |
|-----------------|---------------------------------------------------------------|
| Frame rate      | 60 FPS (interpolation from demo's native 64-tick)             |
| Variable speed  | 0.25x (slow-mo), 0.5x, 1x (normal), 2x, 4x, 8x (fast-forward) |
| Interpolation   | Linear between adjacent frames for smooth movement            |
| Buffering       | Pre-loads 120 frames ahead to avoid lag                       |
| Seek            | Direct access to any tick via index                           |
| Round selection | Jumps directly to the start of a specific round               |

GhostEngine — Real-time inference:

The GhostEngine is the component that transforms the RAP Coach's predictions into **ghost positions visible on the map**:



### ChronovisorScanner — Critical moment detection:

The ChronovisorScanner analyzes the entire match and identifies the **decisive moments** based on advantage change:

| Moment type           | Condition                                              | Marker color |
|-----------------------|--------------------------------------------------------|--------------|
| <b>Critical error</b> | Death while in numerical advantage (e.g., 4v3 → 3v3)   | Red          |
| <b>Excellent play</b> | Kill while in numerical disadvantage (e.g., 2v3 → 2v2) | Green        |
| <b>Bomb plant</b>     | bomb_planted event with timer                          | Yellow       |
| <b>Clutch</b>         | 1vN victory ( $N \geq 2$ )                             | Gold         |
| <b>Eco round win</b>  | Victory with equipment < \$2,000                       | Blue         |

Each moment is placed on the Timeline as a clickable marker. Clicking jumps playback directly to that tick.

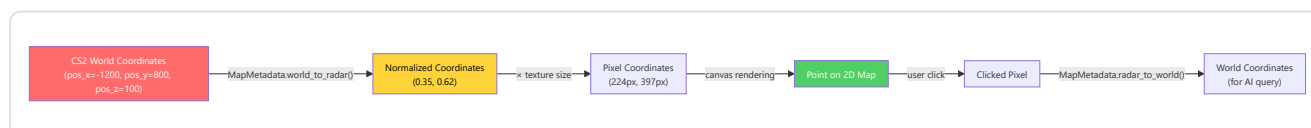
### Viewer loading flow (One-Click):

1. The user clicks "Tactical Viewer" from Home
2. If no demo is loaded → `trigger_viewer_picker()` automatically opens the file picker
3. The user selects a `.dem` file
4. A "Dynamic 2D Reconstruction in Progress..." dialog appears
5. A background thread runs `_execute_viewer_parse(path)` → `DemoLoader.load_demo()`
6. On completion: dialog dismissal, loading frames into PlaybackEngine
7. The map is rendered with players at frame 0

## 12.11 Spatial Data and Map Management

**File:** `Programma_CS2_RENAN/core/spatial_data.py`, `spatial_engine.py`, `data/map_config.json`

The map management system translates **CS2 world coordinates** (typical values: -2000 to +2000 on X/Y) into **pixel coordinates** on the map texture (0.0 to 1.0 normalized), and vice versa.



**MapMetadata** (immutable dataclass for each map):

| Field                       | Example (Dust2) | Purpose                                 |
|-----------------------------|-----------------|-----------------------------------------|
| <code>pos_x</code>          | -2476           | X of top-left corner in world units     |
| <code>pos_y</code>          | 3239            | Y of top-left corner in world units     |
| <code>scale</code>          | 4.4             | Game units per texture pixel            |
| <code>z_cutoff</code>       | None            | Level separator (multi-level maps only) |
| <code>level</code>          | "single"        | Type: "single", "upper", "lower"        |
| <code>texture_width</code>  | 1024            | Radar texture width in pixels           |
| <code>texture_height</code> | 1024            | Radar texture height in pixels          |

### MapManager ( `core/map_manager.py` ):

The MapManager is the high-level component that coordinates map loading for the interface:

| Method                                     | Purpose                                               |
|--------------------------------------------|-------------------------------------------------------|
| <code>get_map_metadata(map_name)</code>    | Returns MapMetadata for the requested map             |
| <code>load_radar_texture(map_name)</code>  | Loads PNG texture of the radar map from cache or disk |
| <code>get_available_maps()</code>          | List of supported maps with metadata                  |
| <code>world_to_pixel(pos, map_name)</code> | Shortcut for world → pixel coordinate conversion      |

### Supported multi-level maps:

| Map            | z_cutoff | Levels        | Notes                               |
|----------------|----------|---------------|-------------------------------------|
| <b>Nuke</b>    | -495     | upper / lower | Two plant sites on different floors |
| <b>Vertigo</b> | 11700    | upper / lower | Skyscraper with two playable areas  |
| All others     | None     | single        | Single-level maps                   |

### SpatialEngine ( `core/spatial_engine.py` ):

The SpatialEngine adds **spatial reasoning** capabilities to the base coordinate system:

| Method                                           | Input                 | Output                  | Use                             |
|--------------------------------------------------|-----------------------|-------------------------|---------------------------------|
| <code>distance_2d(pos_a, pos_b)</code>           | Two world coordinates | Float (world units)     | Engagement distance computation |
| <code>is_visible(pos_a, pos_b, obstacles)</code> | Two pos + obstacles   | Bool                    | Line of sight (simplified)      |
| <code>get_zone(pos, map_name)</code>             | Coordinate + map      | String (e.g., "A_site") | Tactical zone classification    |
| <code>nearest_cover(pos, map_name)</code>        | Coordinate + map      | Cover coordinate        | Positional suggestion           |

**AssetManager** ( `core/asset_manager.py` ):

Handles loading of map textures and UI assets:

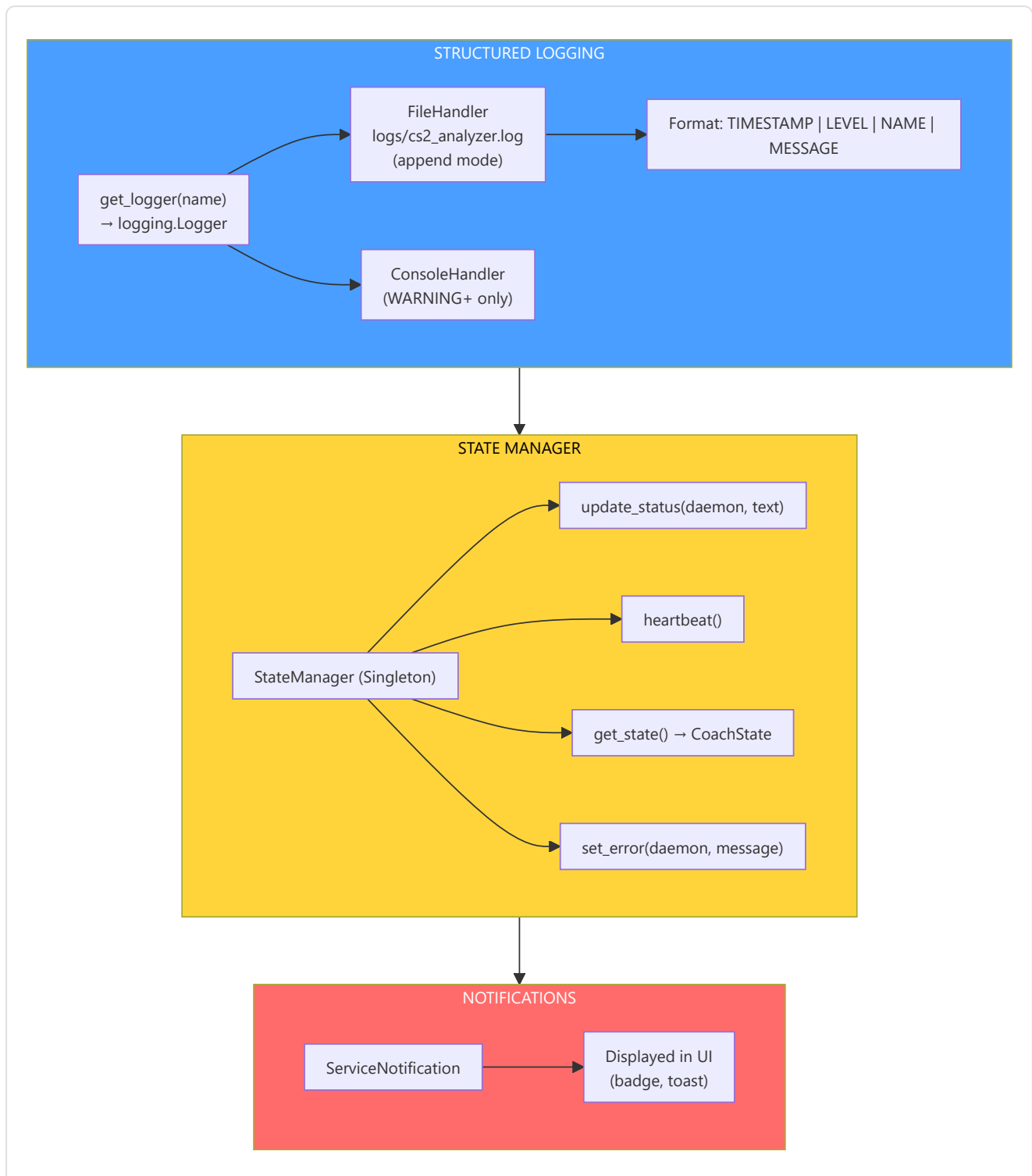
| Asset                         | Format        | Typical size | Cache                    |
|-------------------------------|---------------|--------------|--------------------------|
| Map texture (radar)           | PNG 1024×1024 | ~500KB       | Yes (in-memory)          |
| Player icons                  | PNG 32×32     | ~2KB         | Yes                      |
| Fonts (JetBrains Mono, YUPIX) | TTF           | ~200KB       | Yes (registered in Kivy) |
| Theme wallpapers              | PNG 1920×1080 | ~2MB         | Lazy load                |

## 12.12 Observability and Logging

**File:** `Programma_CS2_RENAN/observability/logger_setup.py` **Related files:**

`backend/storage/state_manager.py` , `backend/services/telemetry_client.py`

The observability system ensures that every significant event is **traceable, structured, and persistent**. The telemetry client ( `telemetry_client.py` ) uses `httpx` (asynchronous HTTP) for non-blocking sending of metrics and events, avoiding network latency affecting application performance.



### Daemons monitored by StateManager:

| Daemon            | Field in CoachState        | Typical values                        |
|-------------------|----------------------------|---------------------------------------|
| Scanner           | <code>hltv_status</code>   | "Scanning", "Paused", "Error"         |
| Digester (worker) | <code>ingest_status</code> | "Processing", "Idle", "Error"         |
| Teacher (trainer) | <code>ml_status</code>     | "Learning", "Idle", "Error"           |
| Global            | <code>status</code>        | "Paused", "Training", "Idle", "Error" |

**Sentry Integration** ( `observability/sentry_setup.py` ):

The system includes optional integration with **Sentry** for remote error tracking, with a **double opt-in** approach for privacy:

| Feature                        | Implementation                                                                                                 |
|--------------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>Double opt-in</b>           | The user must (1) set <code>SENTRY_DSN</code> as an environment variable AND (2) activate the flag in settings |
| <b>PII Scrubbing</b>           | All personal data (player names, Steam IDs, paths) are removed before sending                                  |
| <b>Breadcrumb sanitization</b> | Navigation breadcrumbs are cleaned of sensitive information                                                    |
| <b>Non-blocking</b>            | If Sentry is not configured, the system continues normally (Phase 4 of startup)                                |
| <b>Enriched context</b>        | Every error includes: app version, daemon state, demo count, operating system                                  |

**Logger Setup** ( `observability/logger_setup.py` ):

The centralized logging system provides structured logs with:

| Feature                  | Detail                                                                                                                                                                                     |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Format</b>            | <code>`TIMESTAMP</code>                                                                                                                                                                    |
| <b>File handler</b>      | <code>logs/cs2_analyzer.log</code> (append mode, automatic rotation)                                                                                                                       |
| <b>Console handler</b>   | Only <code>WARNING+</code> to avoid polluting output                                                                                                                                       |
| <b>Naming convention</b> | <code>get_logger("cs2analyzer.&lt;module&gt;")</code> — hierarchical namespace                                                                                                             |
| <b>Levels used</b>       | <code>DEBUG</code> (development), <code>INFO</code> (normal operations), <code>WARNING</code> (non-critical anomalies), <code>ERROR</code> (failures), <code>CRITICAL</code> (termination) |

**Training metrics exposed in CoachState:** `current_epoch`, `total_epochs`, `train_loss`, `val_loss`, `eta_seconds`, `belief_confidence`, `system_load_cpu`, `system_load_mem`.

**Centralized Error Code Registry** ( `observability/error_codes.py` ):

The system implements a formal registry of **24 error codes** classified by severity and module. Each code is defined as `ErrorCodeDef(NamedTuple)` with: `code`, `severity`, `module`, `description`, `remediation`.

| Prefix | Module           | Example                                | Typical Severity |
|--------|------------------|----------------------------------------|------------------|
| LS     | Logger Setup     | LS-01: RotatingFileHandler unavailable | MEDIUM           |
| RP     | RASP Guard       | RP-01: CS2_MANIFEST_KEY not set        | HIGH             |
| DA     | Data Access      | DA-01-03: Malformed JSON from DB       | LOW              |
| P      | Pipeline         | P7-01/P7-02: Critical pipeline errors  | HIGH             |
| F      | Feature/Fix      | F6-SE: Session engine error            | HIGH             |
| SE     | Session Engine   | SE-05: Critical session error          | HIGH             |
| IM     | Ingestion        | IM-03: Ingestion error                 | MEDIUM           |
| NN     | Neural Network   | NN-02: Neural network error            | MEDIUM           |
| CO     | Console Control  | CO-01/CO-03: Critical control errors   | HIGH             |
| R1     | Release/Manifest | R1-12: Release manifest error          | HIGH             |

**Severity** ( `Severity(Enum)` ): `LOW` , `MEDIUM` , `HIGH` , `CRITICAL` .

#### Utilities:

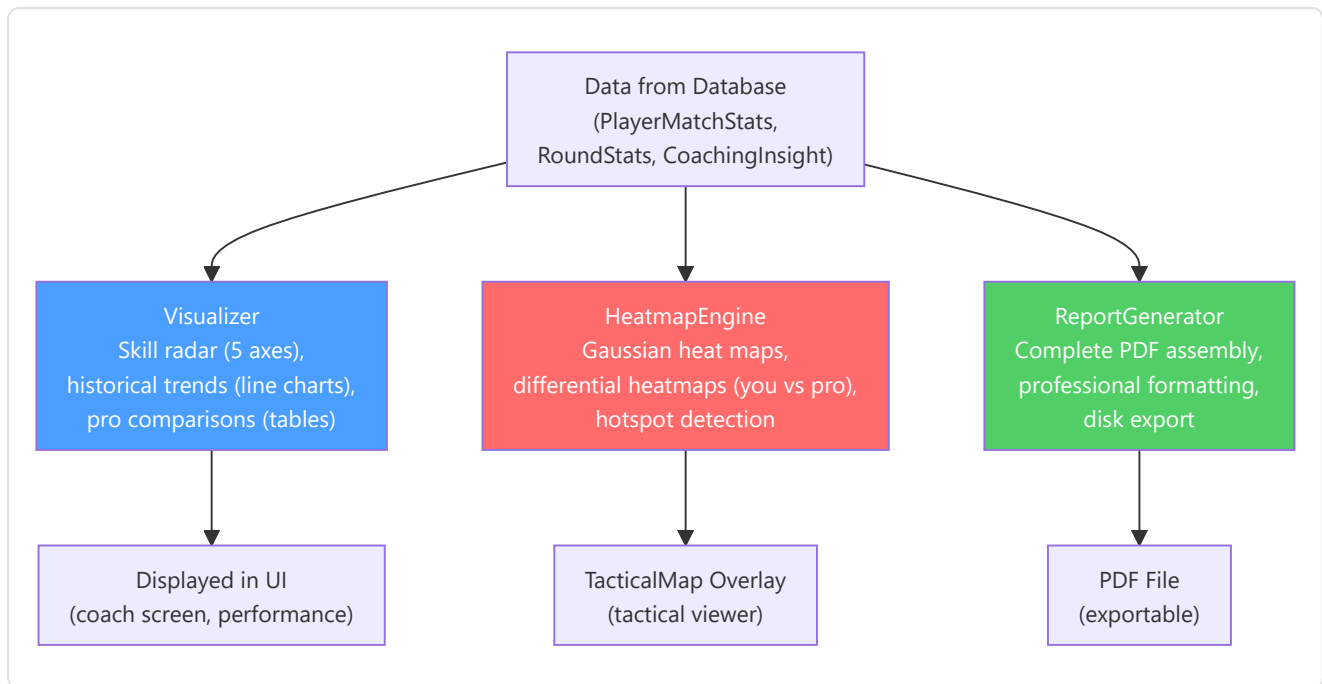
- `log_with_code(error_code, message) → str` — Prefixes the message with the formal code (e.g., `[LS-01] message` )
- `get_all_codes() → list[dict]` — All codes as a list of dictionaries for programmatic access

**Exception Hierarchy** ( `observability/exceptions.py` ): Base class `CS2AnalyzerError(Exception)` with subclasses: `ConfigurationError` , `DatabaseError` , `IngestionError` , `TrainingError` , `IntegrationError` , `UIError` . Each exception can carry an `error_code: ErrorCode | None` for correlation with the registry.

## 12.13 Reporting and Visualization

**File:** `Programma_CS2_RENAN/reporting/visualizer.py` , `report_generator.py` **Related files:** `backend/processing/heatmap_engine.py`

The reporting system transforms raw data into **understandable visualizations** for the user.



### HeatmapEngine — Technical details:

| Feature                 | Detail                                                                             |
|-------------------------|------------------------------------------------------------------------------------|
| <b>Thread-safety</b>    | <code>generate_heatmap_data()</code> runs in a separate thread (does not block UI) |
| <b>Texture creation</b> | Only in the main thread (Kivy OpenGL requirement)                                  |
| <b>Type</b>             | 2D Gaussian occupancy (parametric blur kernel)                                     |
| <b>Differential</b>     | Subtracts pro heatmap from user heatmap → red (too much time), blue (too little)   |
| <b>Hotspot</b>          | Identifies position clusters for positional training                               |

**MatchVisualizer** ( `reporting/visualizer.py` ) — specialized rendering methods:

The MatchVisualizer extends the reporting capability with 6 high-quality rendering methods (cf. section 12.26 for algorithmic details):

| Method                                              | Input              | Output           | Description                                       |
|-----------------------------------------------------|--------------------|------------------|---------------------------------------------------|
| <code>render_skill_radar(user, pro)</code>          | Metrics dict       | PNG radar chart  | 5-axis pentagon with pro baseline overlay         |
| <code>render_trend_chart(history, metric)</code>    | Historical list    | PNG line chart   | Temporal trend with moving average                |
| <code>render_heatmap(positions, map_name)</code>    | Tick coordinates   | PNG heatmap      | 2D Gaussian on map texture                        |
| <code>render_differential(user_pos, pro_pos)</code> | Two position sets  | PNG diff heatmap | Red (excess) / Blue (deficit) / Green (alignment) |
| <code>render_critical_moments(events)</code>        | Round events list  | PNG timeline     | Color markers for kill/death/bomb                 |
| <code>render_comparison_table(user, pro)</code>     | Two stats profiles | HTML/PNG table   | Percentage delta for each metric                  |

**ReportGenerator** ( `reporting/report_generator.py` ):

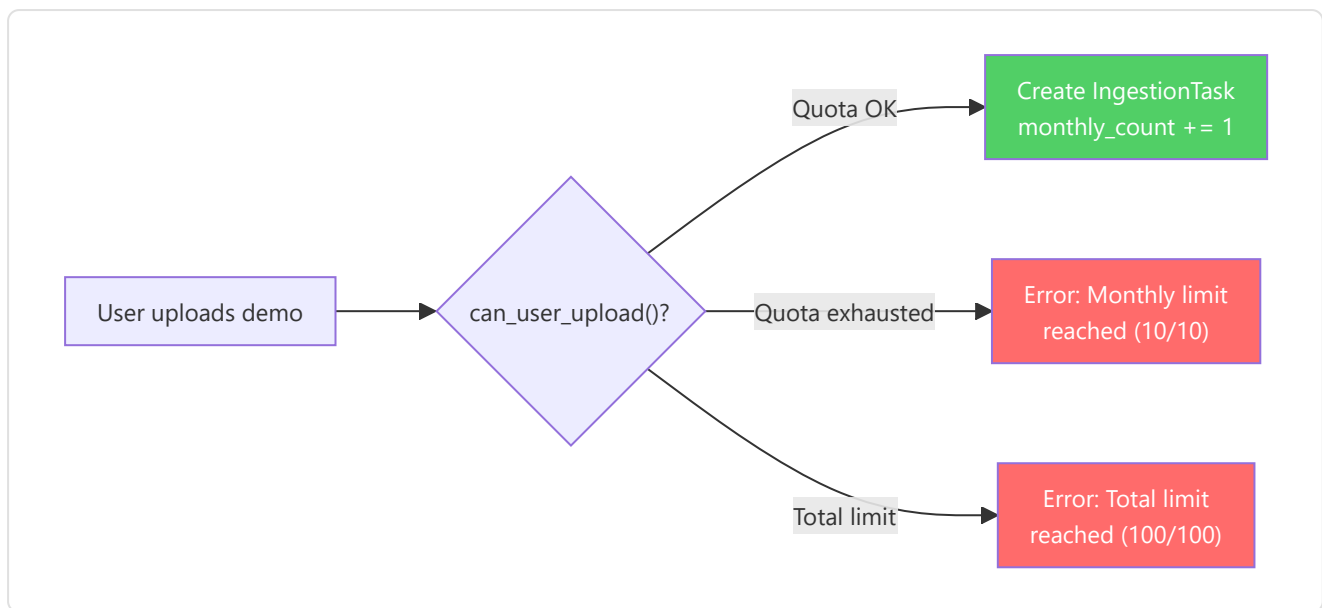
Assembles all visual elements into a **complete PDF document**:

1. **Header**: Player name, date, demo analyzed
2. **Executive Summary**: Key metrics (KPR, ADR, KAST, Rating)
3. **Radar Chart**: 5-axis comparison with pro baseline
4. **Trend Charts**: Trends of the last N matches
5. **Differential Heatmap**: Where to position better
6. **Critical Moments**: The 5 decisive moments of the match
7. **Coach's Advice**: Top-3 prioritized insights
8. **Footer**: Generated by Macena CS2 Analyzer v.X.X

## 12.14 Quota and Limit Management

The system implements a **monthly quota** mechanism to prevent abuse of processing resources and guarantee fair load distribution.

| Limit                         | Value     | Enforcement                                                   |
|-------------------------------|-----------|---------------------------------------------------------------|
| <b>MAX_DEMOS_PER_MONTH</b>    | 10        | <code>StorageManager.can_user_upload()</code>                 |
| <b>MAX_TOTAL_DEMOS</b>        | 100       | <code>StorageManager.can_user_upload()</code>                 |
| <b>MIN_DEMOS_FOR_COACHING</b> | 10        | Threshold for full personalized coaching                      |
| <b>Monthly reset</b>          | Automatic | <code>PlayerProfile.last_upload_month</code> vs current month |



## 12.15 Fault Tolerance and Recovery

The system is designed to **never lose data** and to **recover automatically** from almost all types of failure.

## FAULT TOLERANCE MECHANISMS

## Zombie Task Cleanup

On startup: 'processing' tasks  
→ reset to 'queued'

Automatic recovery without loss

## Automatic Backup

On startup: 'startup\_auto' checkpoint

Rotation: 7 daily + 4 weekly

Full database copy

## Service Supervisor

Monitors external services

Auto-restart with exponential backoff

Max 3 attempts/hour

## HLTV Circuit Breaker

MAX\_FAILURES=10, RESET=3600s

State: CLOSED→OPEN→HALF\_OPEN

Prevents cascade failure

## Connection Pooling

20 persistent connections

30s timeout for WAL contention

Automatic health checks

## Graceful Degradation

Coaching: 4 fallback levels

GhostEngine: (0,0) on error

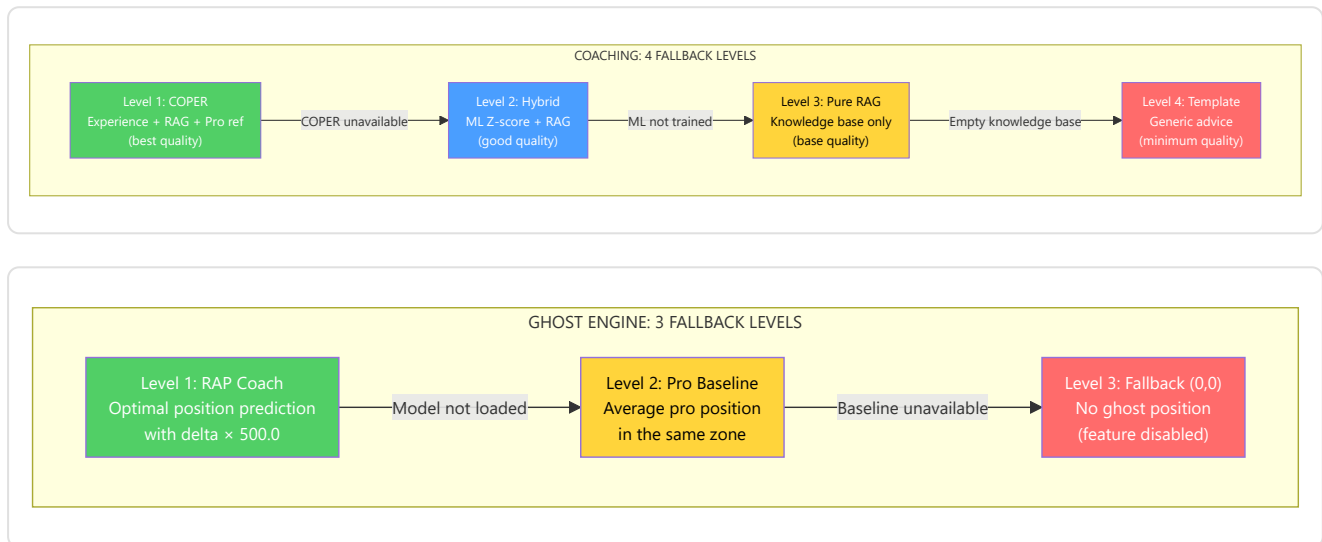
Every service has a plan B

**Failure and recovery matrix:**

| Failure scenario                  | Recovery mechanism                                                                                                                                                                                                                                   | Data loss                         |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| Application crash during parsing  | Zombie Task Cleanup on restart                                                                                                                                                                                                                       | None — task restarted             |
| Database corruption               | Restore from most recent automatic backup                                                                                                                                                                                                            | Up to 24 hours of data            |
| HLTV service unreachable          | Circuit Breaker <code>_CircuitBreaker</code> (MAX_FAILURES=10, RESET_WINDOW_S=3600): after 10 consecutive failures the circuit opens and blocks requests for 1 hour, then passes to HALF_OPEN for a test. Prevents cascade failure on external APIs. | None — pro data delayed           |
| ML model not loadable             | Fallback to random weights (GhostEngine) or base coaching                                                                                                                                                                                            | None — degraded quality           |
| Insufficient RAM during training  | Automatic early stopping, checkpoint saved                                                                                                                                                                                                           | None — last valid checkpoint      |
| Disk full                         | Database Governor detects and notifies via ServiceNotification                                                                                                                                                                                       | Prevention — no data written      |
| Corrupt/truncated demo            | IntegrityChecker rejects before parsing                                                                                                                                                                                                              | None — demo ignored with warning  |
| Ollama unavailable                | LLM fallback to pure RAG, then base coaching                                                                                                                                                                                                         | None — degraded narrative quality |
| Steam/FACEIT API timeout          | Retry with exponential backoff (max 3 attempts)                                                                                                                                                                                                      | None — profile not updated        |
| Playwright browser crash          | BrowserManager: recreates instance, retries operation                                                                                                                                                                                                | None — HLTV scraping delayed      |
| WAL conflict (lock contention)    | 30s timeout with automatic retry                                                                                                                                                                                                                     | None — operation delayed          |
| Corrupt ML checkpoint             | Fallback to previous checkpoint (versioning)                                                                                                                                                                                                         | Partial — loses last training     |
| Zero-norm query vector (M-07)     | <code>VectorIndex.search()</code> returns <code>None</code> with warning — RAG pipeline continues without crash                                                                                                                                      | None — empty RAG result           |
| Corrupt settings.json file (M-08) | <code>load_user_settings()</code> validates structure, falls back to default if non-dict                                                                                                                                                             | None — default settings           |
| Logger not initialized (C-09)     | Early bootstrap uses <code>print()</code> before logger init                                                                                                                                                                                         | None — messages on stdout         |

| Failure scenario                           | Recovery mechanism                                                                                                                 | Data loss                       |
|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| DB handle leak on integrity check (H-02)   | <code>_verify_integrity()</code> uses <code>try/finally</code> + <code>PRAGMA quick_check</code>                                   | None — connection always closed |
| Masked batch errors in RAP training (H-03) | Removed <code>except (KeyError, TypeError): continue</code> — <code>train_step()</code> requires dict with <code>'loss'</code> key | None — errors now visible       |

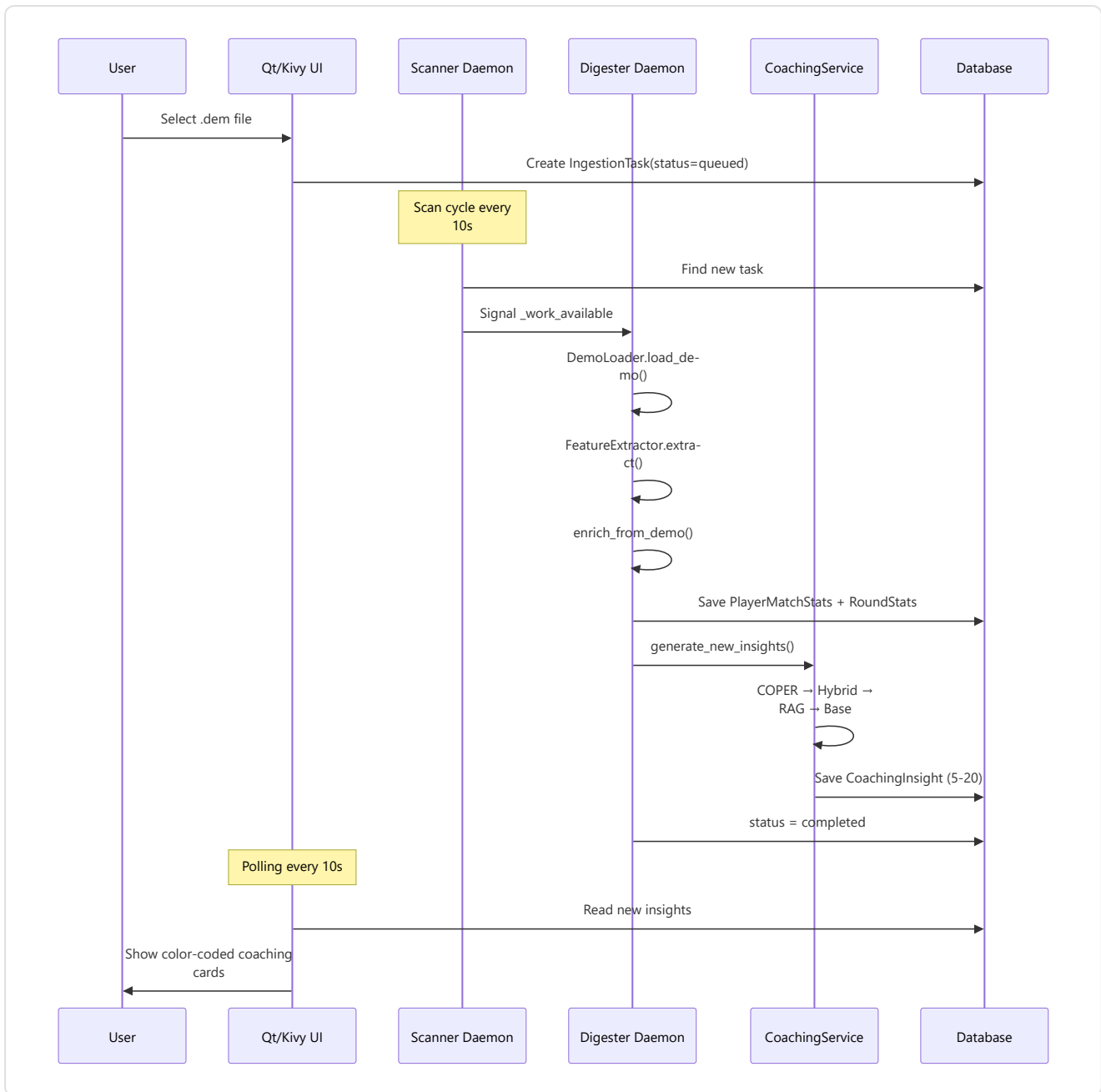
### Graceful degradation — The Fallback Chain:



## 12.16 Complete User Journey — 4 Main Flows

This section describes the **4 main flows** a user goes through while using Macena CS2 Analyzer.

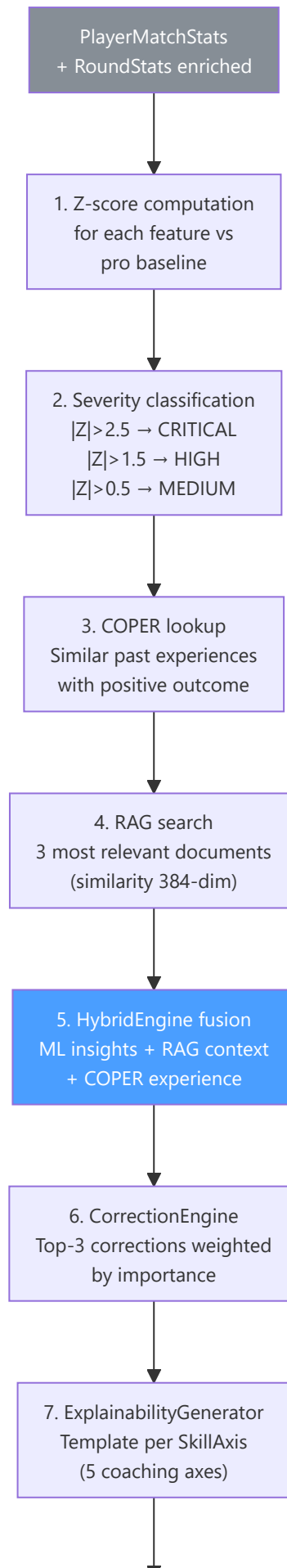
## Flow 1: Demo Upload and Analysis



### Detail: Internal CoachingService Pipeline

When `generate_new_insights()` is invoked, the CoachingService internally runs:





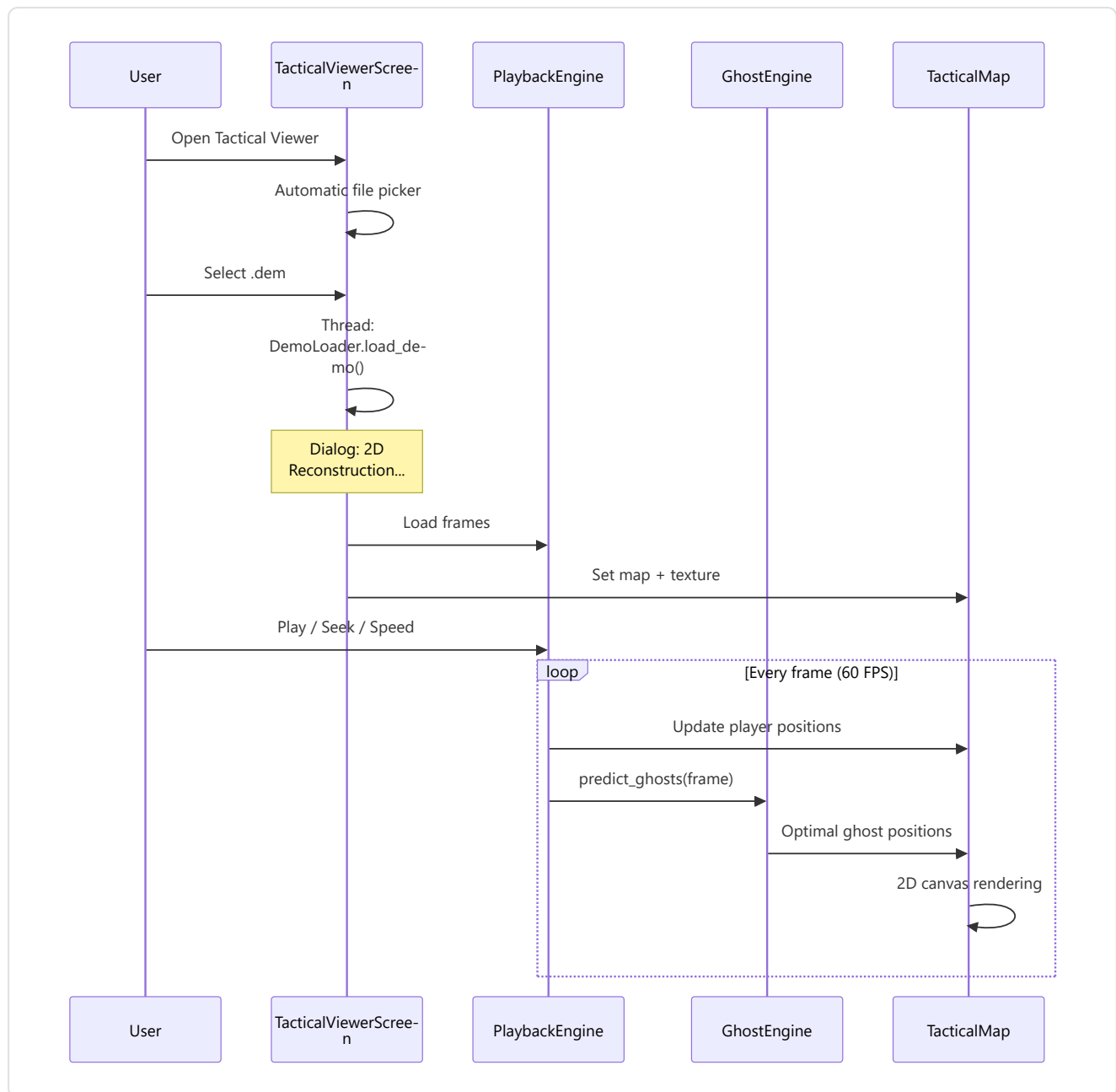
8. Persistence  
CoachingInsight in DB  
(5-20 per match)

**LessonGenerator** — generation of structured lessons from demo:

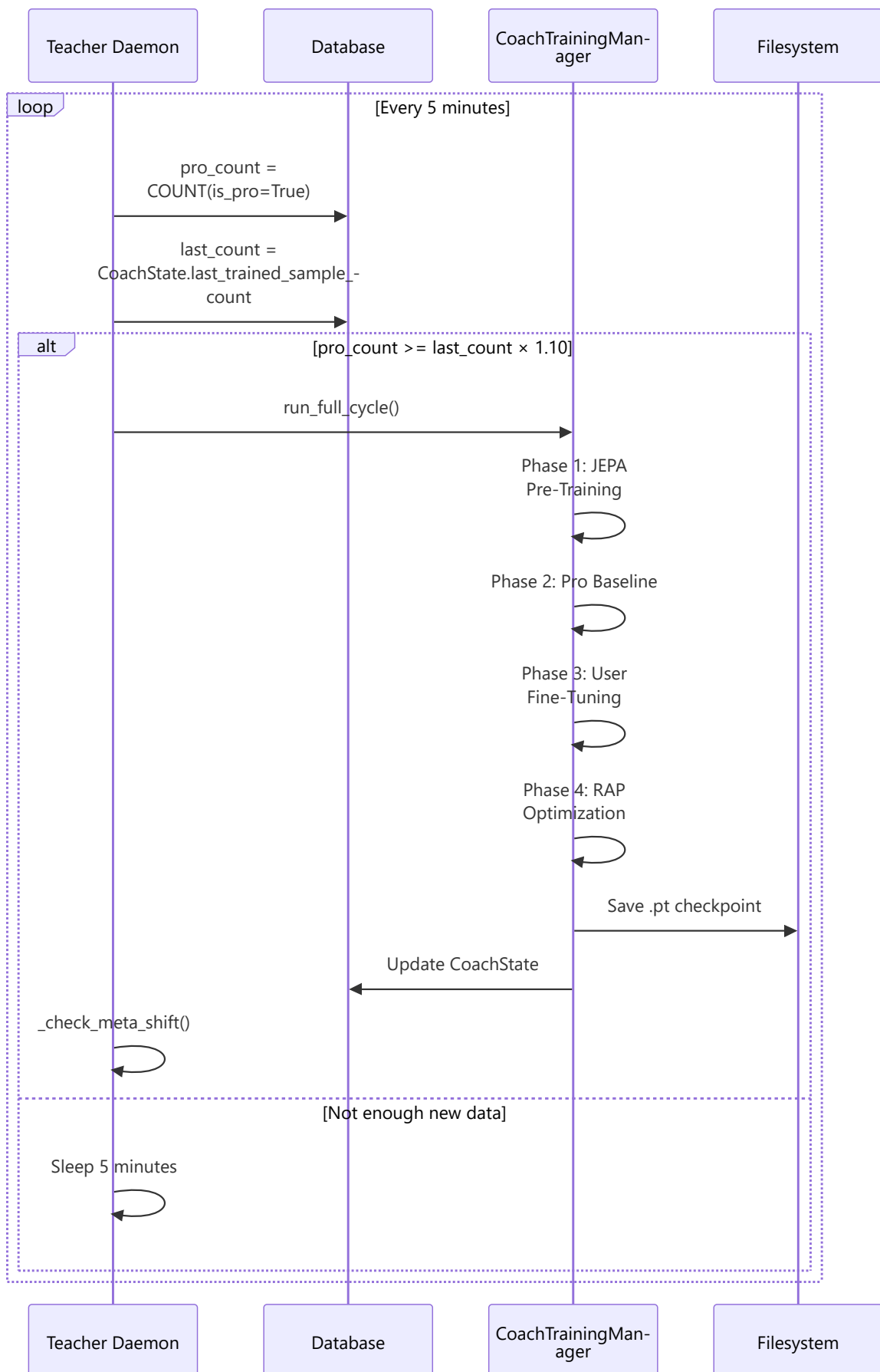
| Parameter     | Value                                                                                                  | Meaning                                                |
|---------------|--------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| ADR_STRONG    | 75                                                                                                     | $ADR \geq 75 \rightarrow$ "Good average damage"        |
| HS_STRONG     | 0.40                                                                                                   | $HS\% \geq 40\% \rightarrow$ "Good headshot accuracy"  |
| KAST_STRONG   | 0.70                                                                                                   | $KAST \geq 70\% \rightarrow$ "Good round contribution" |
| Output format | Section 1 (Overview) + Section 2 (Strengths) + Section 3 (Areas for improvement) + Section 4 (Pro tip) | Standard lesson structure                              |

The LessonGenerator operates in dual mode: if Ollama is available, it generates natural language lessons via `LLMService.generate_lesson()` . If not available, it uses structured templates with numeric data inserted into preformed phrases.

## Flow 2: Tactical Visualization



## Flow 3: Automatic ML Retraining



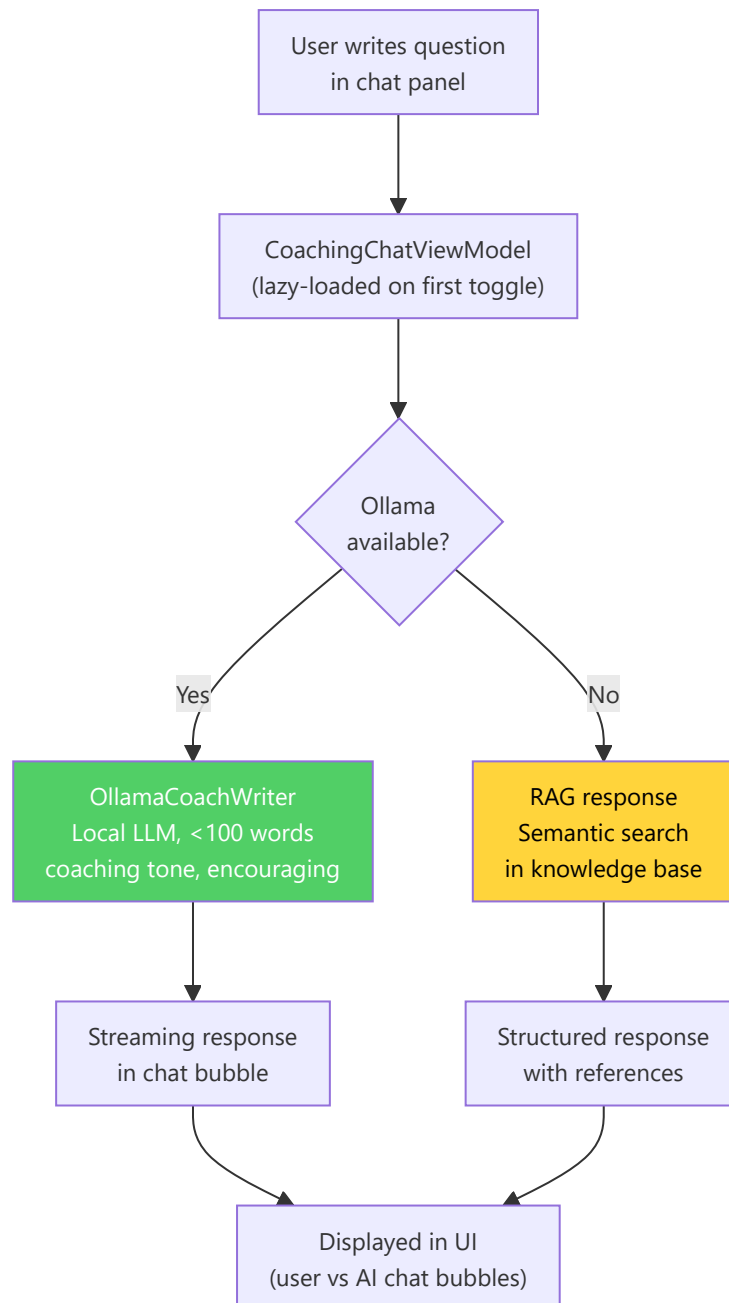
Flow 4: AI Chat with the Coach

CoachingDialogueEngine — Multi-Turn Pipeline:

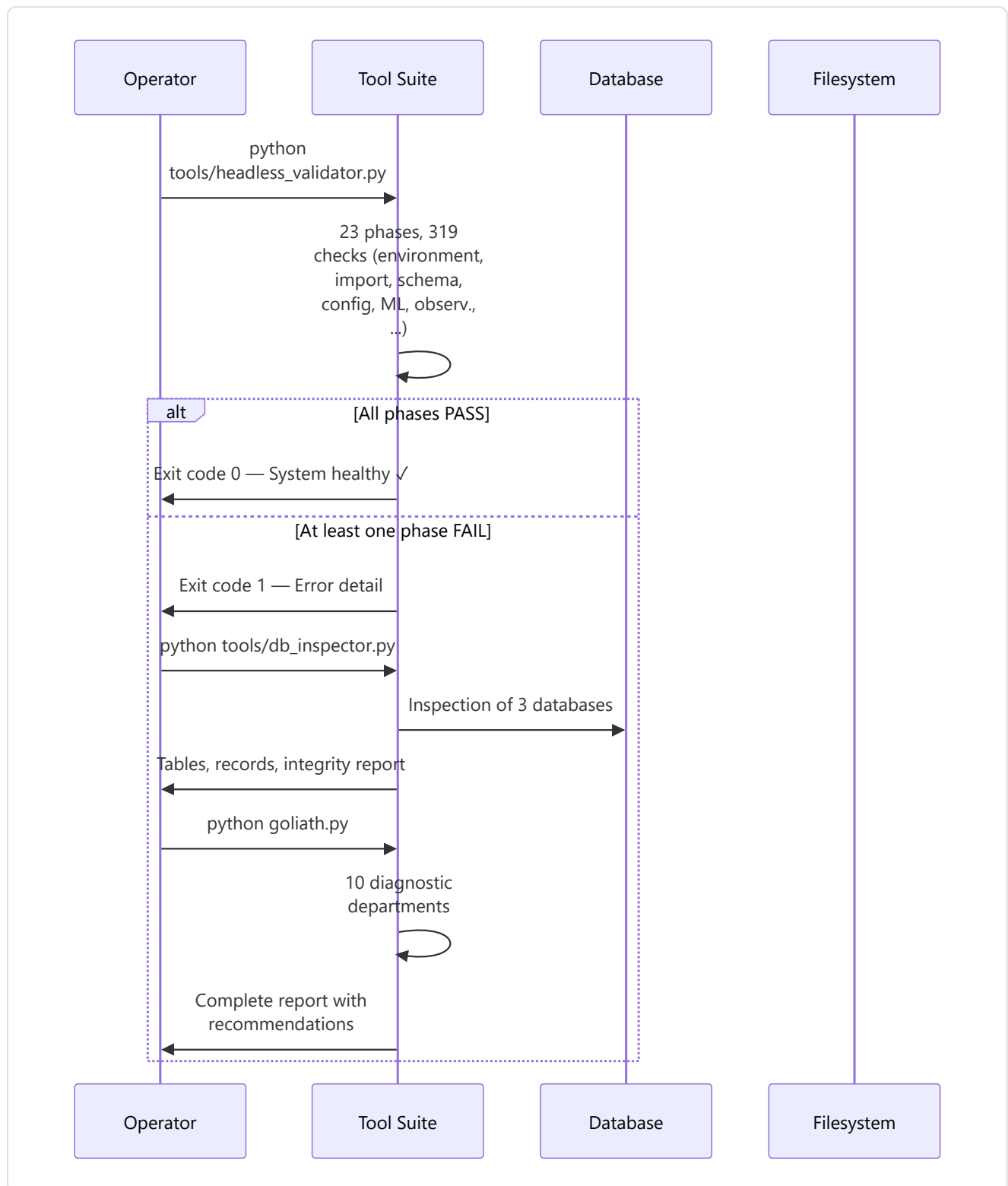
The AI dialogue follows a 5-phase pipeline for each user message:

| Phase | Component             | Description                                                                               |
|-------|-----------------------|-------------------------------------------------------------------------------------------|
| 1     | Intent Classification | Classifies the question: coaching, stats query, comparison, general                       |
| 2     | Context Retrieval     | Retrieves the last N matches, recent insights, player profile                             |
| 3     | RAG Augmentation      | Searches the knowledge base for the 3 most relevant documents (similarity search 384-dim) |
| 4     | COPER Augmentation    | If available, adds past coaching experiences with positive outcome                        |
| 5     | Response Generation   | Generates response via Ollama (if available) or structured RAG template                   |

| Parameter            | Value                   | Purpose                                 |
|----------------------|-------------------------|-----------------------------------------|
| MAX_CONTEXT_TURNS    | 6                       | Maximum number of turns kept in context |
| MAX_RESPONSE_WORDS   | 100                     | Word limit for LLM response             |
| SIMILARITY_THRESHOLD | 0.5                     | Minimum threshold for RAG results       |
| Tone                 | "coaching, encouraging" | System prompt for positive tone         |



## Flow 5: Diagnostics and Maintenance



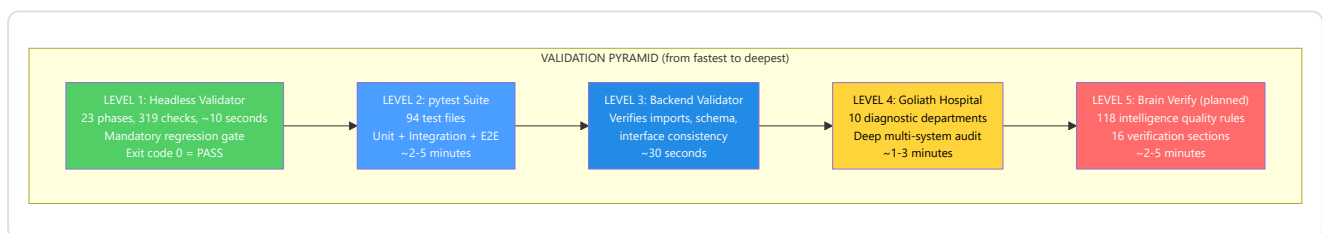
**Tool matrix by scenario:**

| Scenario                | Recommended tool                           | Time  |
|-------------------------|--------------------------------------------|-------|
| Quick check post-deploy | <code>headless_validator.py</code>         | ~10s  |
| Suspicious database     | <code>db_inspector.py</code>               | ~30s  |
| Demo not parsed         | <code>demo_inspector.py</code>             | ~5s   |
| Training diverges       | <code>Ultimate_ML_Coach_Debugger.py</code> | ~2min |
| Complete check-up       | <code>Goliath_Hospital.py</code>           | ~3min |
| Deep ML audit           | <code>brain_verify.py</code>               | ~5min |

## 12.17 Tool Suite — Validation and Diagnostics ( `tools/` )

**Directory:** `tools/` (root) + `Programma_CS2_RENAN/tools/` **Main files:** `headless_validator.py` , `db_inspector.py` , `demo_inspector.py` , `brain_verify.py` (planned, not yet implemented), `Goliath_Hospital.py` , `Ultimate_ML_Coach_Debugger.py` , `_infra.py`

The tool suite is a **collection of 41 scripts** distributed across two directories ( `tools/` root with 29 scripts and `Programma_CS2_RENAN/tools/` with 12 scripts) that form a multi-level validation pyramid. Each tool has a precise purpose and can be executed independently, but together they form a quality assurance system covering every aspect of the project.



**Headless Validator** ( `tools/headless_validator.py` ) — the mandatory regression gate (Dev Rule 9). Executed after **every** development task with 23 phases and 319 checks:

| Phase | Verification          | Detail                                                                                                                                                    |
|-------|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | <b>Environment</b>    | Python $\geq$ 3.10, critical dependencies present (torch, kivy, sqlmodel, demoparser2)                                                                    |
| 2     | <b>Core Import</b>    | <code>config.py</code> , <code>spatial_data.py</code> , <code>lifecycle.py</code> — the fundamental modules load without errors                           |
| 3     | <b>Backend Import</b> | <code>nn/</code> , <code>processing/</code> , <code>storage/</code> , <code>services/</code> , <code>coaching/</code> — all backend subsystems importable |
| 4     | <b>DB Schema</b>      | The 21 SQLAlchemy tables are created correctly, the relationships are valid                                                                               |
| 5     | <b>Configuration</b>  | <code>METADATA_DIM</code> , paths, constants — consistent and reachable values                                                                            |
| 6     | <b>ML Smoke</b>       | Model instantiation (JEPA, RAP, MoE) with random weights — verify dimensions and forward pass                                                             |
| 7     | <b>Observability</b>  | <code>get_logger()</code> working, <code>StateManager</code> initializable, writable log path                                                             |

**Shared Infrastructure** ( `tools/_infra.py` ):

| Component                     | Role                                                                                            |
|-------------------------------|-------------------------------------------------------------------------------------------------|
| <code>BaseValidator</code>    | Base class for all validators — unified console output, pass/fail/warning count                 |
| <code>path_stabilize()</code> | <code>sys.path</code> normalization to avoid inconsistent relative imports                      |
| <code>Console</code> (Rich)   | Color output with tables, progress bars, status emoji                                           |
| <code>Severity</code> enum    | 4 levels: <code>INFO</code> , <code>WARNING</code> , <code>ERROR</code> , <code>CRITICAL</code> |

**DB Inspector** ( `tools/db_inspector.py` ) — deep database inspection:

- Opens `database.db` and `hlrv_metadata.db` separately
- For each table: counts records, verifies schema, checks index integrity
- Displays space metrics (file size, WAL pages, fragmentation)
- Detects anomalies: unexpected empty tables, orphan records, out-of-range timestamps
- Output: color Rich table with status for each table

**Demo Inspector** ( `tools/demo_inspector.py` ) — demo file inspection:

- Verifies magic bytes ( `PBDEMS2\0` for CS2, `HL2DEMO\0` for legacy)
- Checks file size (1KB min, 5GB max)
- Extracts header metadata without full parsing
- Detects partial corruption (truncation, damaged header)
- Output: report on demo validity with error details if invalid

**Brain Verify** ( `tools/brain_verify.py` + `brain_verification/` , 16 sections) — *(planned, not yet implemented)*:

The intelligence verification system is designed with **16 thematic sections** that will cover **118 rules** of quality:



## BRAIN VERIFY — 16 SECTIONS (118 RULES)

§1 Dimensional Contracts  
METADATA\_DIM, latent\_dim,  
input shapes

§2 Forward Pass  
Smoke test all models  
with synthetic input

§3 Loss Functions  
Gradient flow, NaN check,  
loss monotonicity

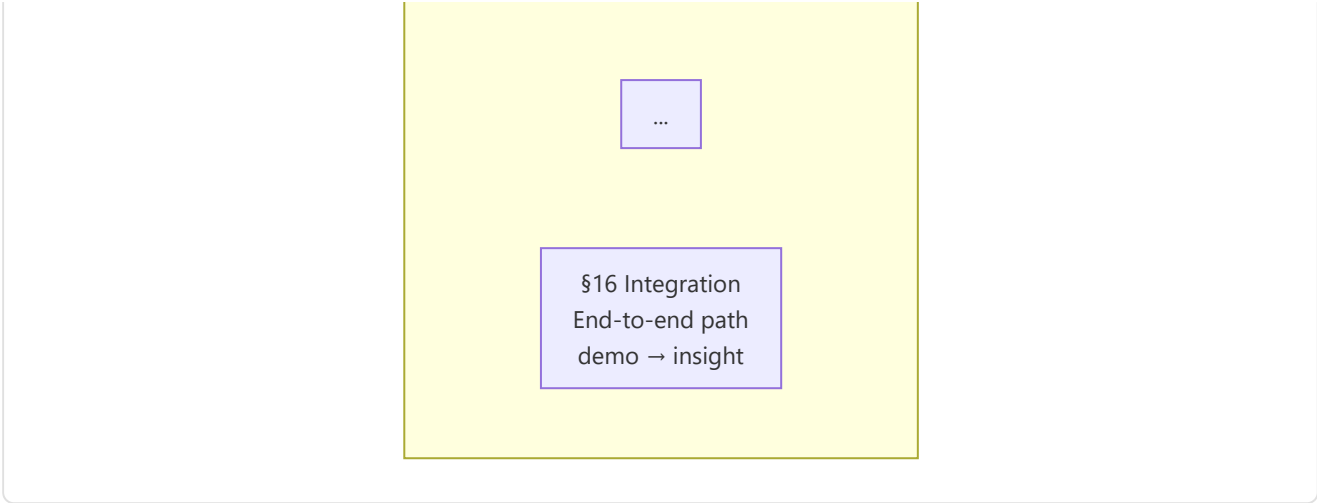
§4 Training Pipeline  
DataLoader, optimizer,  
scheduler config

§5 Feature Engineering  
Vectorizer alignment,  
normalization bounds

§6 Coaching Pipeline  
COPER → Hybrid → RAG  
fallback chain

§7 Data Integrity  
DB schema, FK constraints,  
orphan detection

§8 Concept Alignment  
16 coaching concepts,  
label correctness



**Goliath Hospital** ( `tools/Goliath_Hospital.py` ) — multi-department diagnostics:

The "Goliath Hospital" is the **most complete diagnostic system** of the project, organized as a hospital with 10 specialized departments:

| Department | Name                 | Checks                                               |
|------------|----------------------|------------------------------------------------------|
| 1          | Emergency Room (ER)  | Critical imports, immediate crashes, path resolution |
| 2          | Radiology            | File/directory structure, missing files, permissions |
| 3          | Pathology            | DB schema, data integrity, anomalous records         |
| 4          | Cardiology           | Session Engine, daemon heartbeat, IPC                |
| 5          | Neurology            | ML models, forward pass, gradient flow               |
| 6          | Oncology             | Dead code, unused imports, circular dependencies     |
| 7          | Pediatrics           | Onboarding, first-user flows, wizard                 |
| 8          | Intensive Care (ICU) | Concurrency, race conditions, WAL contention         |
| 9          | Pharmacy             | Dependencies, versions, compatibility                |
| 10         | Tool Clinic          | Validation of other tools (meta-test)                |

**Ultimate ML Coach Debugger** ( `tools/Ultimate_ML_Coach_Debugger.py` ) — neural belief falsification:

This tool performs a **3-phase audit** on the ML pipeline:

1. **Structural Verification** — Tensor sizes, model architecture, learnable parameters
2. **Behavioral Verification** — Forward pass with real data, gradient flow, loss convergence
3. **Belief Falsification** — Tests whether the model "believes" wrong things: overconfident predictions, systematic biases, degenerate patterns

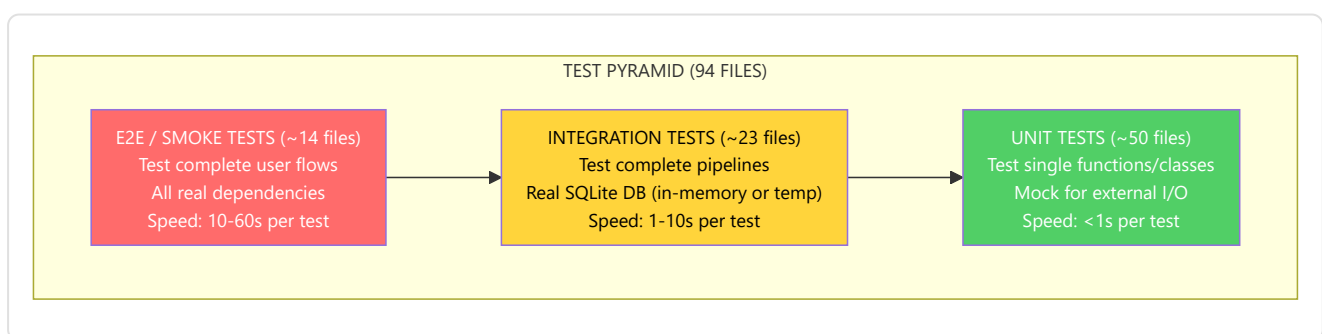
**Other tools:**

| Tool                                    | File                     | Purpose                                                          |
|-----------------------------------------|--------------------------|------------------------------------------------------------------|
| <code>backend_validator.py</code>       | <code>tools/</code>      | Verifies backend import and interface consistency                |
| <code>build_tools.py</code>             | <code>tools/</code>      | PyInstaller build automation                                     |
| <code>user_tools.py</code>              | <code>tools/</code>      | Utilities for the end user (reset, export)                       |
| <code>dev_health.py</code>              | <code>tools/</code>      | Quick development health check                                   |
| <code>context_gatherer.py</code>        | <code>tools/</code>      | Gathers context for debug reports                                |
| <code>dead_code_detector.py</code>      | <code>tools/</code>      | Identifies unreferenced dead code                                |
| <code>sync_integrity_manifest.py</code> | <code>tools/</code>      | Updates <code>integrity_manifest.json</code> with SHA-256 hashes |
| <code>project_snapshot.py</code>        | <code>tools/</code>      | Complete snapshot of project state                               |
| <code>ui_diagnostic.py</code>           | <code>tools/</code>      | UI-specific diagnostics (Qt/Kivy)                                |
| <code>build_pipeline.py</code>          | Root <code>tools/</code> | Complete build pipeline                                          |
| <code>Feature_Audit.py</code>           | Root                     | Audit of the 25-dim feature vector                               |
| <code>Sanitize_Project.py</code>        | Root                     | Cleanup of temporary files, cache, artifacts                     |
| <code>audit_binaries.py</code>          | Root                     | Verifies integrity of executables and .pt                        |

## 12.18 Test Suite Architecture ( `tests/` )

**Directory:** `Programma_CS2_RENAN/tests/` **Total files:** 94 test files + `conftest.py` + 10 forensics scripts + 15 verification scripts

The test suite is organized according to the **test pyramid principle**: many unit tests (fast, isolated), fewer integration tests (slower, with real dependencies), and few end-to-end tests (complete but costly).



**Conftest** ( `tests/conftest.py` ) — shared fixtures:

| Fixture                         | Scope                 | Description                                                                 |
|---------------------------------|-----------------------|-----------------------------------------------------------------------------|
| <code>test_db</code>            | <code>function</code> | In-memory SQLite database with all tables created, auto-cleanup             |
| <code>sample_match_stats</code> | <code>function</code> | <code>PlayerMatchStats</code> with realistic data derived from real matches |
| <code>mock_demo_data</code>     | <code>function</code> | Mock <code>DemoData</code> with frames and events for parsing tests         |
| <code>tmp_models_dir</code>     | <code>function</code> | Temporary directory for <code>.pt</code> checkpoints                        |
| <code>coaching_service</code>   | <code>function</code> | <code>CoachingService</code> configured with test DB                        |

**Automated Suite** ( `tests/automated_suite/` ):

| Category          | File                              | Coverage                                                     |
|-------------------|-----------------------------------|--------------------------------------------------------------|
| <b>Smoke</b>      | <code>test_smoke_*.py</code>      | Critical imports, model instantiation, DB connection         |
| <b>Unit</b>       | <code>test_unit_*.py</code>       | Pure functions, calculations, transformations                |
| <b>Functional</b> | <code>test_functional_*.py</code> | Complete pipelines, workflows                                |
| <b>E2E</b>        | <code>test_e2e_*.py</code>        | Demo upload → parsing → coaching → insight                   |
| <b>Regression</b> | <code>test_regression_*.py</code> | Specific bug fixes (G-01 label leakage, G-07 Bayesian, etc.) |

**Test per subsystem:**

| Subsystem    | Test files                                                                                                             | What they test                                            |
|--------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| NN Core      | <code>test_jepa_model.py</code> , <code>test_rap_model.py</code> ,<br><code>test_advanced_nn.py</code>                 | Forward pass, tensor dimensions, gradient flow            |
| Coaching     | <code>test_coaching_service.py</code> , <code>test_hybrid_engine.py</code> ,<br><code>test_correction_engine.py</code> | Coaching pipeline, insight prioritization, fallback chain |
| Processing   | <code>test_vectorizer.py</code> , <code>test_heatmap.py</code> ,<br><code>test_feature_eng.py</code>                   | Feature extraction, normalization, METADATA_DIM=25        |
| Storage      | <code>test_database.py</code> , <code>test_models.py</code> , <code>test_backup.py</code>                              | CRUD, schema, backup/restore, WAL mode                    |
| Data Sources | <code>test_demo_parser.py</code> , <code>test_hltv_scraper.py</code> ,<br><code>test_steam_api.py</code>               | Parsing, scraping, API timeout/retry                      |
| Analysis     | <code>test_game_theory.py</code> , <code>test_belief.py</code> ,<br><code>test_role_engine.py</code>                   | Analysis engines, calibration, heuristics                 |
| Knowledge    | <code>test_rag.py</code> , <code>test_experience_bank.py</code> ,<br><code>test_knowledge_graph.py</code>              | Retrieval, COPER effectiveness, KG query                  |
| Ingestion    | <code>test_ingest_pipeline.py</code> , <code>test_registry.py</code>                                                   | Complete pipeline, deduplication                          |

**Forensics** ( `tests/forensics/` , 10 scripts):

Forensics scripts are diagnostic tools for post-mortem investigations:

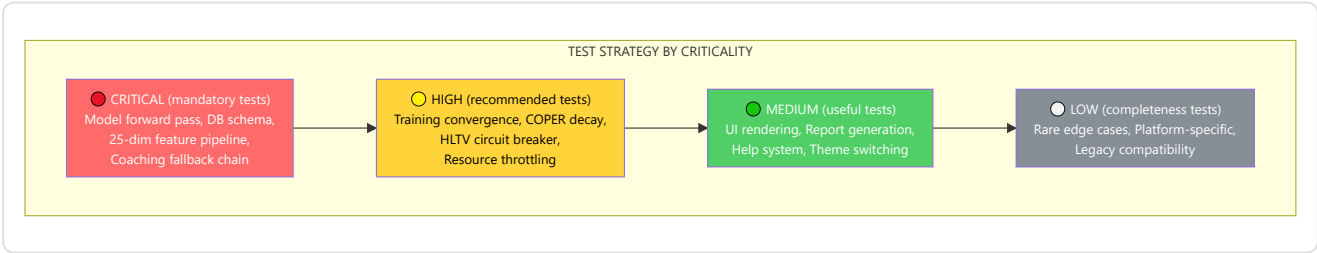
| Script                       | Purpose                                                                |
|------------------------------|------------------------------------------------------------------------|
| diagnose_training_failure.py | Analyzes training logs to identify divergence, NaN, gradient explosion |
| inspect_model_weights.py     | Weight distribution, layer statistics, dead neurons                    |
| replay_ingestion.py          | Re-runs ingestion of a specific demo with verbose logging              |
| trace_coaching_path.py       | Traces the path of an insight from demo to UI                          |
| db_consistency_check.py      | Verifies consistency across the 3 databases                            |

**Verification Scripts** (15 files in tests/ root):

One-shot verification scripts to validate specific aspects of the system:

| Script                      | Verification                                             |
|-----------------------------|----------------------------------------------------------|
| verify_feature_pipeline.py  | METADATA_DIM=25 respected in all paths                   |
| verify_training_cycle.py    | 4 training phases complete without error                 |
| verify_db_schema.py         | 21 tables present with correct schema                    |
| verify_coaching_pipeline.py | Demo → insight end-to-end path                           |
| verify_imports.py           | All modules importable without circular errors           |
| verify_rag_index.py         | Knowledge base indexed with correct dimensions (384-dim) |
| verify_pro_baseline.py      | Pro baseline loaded and valid                            |
| verify_hltv_sync.py         | HLTV sync service configured correctly                   |

**Test strategy by criticality:**



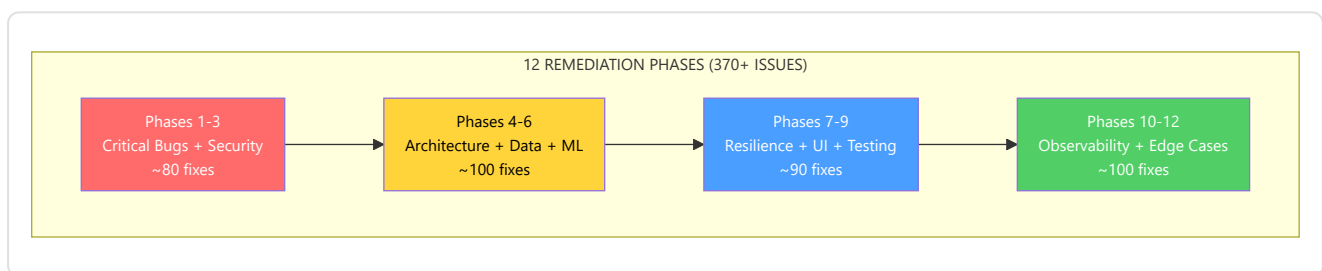
**Identified coverage gaps:**

The following modules have high complexity (>500 LOC) but limited test coverage:

| Module                                | LOC | Test coverage | Risk                                   |
|---------------------------------------|-----|---------------|----------------------------------------|
| <code>training_orchestrator.py</code> | 733 | Low           | HIGH — multi-phase orchestration       |
| <code>session_engine.py</code>        | 538 | Medium        | MEDIUM — testable only with subprocess |
| <code>coaching_service.py</code>      | 585 | Medium        | MEDIUM — 4 fallback levels             |
| <code>experience_bank.py</code>       | 751 | Low           | HIGH — complex COPER logic             |
| <code>tensor_factory.py</code>        | 686 | Low           | HIGH — DataLoader/Dataset              |
| <code>coach_manager.py</code>         | 878 | Low           | HIGH — global training state           |

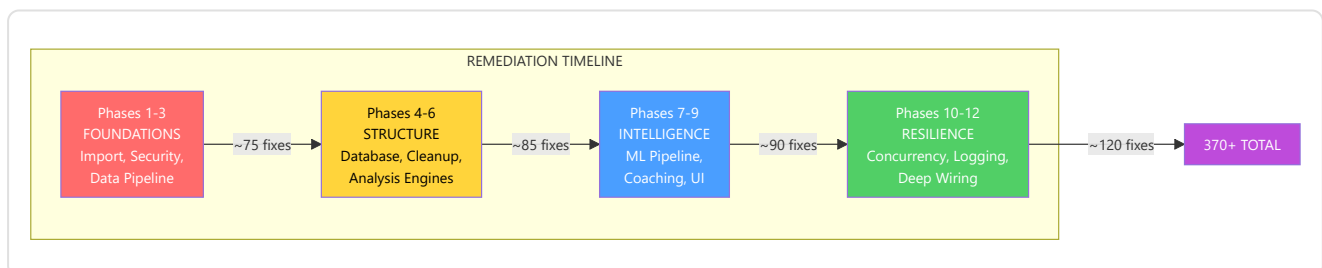
## 12.19 The 12 Phases of Systematic Remediation

The project went through a **12-phase remediation** process that overall resolved **370+ issues** identified during progressive quality audits. Each phase focused on a specific category of problems, from critical bug fixes to architectural restructuring.



**Detail per phase:**

| Phase | Focus                      | Issues resolved | Key example                                                                     |
|-------|----------------------------|-----------------|---------------------------------------------------------------------------------|
| 1     | Import and base structure  | ~20             | Circular imports, path resolution, missing <code>__init__.py</code>             |
| 2     | Security and secrets       | ~25             | Hard-coded API keys → env vars/keyring, input validation                        |
| 3     | Data pipeline and features | ~30             | G-01 label leakage, G-02 normalization bounds, feature alignment                |
| 4     | Database and schema        | ~30             | WAL mode enforcement, missing indexes, schema migration safety                  |
| 5     | Dead code and cleanup      | ~25             | G-06 elimination of <code>nn/advanced/</code> , unused imports, duplicate files |
| 6     | Analysis engines           | ~30             | Graceful degradation for all 11 engines, edge case handling                     |
| 7     | ML pipeline                | ~35             | G-07 Bayesian calibration, gradient clipping, checkpoint versioning             |
| 8     | Coaching and COPER         | ~30             | G-08 experience decay, RAG index validation, fallback chain                     |
| 9     | UI and UX                  | ~25             | F8-XX visual feedback, state consistency, error prevention                      |
| 10    | Resilience and concurrency | ~40             | F5-35 threading.Event, timeout enforcement, circuit breaker                     |
| 11    | Observability and logging  | ~35             | F5-33 structured logging, correlation IDs, Sentry integration                   |
| 12    | Deep debug and wiring      | ~45             | 18 issues: wiring verification, integration testing, edge case audit            |



### Directory `reports/` :

Each remediation phase produced a detailed report saved in the `reports/` directory. Each report includes:

- Numbered list of issues found with code (G-XX or F-XX)
- Problem description with affected files and lines
- Implemented solution with conceptual diff
- Verification: confirmation that the fix did not introduce regressions

### Key fixes by code (G-XX):

| Code        | Phase | Description                                                                                             | Impact                                    |
|-------------|-------|---------------------------------------------------------------------------------------------------------|-------------------------------------------|
| <b>G-01</b> | 3     | Label leakage in <code>ConceptLabeler.label_tick()</code> — future features leaked into training labels | CRITICAL — invalidated VL-JEPA training   |
| <b>G-02</b> | 4     | Danger zone in <code>vectorizer.py</code> — normalization without bounds check                          | HIGH — NaN propagation in training        |
| <b>G-06</b> | 5     | Dead code in <code>backend/nn/advanced/</code> — 3 unreferenced files                                   | MEDIUM — confusion and accidental imports |
| <b>G-07</b> | 7     | Bayesian death estimator not calibrated — static model without auto-calibration                         | HIGH — inaccurate predictions             |
| <b>G-08</b> | 8     | COPER experience bank without decay — stale experiences never removed                                   | MEDIUM — coaching based on outdated data  |

### Fixes per feature (F-XX):

| Code  | Area          | Description                                                                        |
|-------|---------------|------------------------------------------------------------------------------------|
| F3-29 | Processing    | ResNet resize inconsistent — image dimensions not normalized                       |
| F5-19 | Services      | Matplotlib rendering without error handling — crash on empty stats                 |
| F5-21 | Storage       | Explicit <code>session.commit()</code> inside context manager — double commit      |
| F5-22 | Security      | Hard-coded API keys in source — migrated to env vars + keyring                     |
| F5-33 | Observability | <code>print()</code> debugging — migrated to structured logging                    |
| F5-35 | Control       | <code>time.sleep()</code> loop for stop — migrated to <code>threading.Event</code> |
| F6-XX | Analysis      | Analysis engines without graceful degradation — crash on incomplete input          |
| F7-XX | Knowledge     | RAG without index validation — inconsistent embedding dimensions                   |
| F8-XX | UI            | Kivy widget without visual feedback — silent actions confuse the user              |

## 12.20 Pre-commit Hooks and Quality Gates

**File:** `.pre-commit-config.yaml`

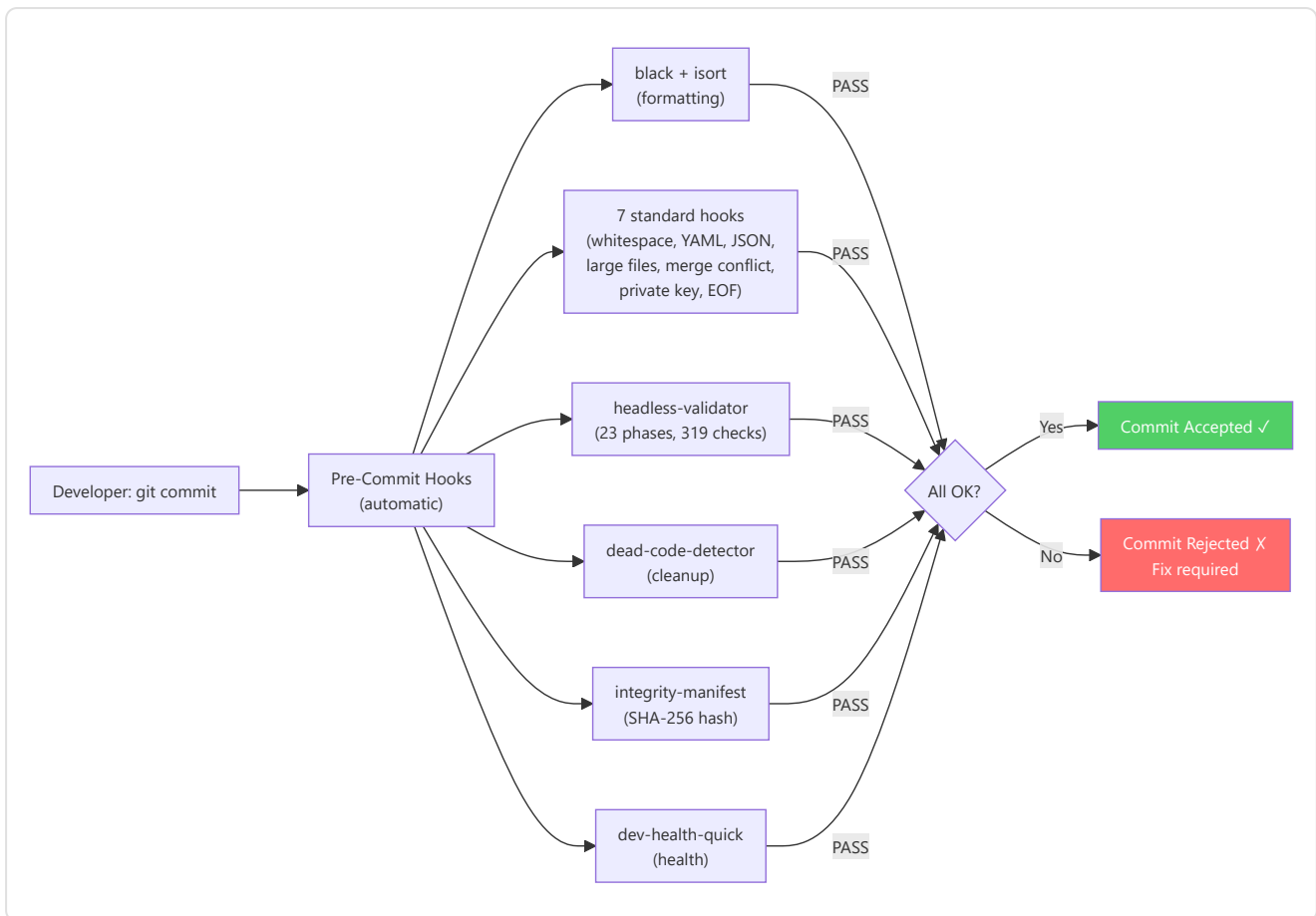
The project uses a **pre-commit hooks** system that activates automatically before every commit, preventing non-compliant code from reaching the repository.

### 4 Local Hooks (project custom):

| Hook                                  | Script                                        | Timeout | Description                                                |
|---------------------------------------|-----------------------------------------------|---------|------------------------------------------------------------|
| <code>headless-validator</code>       | <code>tools/headless_validator.py</code>      | 20s     | 23 phases, 319 checks regression gate — the most important |
| <code>dead-code-detector</code>       | <code>tools/dead_code_detector.py</code>      | 15s     | Identifies unreferenced imports and functions              |
| <code>integrity-manifest-check</code> | <code>tools/sync_integrity_manifest.py</code> | 10s     | Verifies SHA-256 manifest hash consistency                 |
| <code>dev-health-quick</code>         | <code>tools/dev_health.py</code>              | 10s     | Quick project health check                                 |

### Standard Hooks (7 from `pre-commit/pre-commit-hooks` + 2 Python):

| Hook                                 | Source                                   | Description                                                  |
|--------------------------------------|------------------------------------------|--------------------------------------------------------------|
| <code>trailing-whitespace</code>     | <code>pre-commit/pre-commit-hooks</code> | Removal of trailing whitespace                               |
| <code>end-of-file-fixer</code>       | <code>pre-commit/pre-commit-hooks</code> | Required final newline                                       |
| <code>check-yaml</code>              | <code>pre-commit/pre-commit-hooks</code> | YAML syntax validation                                       |
| <code>check-json</code>              | <code>pre-commit/pre-commit-hooks</code> | JSON syntax validation                                       |
| <code>check-added-large-files</code> | <code>pre-commit/pre-commit-hooks</code> | Blocks files > 1MB (prevents accidental <code>.pt</code> )   |
| <code>check-merge-conflict</code>    | <code>pre-commit/pre-commit-hooks</code> | Detects unresolved merge conflict markers                    |
| <code>detect-private-key</code>      | <code>pre-commit/pre-commit-hooks</code> | Blocks commits of private keys                               |
| <code>black</code>                   | <code>psf/black</code>                   | Automatic Python formatting (line length 100, target py3.12) |
| <code>isort</code>                   | <code>pycqa/isort</code>                 | Automatic import sorting (black profile, line length 100)    |



## 12.21 Build, Packaging, and Deployment

**Key files:** `packaging/windows_installer.iss` , `setup_new_pc.bat` , `export_env.bat`

The project includes a build and packaging system for distributing the desktop application on Windows.

### 3 Requirements Files:

| File                               | Purpose                                              | Dependencies                                                                |
|------------------------------------|------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>requirements.txt</code>      | <b>Base</b> — main dependencies for execution        | 28 packages (torch, kivy, sqlalchemy, demoparser2, playwright, httpx, etc.) |
| <code>requirements-ci.txt</code>   | <b>CI/CD</b> — adds test and analysis tools          | pytest, coverage, mypy, black, isort, pre-commit                            |
| <code>requirements-lock.txt</code> | <b>Lock</b> — exact versions for reproducible builds | Exact pin of each dependency and sub-dependency                             |

### Windows Packaging ( `packaging/windows_installer.iss` ):

- Inno Setup installer with graphical wizard
- Custom CS2 Analyzer icon
- Desktop and Start menu shortcut
- Estimated size: ~500MB (includes PyTorch and models)

- Target: Windows 10/11 64-bit

### Setup Scripts:

| Script                                  | Purpose                                                         |
|-----------------------------------------|-----------------------------------------------------------------|
| <code>setup_new_pc.bat</code>           | Environment setup from scratch: Python, pip, venv, dependencies |
| <code>export_env.bat</code>             | Export of required environment variables                        |
| <code>run_full_training_cycle.py</code> | Full ML training cycle orchestration                            |

## 12.22 Alembic Migration System

**Directory:** `alembic/` (root) + `Programma_CS2_RENAN/backend/storage/migrations/`

The project uses **two separate Alembic setups** to manage database schema migrations in an organized way.

### Alembic Root ( `alembic/` , 13 versions):

Contains the main schema migrations, from initial table creation to the most recent changes. Each migration:

- Has a unique hash as identifier
- Contains `upgrade()` (applies change) and `downgrade()` (reverts change)
- Is idempotent — can be reapplied without errors
- Is tested on a production-like schema before deploy

### Backend Migrations ( `backend/storage/migrations/` , 2 versions):

Migrations specific to the storage layer, kept separate for modularity. They handle changes to tables that are the exclusive responsibility of the `backend/storage/` module.

**Automatic execution:** Migrations are executed automatically during phase 3 of the startup sequence ( `main.py` ), ensuring the schema is always up to date before any DB operation.

## 12.23 Main Ingestion Orchestrator ( `run_ingestion.py` )

**File:** `Programma_CS2_RENAN/run_ingestion.py`

`run_ingestion.py` is the **orchestrating heart** of the entire ingestion pipeline. It is the largest file dedicated to ingestion and coordinates all phases from demo discovery to persisting results in the database.

### 12+ main functions:

| Function                                  | Role                                                                |
|-------------------------------------------|---------------------------------------------------------------------|
| <code>discover_demos()</code>             | Scans configured directories, filters valid <code>.dem</code> files |
| <code>validate_demo_file()</code>         | Checks magic bytes, size, pre-parsing integrity                     |
| <code>process_single_demo()</code>        | Complete pipeline for a demo: parse → extract → enrich → persist    |
| <code>batch_process()</code>              | Processes multiple demos in sequence with progress tracking         |
| <code>enrich_round_stats()</code>         | Post-processing: computes trade kills, blind kills, flash assists   |
| <code>compute_hltv_rating()</code>        | Computes HLTV 2.0 rating for each player                            |
| <code>assign_dataset_split()</code>       | Assigns 70% train / 15% val / 15% test with temporal split          |
| <code>generate_coaching_insights()</code> | Invokes CoachingService to generate 5-20 insights                   |
| <code>update_ingestion_task()</code>      | Updates task status in DB (queued → processing → completed/failed)  |
| <code>cleanup_failed_tasks()</code>       | Cleans up failed tasks, resets to queued if recoverable             |
| <code>report_progress()</code>            | Structured logging of overall progress                              |
| <code>handle_duplicate_detection()</code> | Verifies via Registry whether the demo has already been processed   |

**ResourceManager** ( `ingestion/resource_manager.py` ):

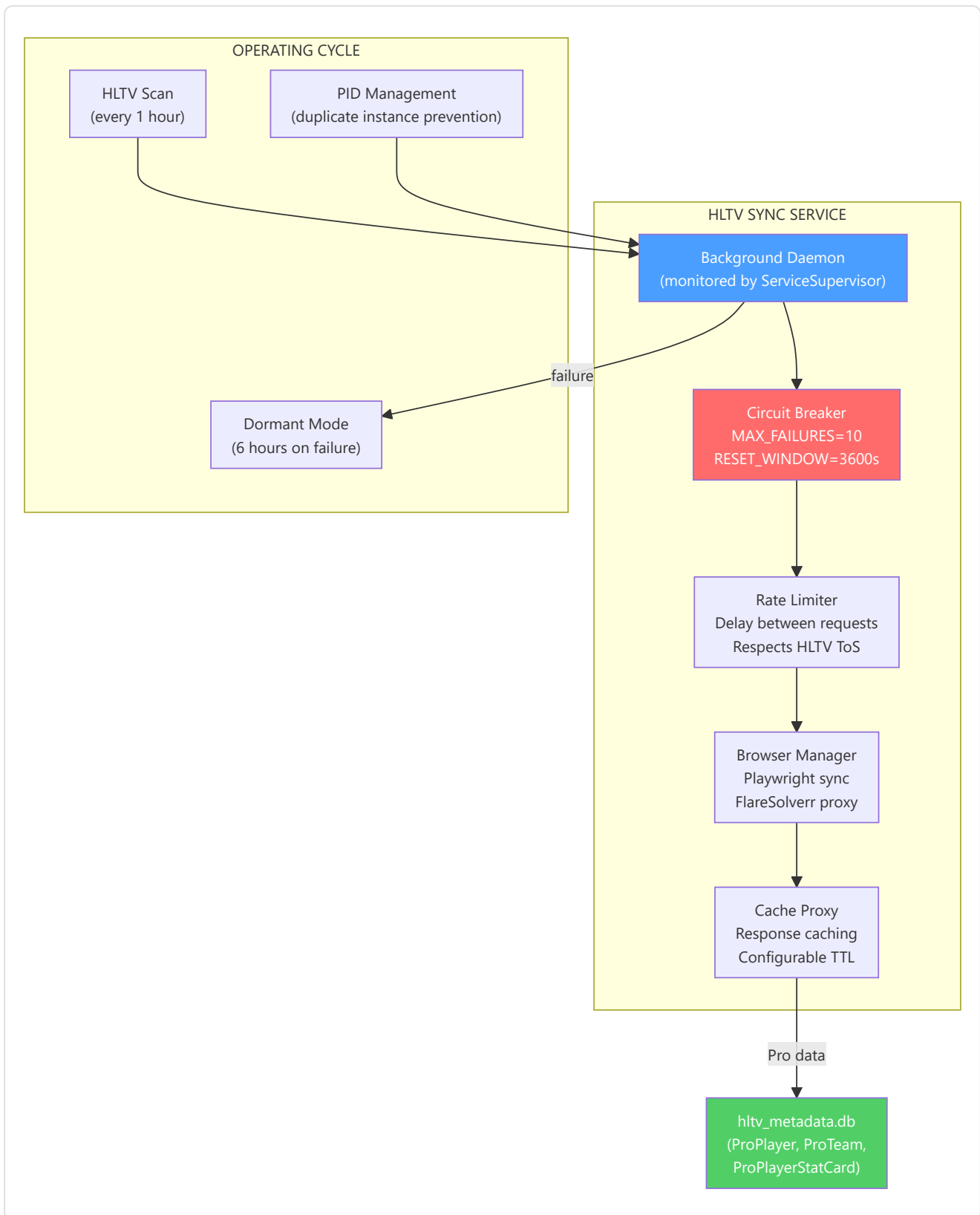
The ResourceManager manages **hardware resources** during ingestion to avoid system overload:

| Parameter      | Default Value | Purpose                                          |
|----------------|---------------|--------------------------------------------------|
| CPU threshold  | 80%           | Pauses ingestion if CPU exceeds this threshold   |
| RAM threshold  | 85%           | Pauses ingestion if RAM exceeds this threshold   |
| Disk check     | Yes           | Verifies sufficient disk space before processing |
| Throttle delay | 2s            | Delay between consecutive tasks for cooldown     |

## 12.24 HLTV Sync Service and Background Daemon

**File:** `Programma_CS2_RENAN/hltv_sync_service.py` **Related files:** `backend/data_sources/hltv/` , `backend/services/telemetry_client.py`

The HLTV Sync Service is a **background daemon** that automatically synchronizes pro player data from HLTV.org. It operates as a service monitored by the Console's `ServiceSupervisor` .



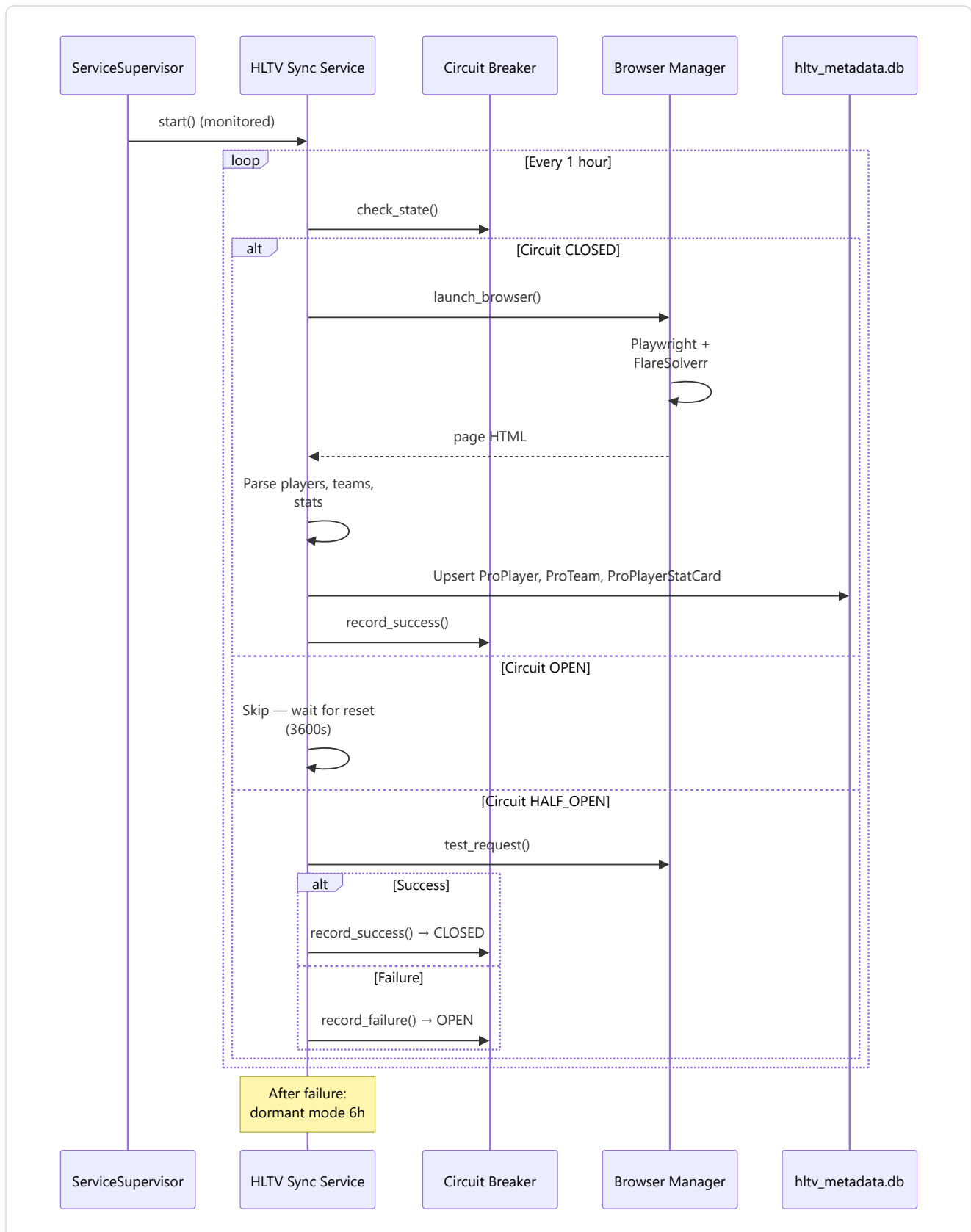
| Component             | File                            | Role                                                                                   |
|-----------------------|---------------------------------|----------------------------------------------------------------------------------------|
| <b>CircuitBreaker</b> | <code>circuit_breaker.py</code> | Protects against cascade failure: CLOSED→OPEN (after 10 fails)→HALF_OPEN (test)→CLOSED |
| <b>BrowserManager</b> | <code>browser_manager.py</code> | Headless Playwright browser management with FlareSolverr for Cloudflare bypass         |
| <b>CacheProxy</b>     | <code>cache_proxy.py</code>     | Local cache of HLTV responses to reduce requests                                       |
| <b>RateLimiter</b>    | <code>rate_limiter.py</code>    | Configurable delay between requests to respect ToS                                     |

### HLTV Internal Architecture — Collectors and Selectors:

The `ingestion/hltv/` tree also contains specialized modules for data collection:

| Module                           | Type      | Description                                          |
|----------------------------------|-----------|------------------------------------------------------|
| <code>player_collector.py</code> | Collector | Collects pro player profiles from HLTV pages         |
| <code>team_collector.py</code>   | Collector | Collects rosters and team statistics                 |
| <code>match_collector.py</code>  | Collector | Collects match results and demo links                |
| <code>stat_selector.py</code>    | Selector  | Extracts specific statistics from parsed HTML pages  |
| <code>demo_selector.py</code>    | Selector  | Identifies and downloads demo links from match pages |

### Detailed operating cycle:



## 12.25 RASP Guard — Runtime Code Integrity

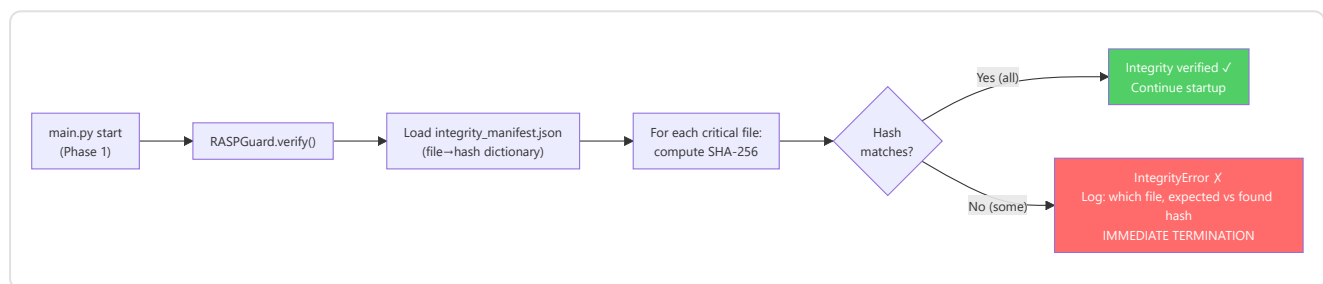
**File:** `Programma_CS2_RENAN/observability/rasp.py` **Data file:** `data/integrity_manifest.json`

The RASP (Runtime Application Self-Protection) Guard is the **first check** executed on application startup (Phase 1 of the boot sequence). It verifies that no source file has been modified compared to the integrity manifest.

### RASP Guard Components:

| Component                               | Description                                                        |
|-----------------------------------------|--------------------------------------------------------------------|
| <code>RASPGuard</code>                  | Main class — loads the manifest, verifies hashes, raises exception |
| <code>integrity_manifest.json</code>    | JSON file with SHA-256 hashes of each critical source file         |
| <code>IntegrityError</code>             | Custom exception — immediate termination with detailed log         |
| <code>sync_integrity_manifest.py</code> | Tool to update the manifest after legitimate changes               |

### Verification flow:



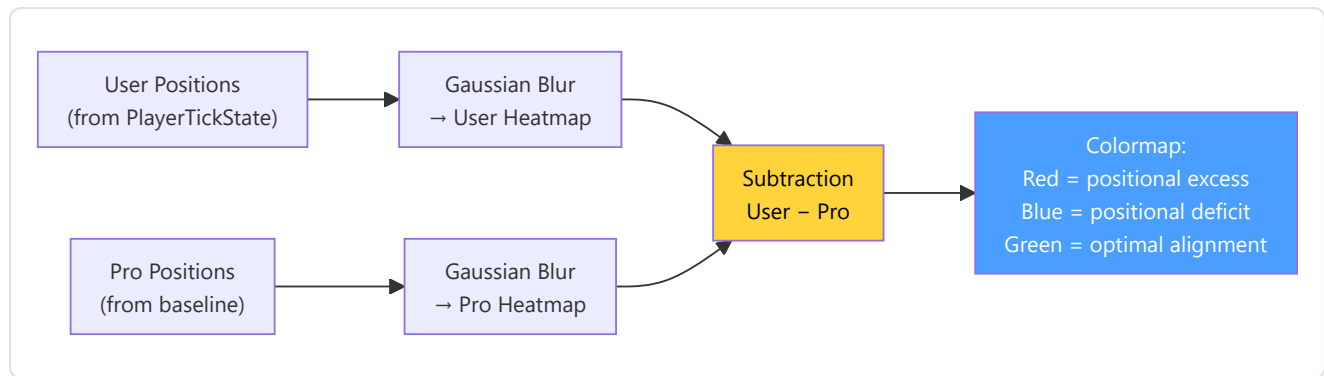
## 12.26 MatchVisualizer — Advanced Rendering

**File:** `Programma_CS2_RENAN/reporting/visualizer.py`

The `MatchVisualizer` extends the reporting system (section 12.13) with **6 specialized rendering methods** for generating charts and visualizations.

### 6 Rendering methods:

| Method                                     | Output                 | Description                                                                          |
|--------------------------------------------|------------------------|--------------------------------------------------------------------------------------|
| <code>render_skill_radar()</code>          | PNG radar chart        | 5-axis pentagon (Mechanics, Positioning, Utility, Timing, Decision) with pro overlay |
| <code>render_trend_chart()</code>          | PNG line chart         | Historical trend of key metrics (KPR, ADR, KAST) over N matches                      |
| <code>render_heatmap()</code>              | PNG heatmap            | 2D Gaussian occupancy on tactical map                                                |
| <code>render_differential_heatmap()</code> | PNG diff heatmap       | user-pro subtraction: red=too much, blue=too little, green=optimal                   |
| <code>render_critical_moments()</code>     | PNG annotated timeline | Timeline with color markers for key moments (kills, deaths, plants)                  |
| <code>render_comparison_table()</code>     | PNG/HTML table         | user vs pro comparison table with percentage delta for each metric                   |

**Differential heatmap — algorithm:****12.27 Runtime Data and Configuration Files**

The project includes several data files that are read or written during execution. These files are not Python code but are essential for the system to function.

**data/map\_config.json** — CS2 map configuration:

Contains the coordinate transformation parameters for each supported map (Dust2, Mirage, Inferno, Nuke, Vertigo, Overpass, Ancient, Anubis):

```

{
  "de_dust2": {
    "pos_x": -2476, "pos_y": 3239, "scale": 4.4,
    "z_cutoff": null, "level": "single"
  },
  "de_nuke": {
    "pos_x": -3453, "pos_y": 2887, "scale": 7.0,
    "z_cutoff": -495, "level": "multi"
  }
}

```

**data/integrity\_manifest.json** — RASP integrity manifest:

Dictionary `{file_path: sha256_hash}` for all critical Python files. Generated/updated by `tools/sync_integrity_manifest.py`. Verified on startup by `RASPGuard` (section 12.25).

**data/training\_progress.json** — Training progress:

Persists training state between restarts: current epoch, best loss, learning rate, last processed demo. Consulted by the Teacher daemon to resume training from the exact point where it stopped.

**user\_settings.json** — User settings:

Level 2 configuration file (cf. section 12.3). Saved in the project directory, contains customizable preferences: demo paths, UI theme, language, feature flags.

## 12.28 Root-Level Entry Points

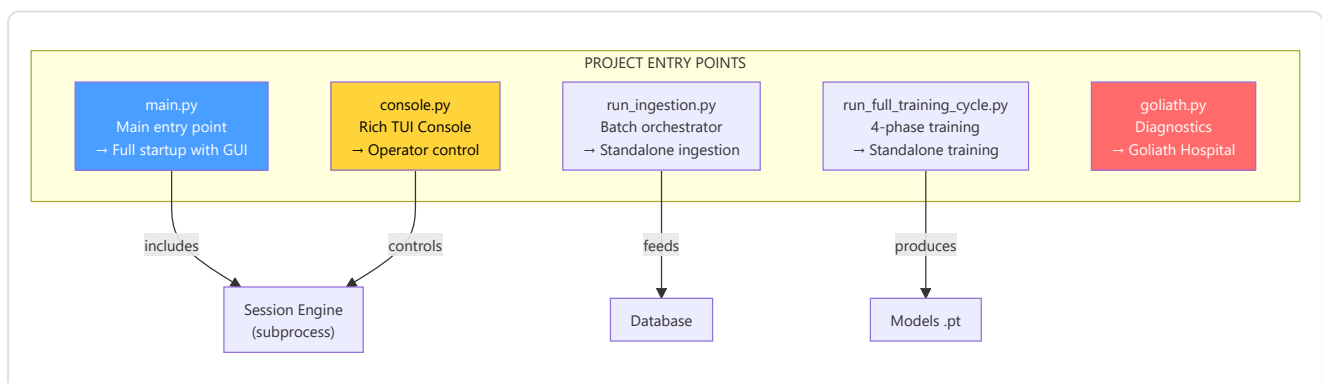
Besides `main.py`, the project includes several **executable scripts** at the root level that serve as alternative entry points for specific operations:

| Script                                  | Lines | Purpose                                                                                                              | Invocation                                     |
|-----------------------------------------|-------|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| <code>console.py</code>                 | ~61KB | Complete interactive TUI console with Rich — command registry, CLI and interactive modes, complete system management | <code>python console.py</code>                 |
| <code>run_ingestion.py</code>           | 1,057 | Standalone ingestion orchestrator (cf. 12.23) — batch demo processing                                                | <code>python run_ingestion.py [path]</code>    |
| <code>goliath.py</code>                 | ~200  | Launcher for Goliath Hospital diagnostics (cf. 12.17)                                                                | <code>python goliath.py</code>                 |
| <code>schema.py</code>                  | ~100  | Generation and visualization of the current DB schema                                                                | <code>python schema.py</code>                  |
| <code>run_full_training_cycle.py</code> | ~150  | Standalone 4-phase complete training (JEPA→Pro→User→RAP)                                                             | <code>python run_full_training_cycle.py</code> |
| <code>hflayers.py</code>                | ~50   | Utility for Hopfield layers (NCPs integration)                                                                       | Import only                                    |

**Interactive console** ( `console.py` , ~61KB):

The console is the **most powerful entry point** for advanced operators. It offers a Rich-based TUI (Terminal User Interface) with:

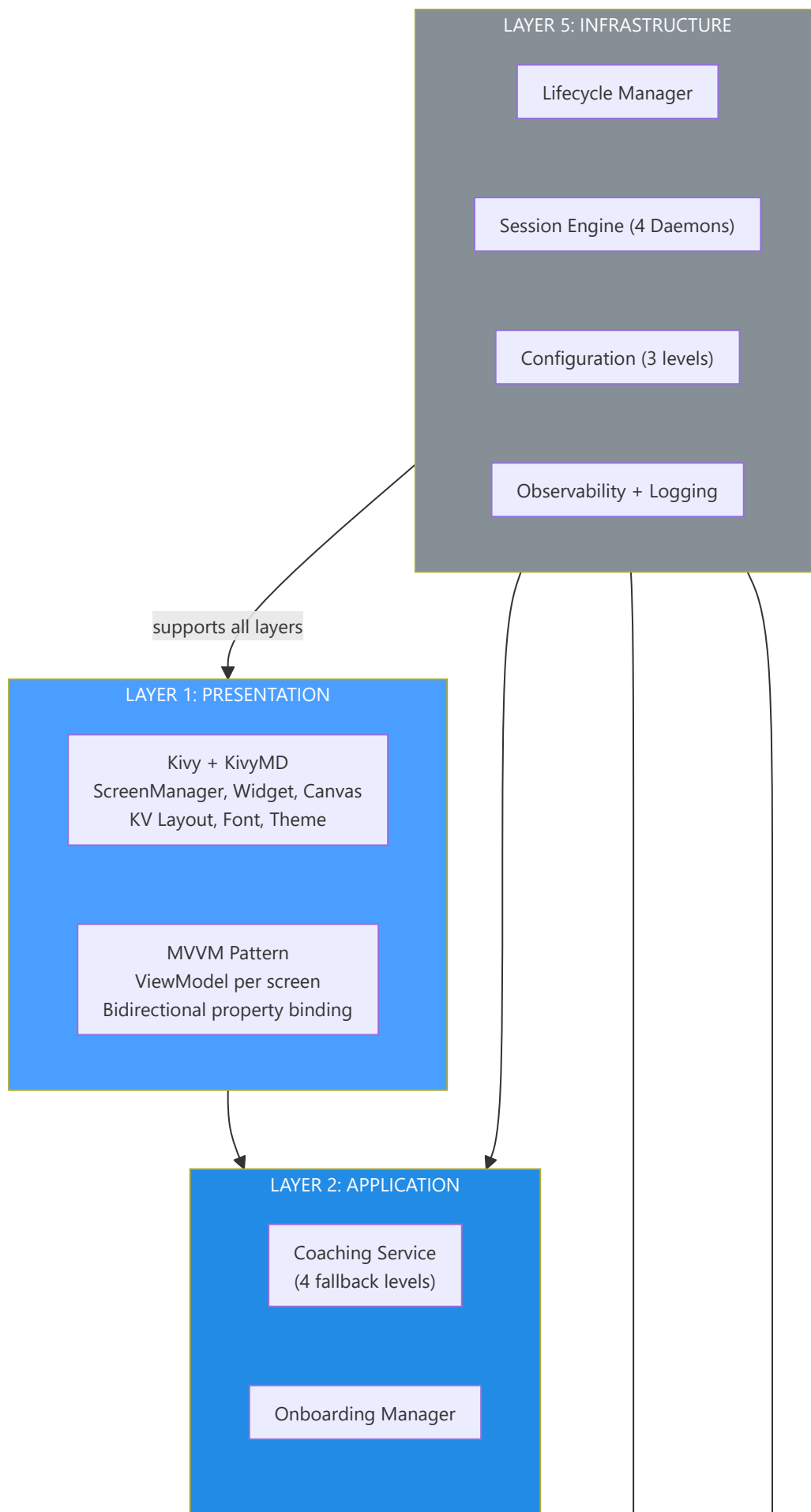
- **Command Registry:** System of registered commands with autocompletion
- **CLI Mode:** Single-shot command execution ( `console.py status` , `console.py train` )
- **Interactive Mode:** REPL shell with custom prompt and command history
- **Rich Panels:** Dashboard with color tables, progress bars, status tree view

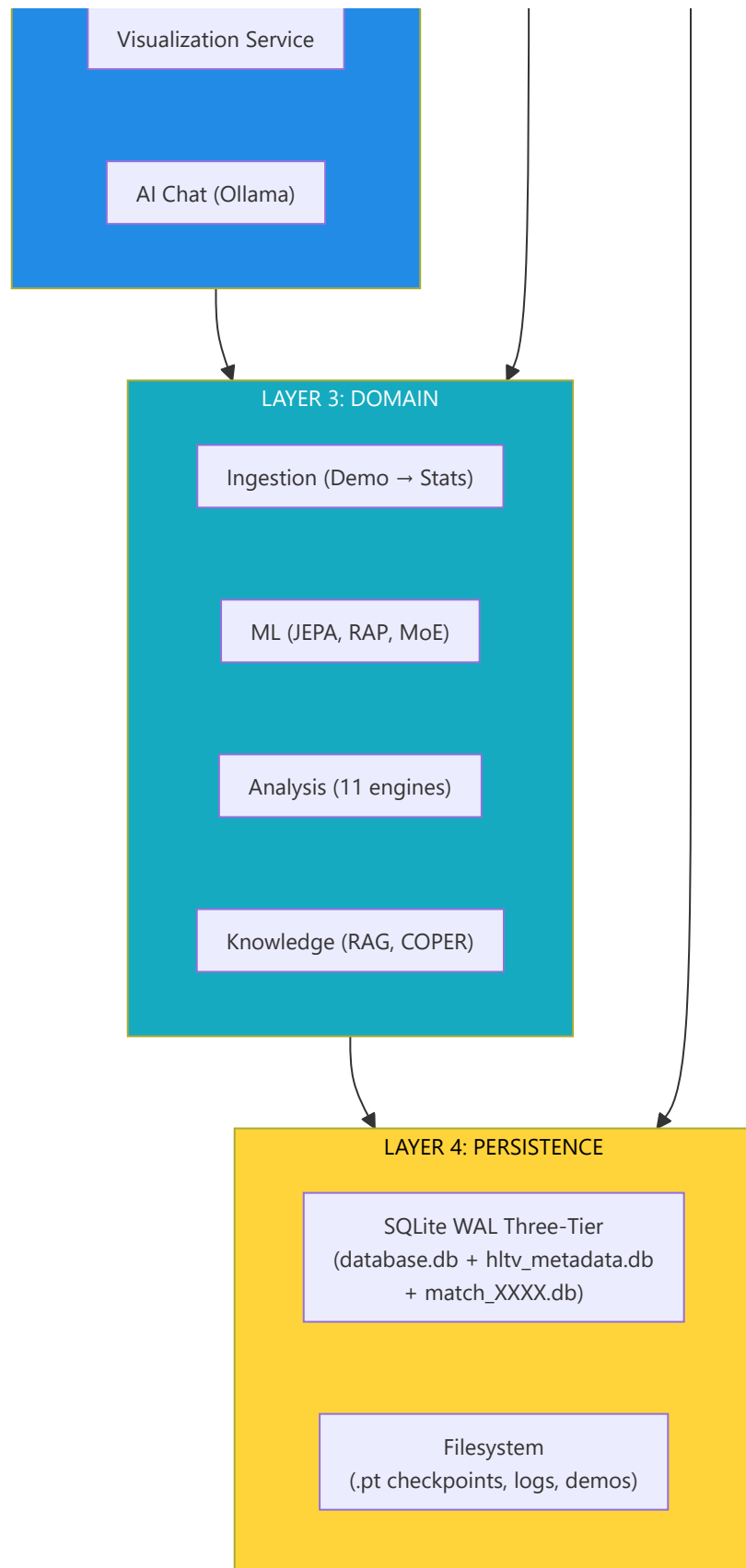


## Architectural Summary

Macena CS2 Analyzer is a **layered and modular** application organized across 5 architectural layers with 3 separate processes.







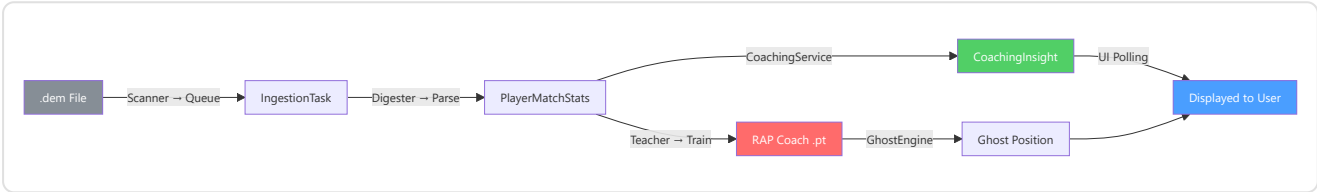
**Complete technology stack (28 dependencies):**

| Category           | Library               | Version | Role in the project                                       |
|--------------------|-----------------------|---------|-----------------------------------------------------------|
| <b>ML Core</b>     | PyTorch               | 2.x     | Neural networks (JEPA, RAP, MoE, NeuralRoleHead, WinProb) |
| <b>ML Ext</b>      | ncps                  | latest  | Liquid Time-Constant Networks (LTC) for RAP memory        |
| <b>ML Ext</b>      | hflayers              | latest  | Hopfield layers for associative attention                 |
| <b>UI</b>          | Kivy                  | 2.x     | Cross-platform UI framework                               |
| <b>UI</b>          | KivyMD                | 1.x     | Material Design widgets                                   |
| <b>DB</b>          | SQLModel              | latest  | ORM (Pydantic + SQLAlchemy)                               |
| <b>DB</b>          | SQLAlchemy            | 2.x     | Underlying database engine                                |
| <b>DB</b>          | Alembic               | 1.x     | Schema migrations                                         |
| <b>HTTP</b>        | requests              | 2.x     | Steam/FACEIT API (synchronous)                            |
| <b>HTTP</b>        | httpx                 | latest  | Telemetry (asynchronous)                                  |
| <b>Scraping</b>    | playwright            | latest  | Headless browser for HLTV                                 |
| <b>Parsing</b>     | demoparser2           | latest  | CS2 demo parser (Rust-based, fast)                        |
| <b>Data</b>        | pandas                | 2.x     | Tabular data manipulation                                 |
| <b>Data</b>        | numpy                 | 1.x     | Numerical computations                                    |
| <b>Viz</b>         | matplotlib            | 3.x     | Charts and visualizations                                 |
| <b>NLP</b>         | sentence-transformers | latest  | 384-dim embeddings for RAG                                |
| <b>TUI</b>         | rich                  | latest  | Color console, tables, progress bars                      |
| <b>Logging</b>     | logging (stdlib)      | —       | Structured logging                                        |
| <b>Security</b>    | keyring               | latest  | Secure management of API keys                             |
| <b>Monitoring</b>  | sentry-sdk            | latest  | Remote error tracking (optional)                          |
| <b>Testing</b>     | pytest                | latest  | Test framework                                            |
| <b>Formatting</b>  | black                 | latest  | Code formatting                                           |
| <b>Import</b>      | isort                 | latest  | Import sorting                                            |
| <b>QA</b>          | pre-commit            | latest  | Pre-commit hooks                                          |
| <b>Crypto</b>      | hashlib (stdlib)      | —       | SHA-256 for RASP manifest                                 |
| <b>Concurrency</b> | threading (stdlib)    | —       | Daemon threads, MLControlContext                          |
| <b>IPC</b>         | subprocess (stdlib)   | —       | Session Engine as subprocess                              |
| <b>Config</b>      | json (stdlib)         | —       | user_settings.json, map_config.json                       |

### The 3 application processes:

| Process           | Type                        | Responsibility                         | Communication                  |
|-------------------|-----------------------------|----------------------------------------|--------------------------------|
| Main              | Kivy GUI                    | User interface, rendering, interaction | DB polling every 10-15s        |
| Daemon            | Subprocess (Session Engine) | Scanner, Digester, Teacher, Pulse      | stdin pipe (IPC) + shared DB   |
| Optional Services | External processes          | HLTV sync, local Ollama LLM            | HTTP/API + Console supervision |

Main data flows:



Remediation Note — Eliminated Code (G-06)

During the 12-phase remediation process (370+ issues resolved overall — cf. section 12.19), the `backend/nn/advanced/` directory was **completely eliminated** because it contained unreferenced dead code:

- `superposition_net.py` — An experimental superposition network never integrated into training or inference flow. Active superposition functionality is implemented in `layers/superposition.py` (used by the RAP Strategy layer).
- `brain_bridge.py` — An experimental model bridge never called by any module.
- `feature_engineering.py` — Duplicate feature engineering; the canonical version resides in `backend/processing/feature_engineering/`.

**Rationale (G-06):** Keeping dead code creates risks of confusion (which `superposition` is the real one?), increases maintenance surface, and can introduce accidental imports. The elimination was verified through static dependency analysis: no file in the project imported from `backend/nn/advanced/`.

Shared Utility — `round_utils.py`

**File:** `Programma_CS2_RENAN/backend/knowledge/round_utils.py`

The function `infer_round_phase(equipment_value)` is a **shared utility** used by both the coaching service and the knowledge system to classify the economic phase of a round. It resides in `backend/knowledge/` and is imported by modules in `services/` and `processing/`.

| Equipment value   | Returned phase |
|-------------------|----------------|
| < \$1,500         | "pistol"       |
| \$1,500 – \$2,999 | "eco"          |
| \$3,000 – \$3,999 | "force"        |
| ≥ \$4,000         | "full_buy"     |

## Core Utilities — Lock Files and Platform

**Lock Files** ( `core/lock_files.py` ):

Lock file system for concurrency between D-track / HLTV-track processes. Uses a local `.locks/` directory in the repository (survives sessions, not restarts).

| Component                                          | Description                                                                                                        |
|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>Lock file format</b>                            | <code>&lt;pid&gt; &lt;iso_timestamp&gt;</code> — identifies the owning process                                     |
| <b>Stale lock</b>                                  | Automatically recovered if the owning PID is dead ( <code>os.kill(pid, 0)</code> )                                 |
| <code>LockConflict(RuntimeError)</code>            | Raised when the lock is held by a live process                                                                     |
| <code>acquire(name) → Path</code>                  | Creates lock, checks conflicts, recovers stale, writes PID + timestamp                                             |
| <code>release(name)</code>                         | Removes lock file. Idempotent                                                                                      |
| <code>lock(name)</code> ( <b>context manager</b> ) | Acquire on entry, release on exit                                                                                  |
| <b>Signal handlers</b>                             | <code>install_signal_handlers()</code> registers SIGTERM/SIGINT handlers that release all locks before termination |
| <code>_held_locks: Set[str]</code>                 | Module-level state tracking currently held locks                                                                   |

**Platform Utilities** ( `core/platform_utils.py` ):

Platform detection and available drives for cross-platform storage operations.

| Function                                        | Description                                                                                                                                                                                                                           |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>_get_platform() → str</code>              | Returns <code>"win"</code> , <code>"macosx"</code> , <code>"linux"</code> or raw <code>sys.platform</code>                                                                                                                            |
| <code>platform</code> (module-level constant)   | Result of <code>_get_platform()</code> evaluated at import                                                                                                                                                                            |
| <code>get_available_drives() → List[str]</code> | <b>PU-02:</b> Windows → <code>_get_windows_drives()</code> with ctypes bitmask <code>GetLogicalDrives()</code> + fallback <code>psutil.disk_partitions()</code> ; Linux/macOS → <code>["/"]</code> ; Other → validated home directory |

## Architectural Strengths

1. **25-dimensional unified feature contract** — `METADATA_DIM = 25` enforces system-wide training/inference parity.
2. **3-level maturity gating** — Prevents premature model deployment with insufficient data.
3. **4-level coaching fallback** — COPER → Hybrid → RAG → Base guarantees insights are always delivered.
4. **Multi-model diversity** — JEPA, VL-JEPA, LSTM+MoE, RAP, and NeuralRoleHead contribute complementary inductive biases.
5. **Temporal split** — Prevents data leakage by ensuring chronological ordering.
6. **COPER feedback loop** — EMA-based effectiveness monitoring with decay of stale experience.
7. **Phase 6 analysis suite** — 11 analysis engines (role, win probability, game tree, belief, deception, momentum, entropy, blind spots, utility and economy, engagement distance, movement quality).
8. **Threshold persistence** — Role thresholds survive restarts via the DB table `RoleThresholdRecord`.
9. **Configurable heuristics** — `HeuristicConfig` externalizes normalization bounds to JSON.
10. **LLM polishing** — Optional Ollama integration for natural language coaching narratives.
11. **Training Observatory** — 4-level introspection (Callback, TensorBoard, Maturity State Machine, Embedding Projector) with zero impact when disabled and callbacks isolated from errors.
12. **Neural Role Consensus** — Dual heuristic + NeuralRoleHead MLP classification with cold-start protection, guaranteeing reliable role assignment even with partial data.
13. **Per-Round Statistical Isolation** — The `RoundStats` model prevents contamination across rounds, enabling granular round-level coaching and per-round HLTV 2.0 evaluations.
14. **Quad-Daemon Architecture** — Complete separation between GUI and heavy work, with coordinated shutdown and automatic zombie task cleanup.
15. **Pervasive graceful degradation** — Every component has a fallback plan: the system never crashes, it always degrades in a controlled way.
16. **Three-Tier Storage Architecture** — Separation of `database.db` (core + knowledge, 18 tables), `hltnv_metadata.db` (pro data, 3 tables), and `match_data/{id}.db` (per-match telemetry) to eliminate WAL contention and prevent uncontrolled monolith growth.
17. **Live Bayesian Calibration (G-07)** — The death estimator self-calibrates with `extract_death_events_from_db()` → `auto_calibrate()`, transforming from a static model into an adaptive system.
18. **Live Training Control (MLControlContext)** — Real-time pause/resume/stop/throttle of training via `threading.Event`, with custom exception `TrainingStopRequested` instead of `StopIteration`.
19. **Resilient Circuit Breaker** — `_CircuitBreaker` for external APIs (HLTV) with MAX\_FAILURES=10, RESET\_WINDOW\_S=3600, prevents cascade failure with CLOSED→OPEN→HALF\_OPEN pattern.
20. **5-Level Validation Pyramid** — Headless Validator → pytest → Backend Validator → Goliath Hospital → Brain Verify: each level progressively deeper, from quick smoke test (10s) to complete audit of 118 rules.

21. **RASP Guard** — Runtime Application Self-Protection with SHA-256 manifest: verifies source code integrity on startup, prevents accidental and intentional tampering.
22. **13-Hook Pre-commit Gate** — 4 custom hooks + 7 standard + 2 Python prevent non-compliant code from reaching the repository. Formatting, validation, dead code, integrity, merge conflict, and private key verified automatically.
23. **Hardware-Aware ResourceManager** — Automatic CPU/RAM throttling during ingestion: the system autonomously slows down if hardware resources are under pressure, without human intervention.
24. **Test Forensics** — 10 post-mortem investigation scripts to diagnose training failure, weight anomalies, coaching path traces, and DB consistency — the system's "black box".



## ARCHITECTURAL STRENGTHS — THE 24 PILLARS

1. Unified 25-dim contract -  
Everyone speaks the same language

2. 3-level maturity gate - No  
premature release

3. 4-level coaching fallback - Never  
empty-handed

4. Multi-model diversity - 5 brains >  
1 brain

5. Temporal split - No time-travel  
cheating

6. COPER feedback loop - Learns  
from its own advice

7. Phase 6 Analysis (10 eng.) - 10  
specialized detectives

8. Threshold persistence - Survives  
restarts

9. Configurable heuristics - JSON  
override

10. LLM polish (Ollama) - Advice sounds natural

11. Training Observatory - Report card for the brain

12. Neural Role Consensus - Two teachers compare notes

13. Per-Round isolation - Evaluates every question, not just the test

14. Quad-Daemon architecture - Responsive GUI, heavy work in background

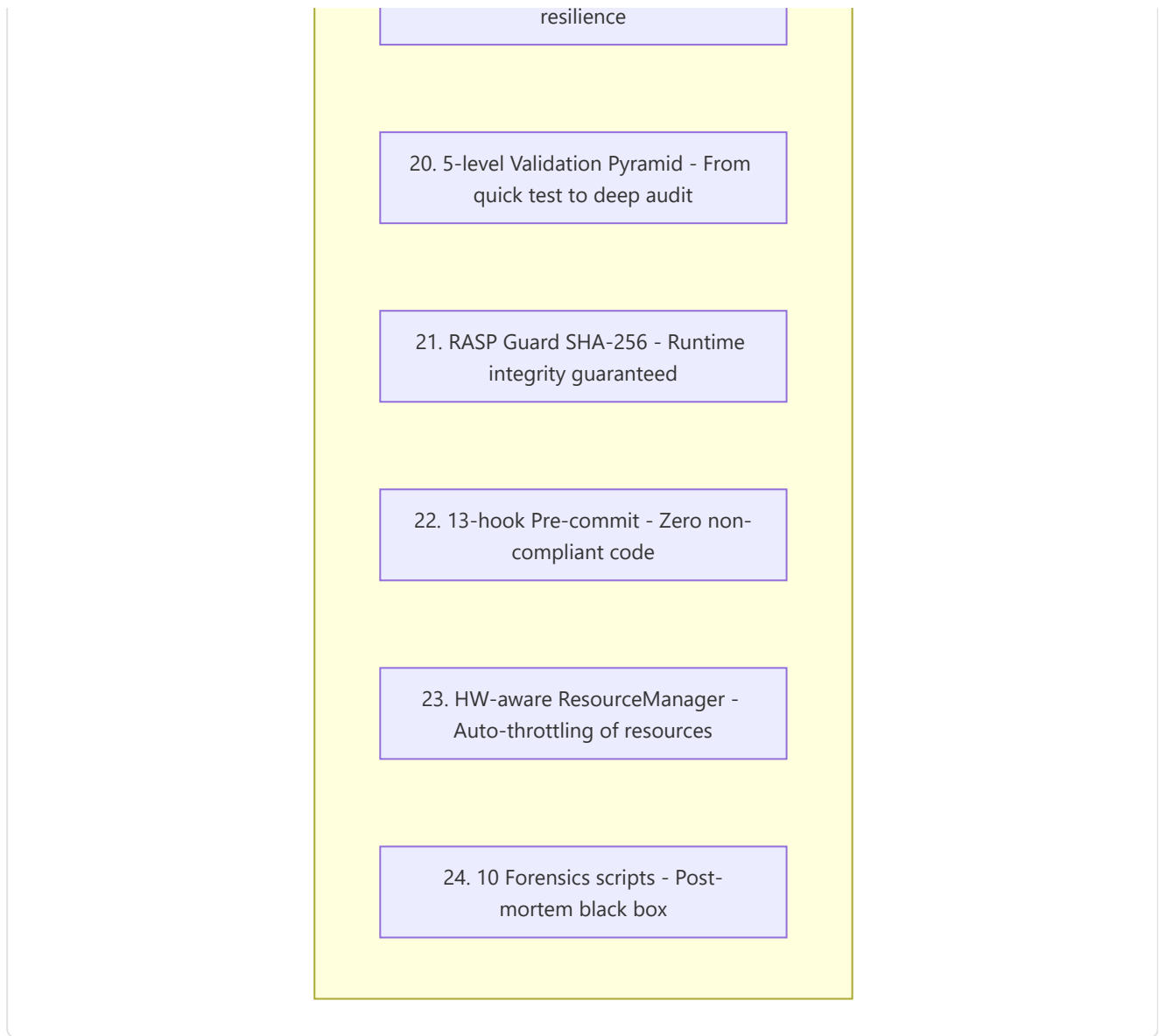
15. Graceful degradation - The system never crashes

16. Three-Tier Storage - No contention between write and read

17. Live Bayesian calibration - Self-calibrates with the data

18. Live Training control - Pause/Stop without loss

19. Circuit Breaker - External API



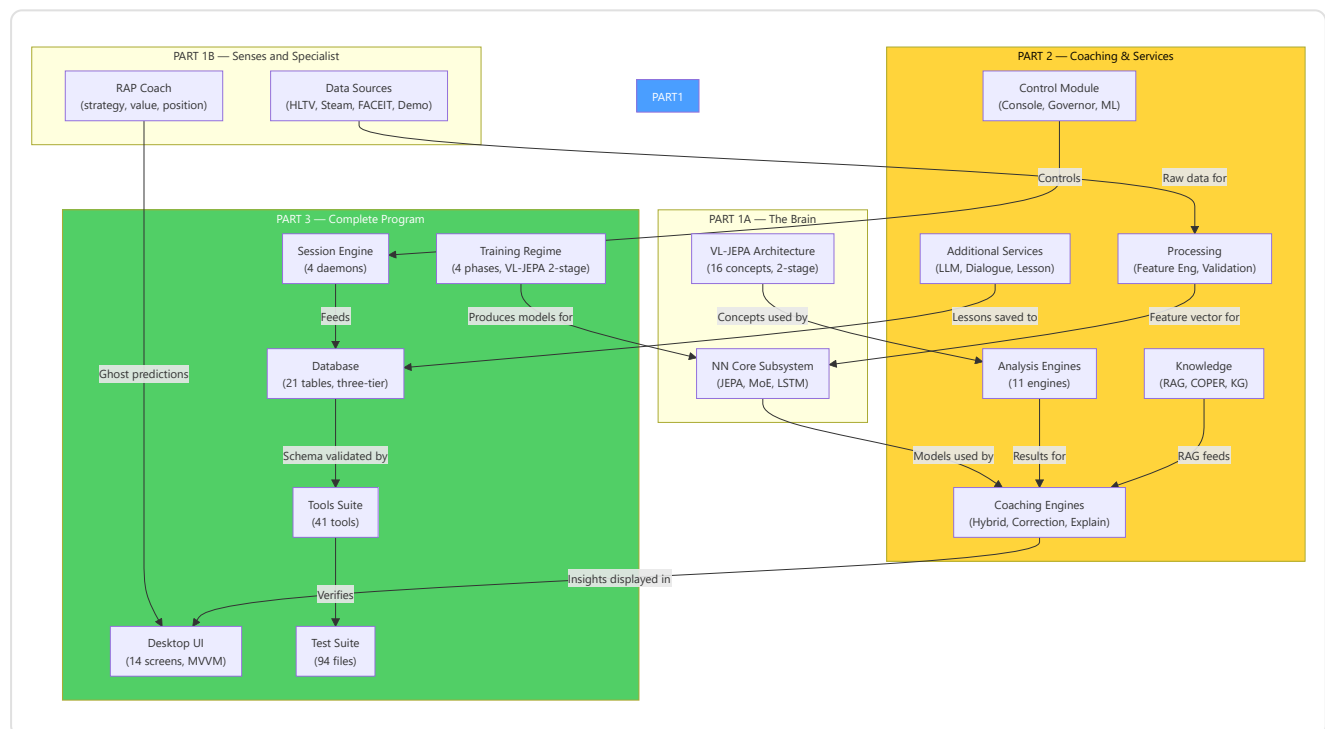
## End of document — Complete guide to Macena CS2 Analyzer

Total `.py` files in the project: **~406** (in `Programma_CS2_RENAN/`, ~480 project-wide) Total verified Python lines of code: **≈ 102,000+** AI subsystems covered: **8** (NN Core, VL-JEPA, RAP Coach, Coaching Services, Coaching Engines, Knowledge, Analysis, Processing + Training Observatory) Program subsystems covered: **18** (Startup, Lifecycle, Configuration, Session Engine, Desktop UI, Ingestion, Storage, Observability, Control Console, RASP Guard, HLTV Sync, Ingestion Orchestrator, ResourceManager, Tools Suite, Test Suite, Pre-commit, Build/Packaging, Alembic Migrations) Documented models: **6** (AdvancedCoachNN/TeacherRefinementNN, JEPA, VL-JEPA, RAPCoachModel, NeuralRoleHead, WinProbabilityNN) Documented analysis engines: **10** (Role, WinProb, GameTree, Belief, Deception, Momentum, Entropy, Blind Spots, Utility and Economy, Engagement Distance) Documented coaching engines: **7** (HybridEngine, CorrectionEngine, ExplainabilityGenerator, NNRefinement, ProBridge, TokenResolver, LongitudinalEngine) Additional services documented: **7** (CoachingDialogue, LessonGenerator, LLMService, VisualizationService, ProfileService, AnalysisService, TelemetryClient) Database tables documented: **21** (distributed across three-tier storage architecture: `database.db`, `hlty_metadata.db`, `match_data/{id}.db`) UI screens documented: **14** (Wizard, Home, Coach, Tactical

Viewer, Settings, Help, Match History, Match Detail, Performance, User Profile, Profile, Pro Comparison, Steam Config, FACEIT Config) — Qt/PySide6 (primary) + Kivy/KivyMD (legacy) Documented daemons: **4** (Scanner, Digester, Teacher, Pulse) Documented validation tools: **41** (Headless Validator, Brain Verify, Goliath Hospital, DB Inspector, Demo Inspector, ML Coach Debugger, Backend Validator, Dead Code Detector, Dev Health, Feature Audit, etc.) Documented test files: **94** (+ conftest.py, 10 forensics, 15 verification scripts) Headless validator phases: **23** (319 automated checks) Documented pre-commit hooks: **13** (4 custom local + 7 standard + 2 Python) Architectural pillars: **24** (including Three-Tier Storage, Live Bayesian Calibration, Live Training Control, Circuit Breaker, Validation Pyramid, RASP Guard, Pre-commit Gate, HW-aware ResourceManager, Forensics) Issues resolved through remediation: **368** (in 12 systematic phases) Documented remediation phases: **12** (with G-XX and F-XX codes)

## Cross-Part Interconnection Map

The three parts of the documentation form an interconnected system. This map shows the main dependencies between sections:



### Key cross-references:

| From (Part)              | To (Part)                     | Connection                                                                                 |
|--------------------------|-------------------------------|--------------------------------------------------------------------------------------------|
| Part 1 §VL-JEPA          | Part 3 §10 (Training)         | The two-stage protocol described in Part 1 is implemented by the training regime in Part 3 |
| Part 1 §Data Sources     | Part 3 §12.6 (Ingestion)      | Data sources feed the ingestion pipeline                                                   |
| Part 2 §Coaching Engines | Part 3 §12.16 Flow 1          | Coaching engines generate the insights displayed in the UI                                 |
| Part 2 §Control Module   | Part 3 §12.4 (Session Engine) | The control module manages the 4 daemons                                                   |
| Part 2 §Processing       | Part 1 §NN Core               | The vectorizer produces the METADATA_DIM=25 consumed by the models                         |
| Part 3 §9 (Database)     | Part 2 §Knowledge             | The RAG/COPER tables are used by the knowledge system                                      |
| Part 3 §11 (Loss)        | Part 1 §VL-JEPA               | The documented loss functions implement VL-JEPA training                                   |
| Part 3 §12.17 (Tools)    | Part 3 §12.18 (Tests)         | The validation pyramid includes both tools and test suite                                  |

## Technical Glossary

| Term                   | Definition                                                                                          |
|------------------------|-----------------------------------------------------------------------------------------------------|
| <b>ADR</b>             | Average Damage per Round — average damage dealt per round                                           |
| <b>BCE</b>             | Binary Cross-Entropy — loss for binary classification                                               |
| <b>COPER</b>           | Coaching through Past Experiences and References — coaching based on past experiences               |
| <b>EMA</b>             | Exponential Moving Average — exponential moving average used for JEPA target encoder                |
| <b>HLTV 2.0</b>        | HLTV.org rating system based on KPR, survival, impact, damage                                       |
| <b>InfoNCE</b>         | Noise Contrastive Estimation — contrastive loss to align representations                            |
| <b>JEPA</b>            | Joint-Embedding Predictive Architecture — architecture that predicts target embeddings from context |
| <b>KA<sub>ST</sub></b> | Kill/Assist/Survive/Trade — percentage of rounds with positive contribution                         |
| <b>KG</b>              | Knowledge Graph — knowledge graph with entities and relationships                                   |
| <b>KPR</b>             | Kills Per Round — average kills per round                                                           |
| <b>LTC</b>             | Liquid Time-Constant — neural networks with learned time constants                                  |
| <b>METADATA_DIM</b>    | Dimension of the unified feature vector (25 in the project)                                         |
| <b>MoE</b>             | Mixture of Experts — network with specialized experts and gating                                    |
| <b>MVVM</b>            | Model-View-ViewModel — architectural pattern for UI/logic separation                                |
| <b>RAG</b>             | Retrieval-Augmented Generation — generation enriched by retrieval                                   |
| <b>RAP</b>             | Reasoning Action Planning — model for strategic coaching                                            |
| <b>RASP</b>            | Runtime Application Self-Protection — runtime application protection                                |
| <b>TUI</b>             | Terminal User Interface — user interface in the terminal                                            |
| <b>VICReg</b>          | Variance-Invariance-Covariance Regularization — regularization for embedding diversity              |
| <b>VL-JEPA</b>         | Vision-Language JEPA — extension with linguistic concepts                                           |
| <b>WAL</b>             | Write-Ahead Logging — SQLite mode for concurrent reads/writes                                       |
| <b>Z-score</b>         | Number of standard deviations from the mean — measures how far a value is from the norm             |

**Author:** Renan Augusto Macena